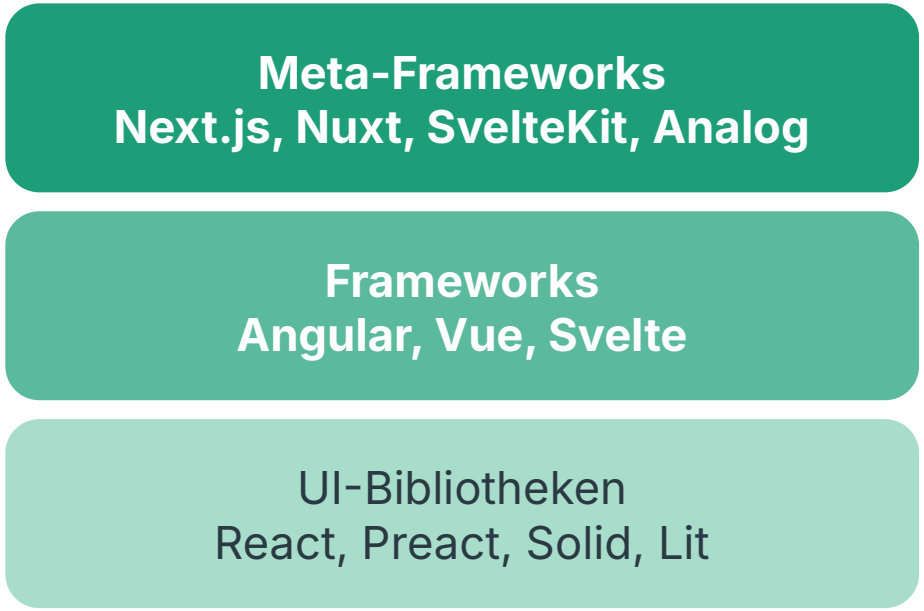
Einordnung moderner Angular-Entwicklung

Wo steht Angular im Frontend-Umfeld?

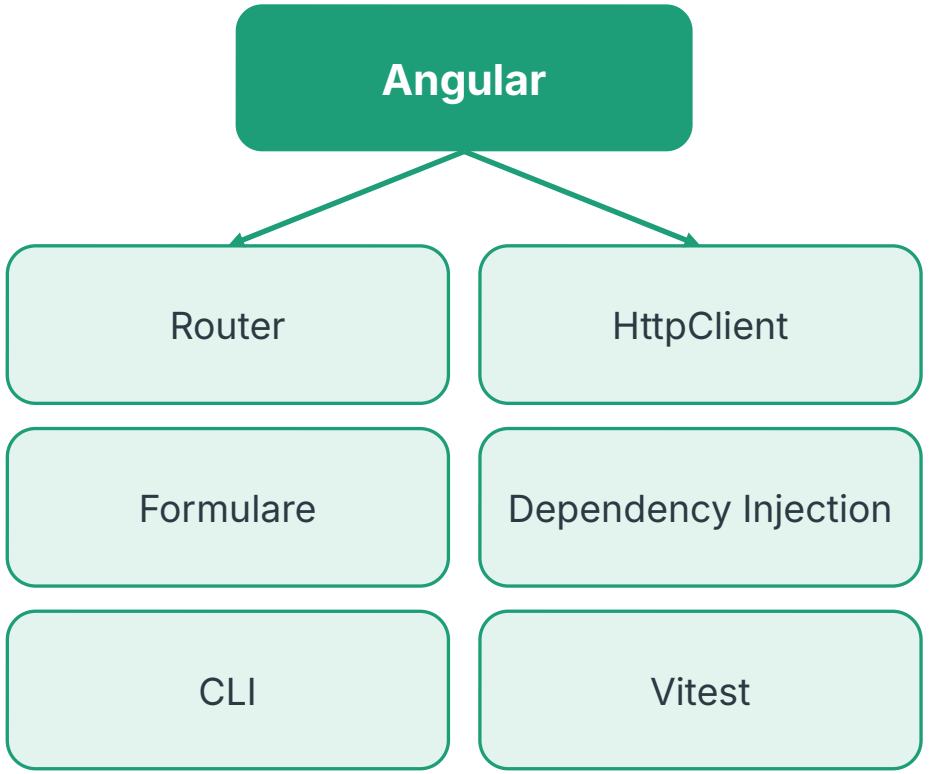
- Werkzeuge für die Oberfläche lassen sich grob in drei Ebenen einteilen:
  - UI-Bibliotheken: React, Preact, Solid, Lit – sie zeigen nur die Oberfläche an
  - Frameworks: Angular, Vue, Svelte – Oberfläche und weitere Bausteine
  - Meta-Frameworks: Next.js, Nuxt, SvelteKit, Analog – Routing, Server-Rendering, Build vorgegeben
- Angular ist ein vollständiges Framework: Router, Formulare, HTTP, Dependency Injection, CLI
- Folge: weniger Auswahl- und Integrationsaufwand, dafür mehr Konvention



Je höher, desto mehr ist vorgegeben

Batteries included: Was Angular mitbringt

- Routing: @angular/router (sonst z. B. React Router oder TanStack Router)
- HTTP: HttpClient mit Interceptors (sonst z. B. fetch oder axios)
- Formulare: Template-Driven Forms und Reactive Forms (sonst z. B. Formik)
- Zustand: Signals, Services und NgRx (sonst z. B. Redux, Zustand oder MobX)
- Abhängigkeiten: Dependency Injection (sonst z. B. Context oder Module-Singletons)
- Werkzeuge: Angular CLI mit dem Befehl ng (sonst z. B. Vite oder eigene Konfiguration)
- Tests: Vitest-Setup ab Version 21 und TestBed (sonst z. B. Jest mit Testing Library)



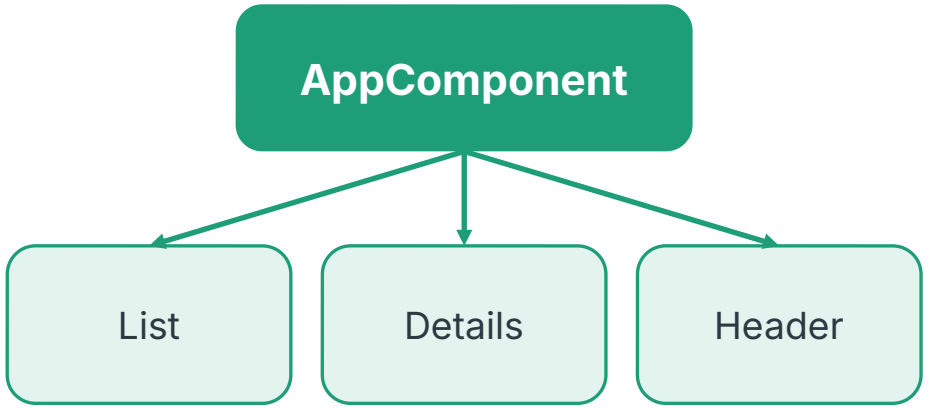
Einsatzgebiete: Wann passt Angular, wann nicht?

- Passt gut: große, langlebige Unternehmensanwendungen mit vielen Teams
- Passt gut: Dashboards und formularlastige Fachanwendungen mit hohen Qualitätsanforderungen
- Passt weniger: reine Inhaltsseiten und Blogs (z. B. Astro oder statische Generatoren)
- Passt weniger: sehr kleine Widgets
- Passt weniger: Teams ohne TypeScript-Erfahrung und ohne Zeit zum Einarbeiten
- Entscheidungskriterien: Teamgröße, Lebensdauer, vorhandene Skills, Integrationsbedarf
- Außerdem: Rendering-Anforderungen wie Suchmaschinen-Optimierung (SEO) und Server-Rendering



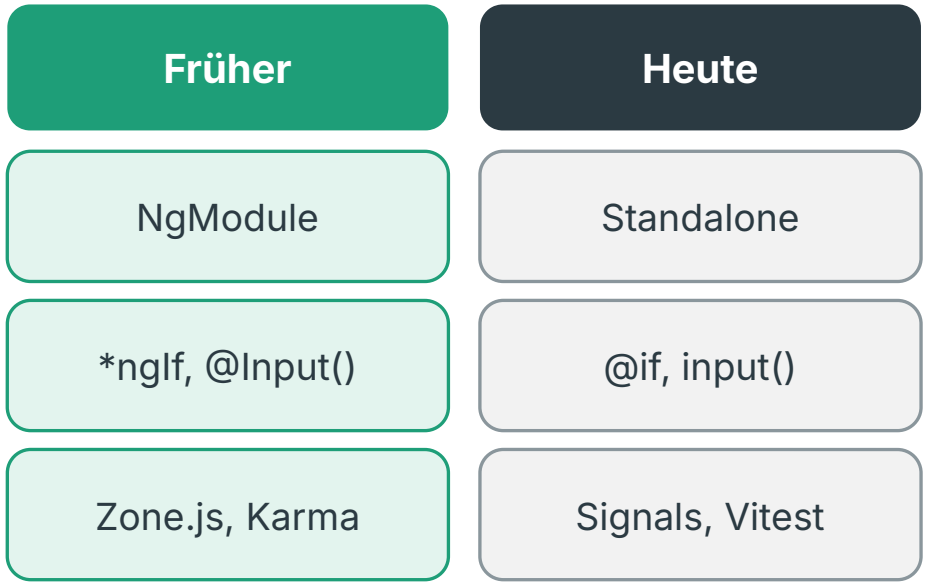
Architekturprinzipien von Angular

- Komponentenbasiert: Die Oberfläche ist ein Baum aus Komponenten (Template, Klasse, Stil)
- Dependency Injection: Abhängigkeiten werden bereitgestellt, nicht selbst erzeugt
  - Dadurch sind sie austauschbar und gut testbar
- Einseitiger Datenfluss: Daten fließen nach unten (input), Ereignisse nach oben (output)
- Starke Typisierung: TypeScript im strict-Modus, typisierte Templates und Formulare
- Konvention vor Konfiguration: gleiche Struktur in jedem Projekt, schnelle Einarbeitung



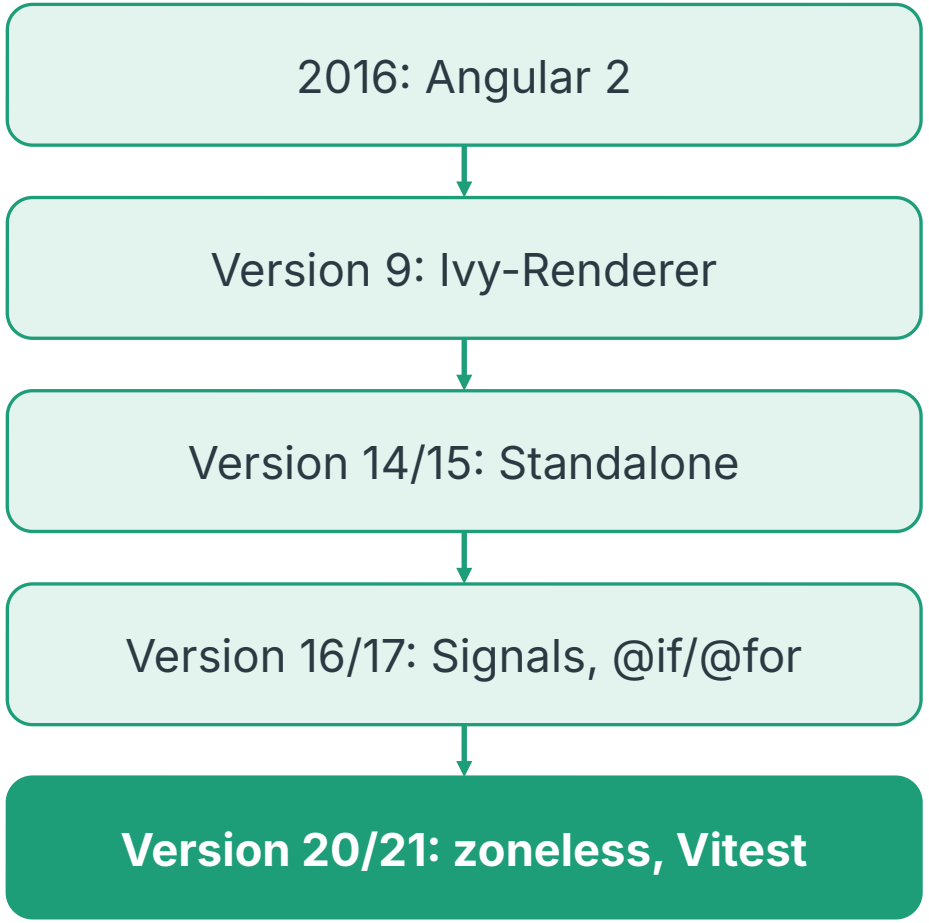
Die Angular-Denkweise heute

- Leitgedanke: weniger Boilerplate, mehr Reaktivität
- NgModule wird zu Standalone-Komponenten (imports: [])
- Zone.js und implizite Change Detection werden zu Signals und zoneless
- *ngIf und *ngFor werden zu @if, @for und @switch
- @Input(), @Output() und EventEmitter werden zu input(), output() und model()
- Konstruktor-Injection wird zu inject()
- Klassen-Guards und -Interceptors werden zu funktionalen Guards und Interceptors
- Karma und Jasmine werden zu Vitest



Entwicklung in Etappen

- 2016: Angular 2 als Neuentwurf gegenüber AngularJS
- Ivy-Renderer (Version 9), Standalone-Komponenten (ab Version 14/15)
- Signals und neuer Control Flow (Version 16/17)
- Zoneless (Version 20/21), Vitest als Standard-Testrunner (Version 21)
- Release-Rhythmus: zwei Major-Versionen pro Jahr, jede etwa 18 Monate unterstützt
- Updates mit ng update und Migrations-Schematics
- Wissen veraltet nicht schlagartig, aber die Muster wandeln sich
- Der Kurs vermittelt deshalb Prinzipien und nicht nur Programmierschnittstellen



Toolchain im Überblick

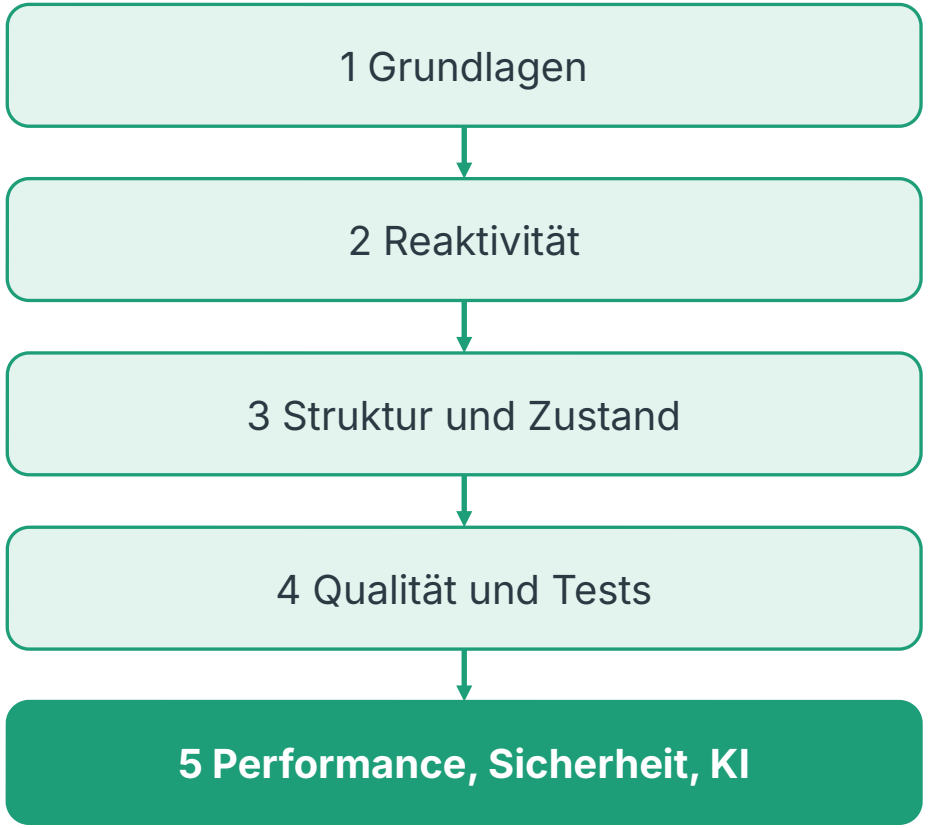
- Angular CLI: ng new, ng serve, ng generate, ng build, ng test, ng update
- Build: Application Builder auf Basis von esbuild und Vite
  - Schneller Dev-Server mit HMR (Hot Module Replacement)
  - HMR: geänderter Code erscheint, ohne die Seite vollständig neu zu laden
- TypeScript im strengen Modus, dazu ESLint und Prettier als Teamstandard
- Angular DevTools im Browser: Komponentenbaum und Profiler

Terminal

```
ng new invoice-app
ng serve        # Dev-Server, HMR
ng generate component customer-list
ng test         # Vitest
ng build        # fertige App
ng update       # neue Version
```

Der rote Faden: die Invoice-App

- Alle Übungen entwickeln eine Anwendung weiter: Kundenverwaltung mit Liste, Details, Login
- Jede Übung startet mit dem Ergebnis der vorherigen (Ordner 04_... bis 38_...)
- Wer eine Übung nicht schafft, startet die nächste im Lösungsordner der vorigen Übung
- Aufbau des Kurses:
  - 1. Grundlagen: Komponenten, Templates, Services, Routing
  - 2. Reaktivität und Datenfluss
  - 3. Anwendungsstruktur und Zustandsverwaltung
  - 4. Qualität, Tests, Wartbarkeit
  - 5. Performance, Sicherheit, KI-gestützte Entwicklung



Eine App, die in jeder Übung weiterwächst

Erwartungen und Vorerfahrung

- Wer hat bisher Weboberflächen entwickelt, und womit?
- Welche Anwendung soll nach dem Kurs mit Angular entstehen oder gepflegt werden?
- Welche zwei Themen sind besonders wichtig?
- Das Ergebnis kommt auf ein Flipchart
- Am Kursende wird es mit dem Erlernten abgeglichen
  - Grundkurs: Ende Tag 3, Kompaktkurs: Ende Tag 5

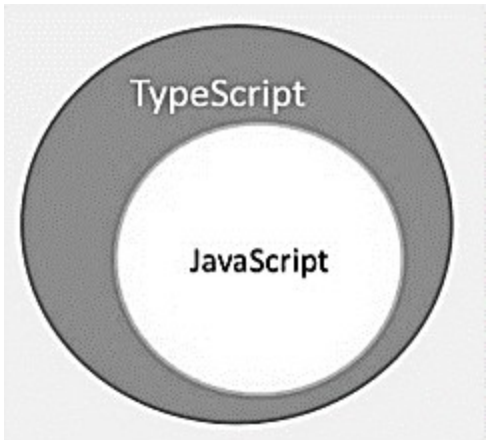


Typescript Primer

Einführung in Typescript

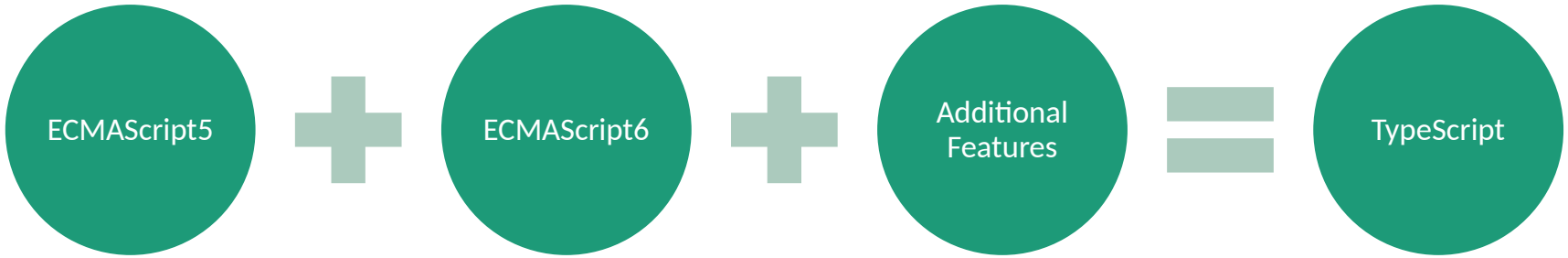
TypeScript is just JavaScript

TypeScript is portable.



TypeScript supports other JS libraries

JavaScript is TypeScript.


ECMAScript5
ECMAScript6
Additional Features
TypeScript



OO Concepts



Supports Type Definitions



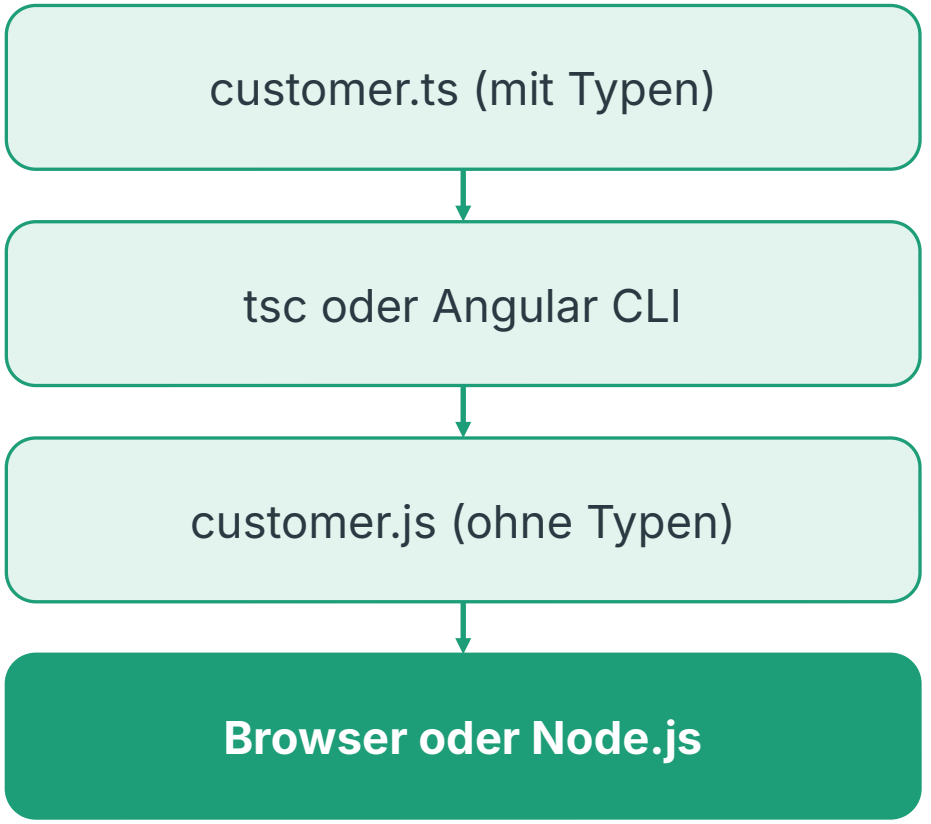
Strong static Type



Compilation

Vom Quelltext zum Browser

- Wir schreiben TypeScript-Dateien mit der Endung .ts
- Der Compiler „tsc“ übersetzt sie in JavaScript-Dateien (.js)
- Browser und Node.js verstehen nur JavaScript, kein TypeScript
- Beim Übersetzen verschwinden alle Typen: Im fertigen Programm gibt es sie nicht mehr
- Die Angular CLI übernimmt das Übersetzen für uns
- Typen prüfen ist nicht dasselbe wie das Programm ausführen



Typen gibt es nur beim Entwickeln

Der Kursordner und die Befehle

- Ordner: 01_typescript_primer, darin einmalig: npm install
- npm run check: prüft alle Typen in src/ und meldet Fehler
- npm run watch: wie check, prüft bei jeder Änderung neu
- npm start -- src/<datei>.ts: führt eine Datei aus
- Achtung: npm start prüft die Typen NICHT
- Fehler zeigen der Editor (rote Wellenlinien) und npm run check
- Aufgaben liegen in src/, Musterlösungen in loesung/

tsconfig.json der Invoice-App

```
{
  "compilerOptions": {
    "strict": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "target": "ES2022",
    "experimentalDecorators": true
  },
  "angularCompilerOptions": {
    "strictTemplates": true
  }
}
```

Ein erster Typfehler

```
function greet(firstname: string, lastname: string): string {
  return `Hallo ${firstname} ${lastname}`;
}

greet("Lisa", "Simpson");   // richtig

greet("Lisa", 42);
// Fehler TS2345: Argument of type 'number' is not
// assignable to parameter of type 'string'.
```

Typen

Typen und Typableitung

```
const firstname: string = "Lisa";
const age: number = 42;
const active: boolean = true;
const since: Date = new Date(2024, 0, 15);
const names: string[] = ["Lisa", "Hans", "Ned"];

// Typableitung: Der Compiler erkennt den Typ selbst
const city = "Springfield";   // string
const count = names.length;   // number
const ids = [1, 2, 3];        // number[]
```

const und let

```
const company = "Simpson GmbH";   // Standard: const
let counter = 0;                  // let nur, wenn sich der Wert ändert
counter = counter + 1;            // erlaubt

// company = "Andere GmbH";       // Fehler: const nicht neu zuweisbar

const names: string[] = ["Lisa"];
names.push("Hans");               // erlaubt: der Inhalt ändert sich
// names = [];                    // Fehler: die Variable bleibt fest
```

Warum nicht mehr var

```
for (var i = 0; i < 10; i++) {
  setTimeout(() ⇒ console.log("var:", i), 0);
}
// Ausgabe: zehnmal "var: 10"

for (let j = 0; j < 10; j++) {
  setTimeout(() ⇒ console.log("let:", j), 0);
}
// Ausgabe: "let: 0" bis "let: 9"
```

any, unknown und never

```
let anything: any = "Lisa";
anything.toUpperCase();     // kein Fehler, auch wenn es knallen kann

function show(value: unknown): string {
  if (typeof value === "string") {
    return value.toUpperCase();   // hier ist value ein string
  }
  return String(value);
}

function fail(message: string): never {
  throw new Error(message);       // kehrt nie zurück
}
```

null und undefined im strikten Modus

```
function cityLabel(city: string | undefined): string {
  return city.toUpperCase();
  // Fehler TS18048: 'city' is possibly 'undefined'.
}

function safeCityLabel(city: string | undefined): string {
  if (city === undefined) {
    return "unbekannt";
  }
  return city.toUpperCase();   // hier ist city sicher ein string
}
```

Sicher mit fehlenden Werten: ?., ?? und !

```
const customer = customers.find(c ⇒ c.id === "3");  // Customer | undefined

const city = customer?.city;                  // ?. : stoppt bei fehlendem Wert
const label = customer?.city ?? "unbekannt";  // ?? : Ersatzwert

const lisa = customers.find(c ⇒ c.id === "1")!;  // ! : "ich weiß es sicher"
console.log(lisa.firstname);

// Im Kurs:  customer()?.firstname
//           customer()!.imageUrl!
```

TS-Übung 1: Typfehler beheben

- **Öffne 01_typescript_primer, Datei src/uebung-1-typfehler.ts**
- **1. Führe npm run check aus und lies die 6 Fehlermeldungen**
- **2. Behebe jeden Fehler, ohne any zu benutzen**
- **3. Manche Fehler behebst du im Code, bei anderen musst du den Typ anpassen**
- **4. Starte das Programm: npm start -- src/uebung-1-typfehler.ts**
- Tipp: string | undefined, customer.city?.toUpperCase() ?? "UNBEKANNT"
- **Musterlösung: loesung/uebung-1-typfehler.ts**

Funktionen

Funktionen typisieren

```
function cityOf(customer: Customer, fallback = "unbekannt"): string {
  return customer.city ?? fallback;
}

function greet(name: string, greeting?: string): string {
  return `${greeting ?? "Hallo"} ${name}`;
}

function logMessage(message: string): void {
  console.log(message);
}

cityOf(customers[2]);                  // "unbekannt"
cityOf(customers[2], "Ogdenville");    // "Ogdenville"
```

Rest-Parameter und Arrow-Funktionen

```
function sum(...nums: number[]): number {
  return nums.reduce((a, b) ⇒ a + b, 0);
}
sum(1, 2, 3);               // 6

const double = (n: number): number ⇒ n * 2;
const add = (a: number, b: number) ⇒ a + b;

function applyTwice(value: number, fn: (n: number) ⇒ number): number {
  return fn(fn(value));
}
applyTwice(3, double);      // 12
applyTwice(3, n ⇒ n + 10);  // 23
```

Listen verarbeiten: filter, map, find

```
const springfield = customers.filter(c ⇒ c.city === "Springfield");
// zwei Kunden

const names = customers.map(c ⇒ `${c.firstname} ${c.lastname}`);
// ["Lisa Simpson", "Hans Maulwurf", "Ned Flanders"]

const ned = customers.find(c ⇒ c.id === "3");   // Customer | undefined
console.log(ned?.firstname ?? "nicht gefunden");

const active = customers
  .filter(c ⇒ !c.discontinued)
  .map(c ⇒ c.lastname);                    // ["Simpson", "Flanders"]
```

TS-Übung 2: Funktionen typisieren

- **Öffne 01_typescript_primer, Datei src/uebung-2-funktionen.ts**
- **1. Schreibe fullName und filterCustomers (Tipp: filter, includes, toLowerCase)**
- **2. Schreibe cityOf mit einem Standardwert für den zweiten Parameter**
- **3. Schreibe joinNames mit einem Rest-Parameter (...customers), map und join**
- **4. Bis Aufgabe 3 und 4 gelöst sind, meldet npm run check zwei Fehler: das ist gewollt**
- Start: npm start -- src/uebung-2-funktionen.ts
- **Musterlösung: loesung/uebung-2-funktionen.ts**

Objekte und Listen

Interfaces: die Form eines Objekts

```
interface Customer {
  id: string;
  firstname: string;
  lastname: string;
  city?: string;
  discontinued?: boolean;
  lastOrderDate?: Date;
}

const lisa: Customer = { id: "1", firstname: "Lisa", lastname: "Simpson" };

// Fehler: Die Pflicht-Eigenschaft "lastname" fehlt
// const hans: Customer = { id: "2", firstname: "Hans" };
```

Optional und readonly

```
interface Invoice {
  readonly id: string;     // nur lesbar, nach dem Anlegen nicht änderbar
  customerId: string;
  note?: string;           // optional, darf fehlen
}

const invoice: Invoice = { id: "R-1", customerId: "1" };
invoice.note = "bezahlt";  // erlaubt
// invoice.id = "R-2";     // Fehler TS2540: id ist readonly

const ids: readonly string[] = ["1", "2"];
// ids.push("3");          // Fehler: Eine readonly-Liste hat kein push
const more = [...ids, "3"];  // neue Liste ist erlaubt
```

Destructuring

```
const [first, , third] = customers;      // Liste zerlegen, zweiten überspringen

const { firstname, lastname } = first;   // Objekt zerlegen
const { city = "unbekannt" } = third;    // Standardwert, falls city fehlt
const { id, ...rest } = first;           // rest: alle übrigen Eigenschaften

// Zerlegen direkt in der Parameterliste
function label({ firstname, lastname }: Customer): string {
  return `${lastname}, ${firstname}`;
}

console.log(firstname, lastname, city, id);   // Lisa Simpson unbekannt 1
console.log(rest.city, label(third));         // Springfield Flanders, Ned
```

Spread: kopieren statt verändern

```
const lisa = customers[0];

const renamed = { ...lisa, firstname: "Liesel" };  // neues Objekt
const more = [...customers, renamed];              // neue Liste

console.log(lisa.firstname, renamed.firstname);    // Lisa Liesel
console.log(customers.length, more.length);        // 3 4

// Achtung: Spread kopiert nur flach, Verschachteltes wird geteilt
const copy = { ...lisa };
console.log(copy.lastOrderDate === lisa.lastOrderDate);  // true
```

Template-Strings

```
const c = customers[0];

// Backticks: Ausdrücke in ${...} werden in den Text eingesetzt
const line = `${c.lastname}, ${c.firstname} (${c.city ?? "unbekannt"})`;

// Auch mehrere Zeilen sind möglich
const mail = `Guten Tag ${c.firstname},
Ihre Rechnung ist fertig.`;

const total = 3 * 19.99;
console.log(`Summe: ${total.toFixed(2)} Euro`);   // Summe: 59.97 Euro
console.log(line);                                 // Simpson, Lisa (Springfield)
console.log(mail);
```

TS-Übung 3: Objekte und Listen

- **Öffne 01_typescript_primer, Datei src/uebung-3-objekte.ts**
- **1. renameCustomer: neue Liste, ein Kunde hat den neuen Vornamen (map und Spread)**
- **2. addCustomer: neue Liste mit einem Kunden mehr am Ende ([...customers, customer])**
- **3. describe: Parameter in der Signatur zerlegen, city-Standardwert, Template-String**
- **4. Prüfe in der Ausgabe: Die ursprüngliche Liste ist unverändert geblieben**
- Ausführen: npm start -- src/uebung-3-objekte.ts
- **Musterlösung: loesung/uebung-3-objekte.ts**

Typen kombinieren

Union Types und Narrowing

```
function show(value: string | number): string {
  return typeof value === "string" ? value.toUpperCase() : value.toFixed(2);
}

type State =
  | { status: "loading" }
  | { status: "loaded"; data: Customer[] }
  | { status: "error"; message: string };

function render(state: State): string {
  switch (state.status) {
    case "loading": return "Lade …";
    case "loaded": return `${state.data.length} Kunden`;
    case "error": return state.message;
  }
}
```

Literal-Typen und type

```
type Status = "active" | "discontinued";   // nur diese zwei Texte sind erlaubt
type CustomerId = string;                  // ein Alias, also ein zweiter Name

function setStatus(c: Customer, status: Status): Customer {
  return { ...c, discontinued: status === "discontinued" };
}

setStatus(customers[0], "discontinued");
// setStatus(customers[0], "gesperrt");    // Fehler TS2345: nicht erlaubter Wert

enum Role { Admin, User }                  // enum gibt es auch
const role: Role = Role.Admin;
```

interface oder type?

- Beide beschreiben die Form eines Objekts, mit optionalen und readonly-Eigenschaften
- Beide lassen sich erweitern: interface mit extends, type mit & (Schnittmenge)
- Nur type kann Unions, Literal-Typen und Alias für einfache Typen: type Status = "a" | "b"
- Nur interface kann später um Eigenschaften ergänzt werden (declaration merging)
- Klassen können ein interface oder einen Objekt-type mit implements erfüllen
- Faustregel im Kurs: Objekt-Formen als interface (Customer)
- Faustregel im Kurs: Unions und Aliase als type (Status, State)
- Wichtiger als die Wahl: im Projekt einheitlich bleiben

Faustregel im Kurs

```
interface Customer {
  id: string;
  lastname: string;
}

type Status = "active" | "gesperrt";

type State =
  | { status: "loading" }
  | { status: "error" };
```

Generics

```
const names: Array<string> = ["Lisa", "Hans"];       // dasselbe wie string[]
const pending: Promise<Customer[]> = Promise.resolve(customers);

function first<T>(items: T[]): T | undefined {
  return items[0];
}

const firstCustomer = first(customers);   // Typ: Customer | undefined
const firstName = first(names);           // Typ: string | undefined

// In Angular begegnen uns dieselben spitzen Klammern:
// signal<Customer[]>([]), input<string>(), Observable<Customer>
```

Hilfstypen: Partial, Pick, Omit

```
type CustomerChanges = Partial<Customer>;       // alles optional
type CustomerName = Pick<Customer, "firstname" | "lastname">;  // nur diese
type NewCustomer = Omit<Customer, "id">;        // alles außer id

function update(c: Customer, changes: Partial<Customer>): Customer {
  return { ...c, ...changes };
}

const changed = update(customers[0], { city: "Shelbyville" });
const draft: NewCustomer = { firstname: "Ned", lastname: "Flanders" };
const label: CustomerName = changed;     // Customer hat diese Felder auch
```

Klassen

Klassen: Attribute, Konstruktor, Methoden

```
class Customer {
  id: string;
  firstname: string;
  lastname: string;
  city?: string;
  constructor(firstname: string, lastname: string, city?: string) {
    this.id = crypto.randomUUID();
    this.firstname = firstname;
    this.lastname = lastname;
    this.city = city;
  }
  getLabel(): string {
    return `Kunde: ${this.firstname} ${this.lastname}`;
  }
}
const lisa = new Customer("Lisa", "Simpson", "Springfield");
```

Parameter Properties

```
class Customer {
  id: string;
  constructor(
    public firstname: string,
    public lastname: string,
    public street?: string,
    public zip?: string,
    public city?: string,
  ) {
    this.id = crypto.randomUUID();
  }
}
// Später in Angular:
// constructor(private service: CustomerService) {}
```

Sichtbarkeit und readonly

```
class Account {
  readonly id = crypto.randomUUID();
  protected owner: string;
  private balance = 0;
  constructor(owner: string) {
    this.owner = owner;
  }
  deposit(amount: number): void {
    this.balance += amount;
  }
}
const account = new Account("Lisa");
account.deposit(50);
console.log(account.balance);
```

Getter und Setter

```
class Customer {
  private _discount = 0;
  constructor(public firstname: string, public lastname: string) {}
  get fullName(): string {
    return `${this.firstname} ${this.lastname}`;
  }
  get discount(): number {
    return this._discount;
  }
  set discount(value: number) {
    if (value < 0 || value > 100) throw new Error("0 bis 100");
    this._discount = value;
  }
}
```

implements: ein Interface erfüllen

```
interface Labelled {
  getLabel(): string;
}
class Customer implements Labelled {
  constructor(public firstname: string, public lastname: string) {}
  getLabel(): string {
    return `Kunde: ${this.firstname} ${this.lastname}`;
  }
}
function printLabel(item: Labelled): void {
  console.log(item.getLabel());
}
printLabel(new Customer("Lisa", "Simpson"));
```

extends und super

```
class VipCustomer extends Customer {
  constructor(
    firstname: string,
    lastname: string,
    public discount: number,
  ) {
    super(firstname, lastname);
  }
  override getLabel(): string {
    return super.getLabel() + ` - VIP -${this.discount}%`;
  }
}
const ned = new VipCustomer("Ned", "Flanders", 10);
console.log(ned.getLabel());
```

Dekoratoren: ein Blick auf Angular

```
function Component(config: { selector: string }) {
  return (target: Function) => {
    console.log(`${target.name} → <${config.selector}>`);
  };
}

@Component({ selector: "customer-list" })
class CustomerListComponent {
  title = "Kunden";
}
```

TS-Übung 4: Die Klasse Customer

- Öffne 01_typescript_primer, Datei src/uebung-4-klassen.ts
- 1. Schreibe die Klasse Customer mit Parameter properties (firstname, lastname, city?)
- 2. Setze id im Konstruktor mit crypto.randomUUID()
- 3. Ergänze den Getter fullName und die Methode getLabel()
- 4. Customer implementiert das Interface Labelled
- 5. VipCustomer extends Customer mit discount: super(...) zuerst, getLabel() ergänzen
- 6. Entferne die Kommentarzeichen am Dateiende und starte: npm start -- src/uebung-4-klassen.ts
- Musterlösung: loesung/uebung-4-klassen.ts

Module

Module: export und import

```
// customer.ts
export interface Labelled {
  getLabel(): string;
}
export class Customer implements Labelled {
  constructor(public firstname: string, public lastname: string) {}
  getLabel(): string {
    return `Kunde: ${this.firstname} ${this.lastname}`;
  }
}
// main.ts
import { Customer, type Labelled } from "./customer";
```

Regeln für import und export

- Imports sind statisch: sie stehen oben in der Datei, nicht in einer if-Abfrage
- Pfad zu eigenen Dateien beginnt mit ./ oder ../ und hat keine Endung
- Benannte Exporte: export class Customer, Import mit geschweiften Klammern
- Standard-Export: export default, Import ohne Klammern (im Kurs selten)
- import type { Labelled } holt nur einen Typ, der nach dem Übersetzen verschwindet
- Pakete wie @angular/core importiert man ohne Pfad: import { Component } from "@angular/core"
- Eine Datei mit import oder export ist ein Modul; ihre Namen stören andere Dateien nicht

TS-Übung 5: Module

- **Öffne 01_typescript_primer, Datei src/uebung-5-module.ts**
- **1. Lege die Datei src/customer.ts an**
- **2. Verschiebe die Klasse Customer dorthin und exportiere sie mit export**
- **3. Importiere sie in src/uebung-5-module.ts: import { Customer } from "./customer";**
- **4. Starte: npm start -- src/uebung-5-module.ts**
- Zusatz: Exportiere und importiere auch das Interface Labelled
- **Musterlösung: loesung/uebung-5-module/**

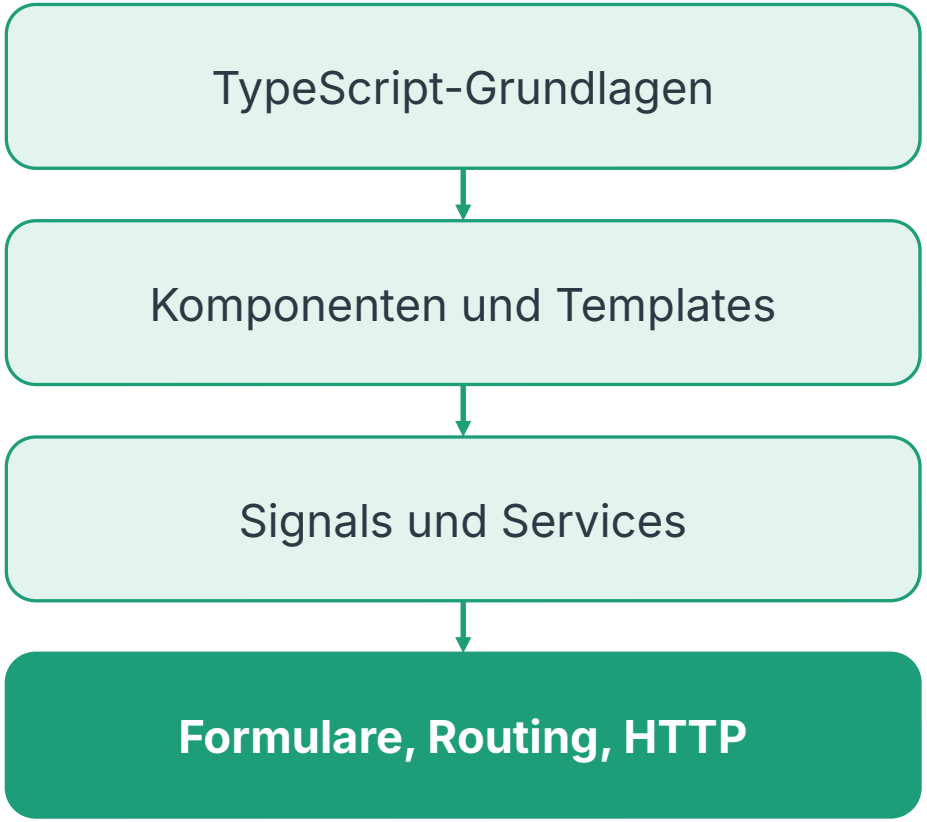
Von TypeScript zu Angular

```
import { Component } from "@angular/core";
import { Customer } from "./customer";

@Component({
  selector: "app-customer-list",
  template: `<h1>Kunden</h1>`,
})
export class CustomerListComponent {
  customers: Customer[] = [];
  constructor(private service: CustomerService) {}
}
```

Zusammenfassung: Was wir für Angular mitnehmen

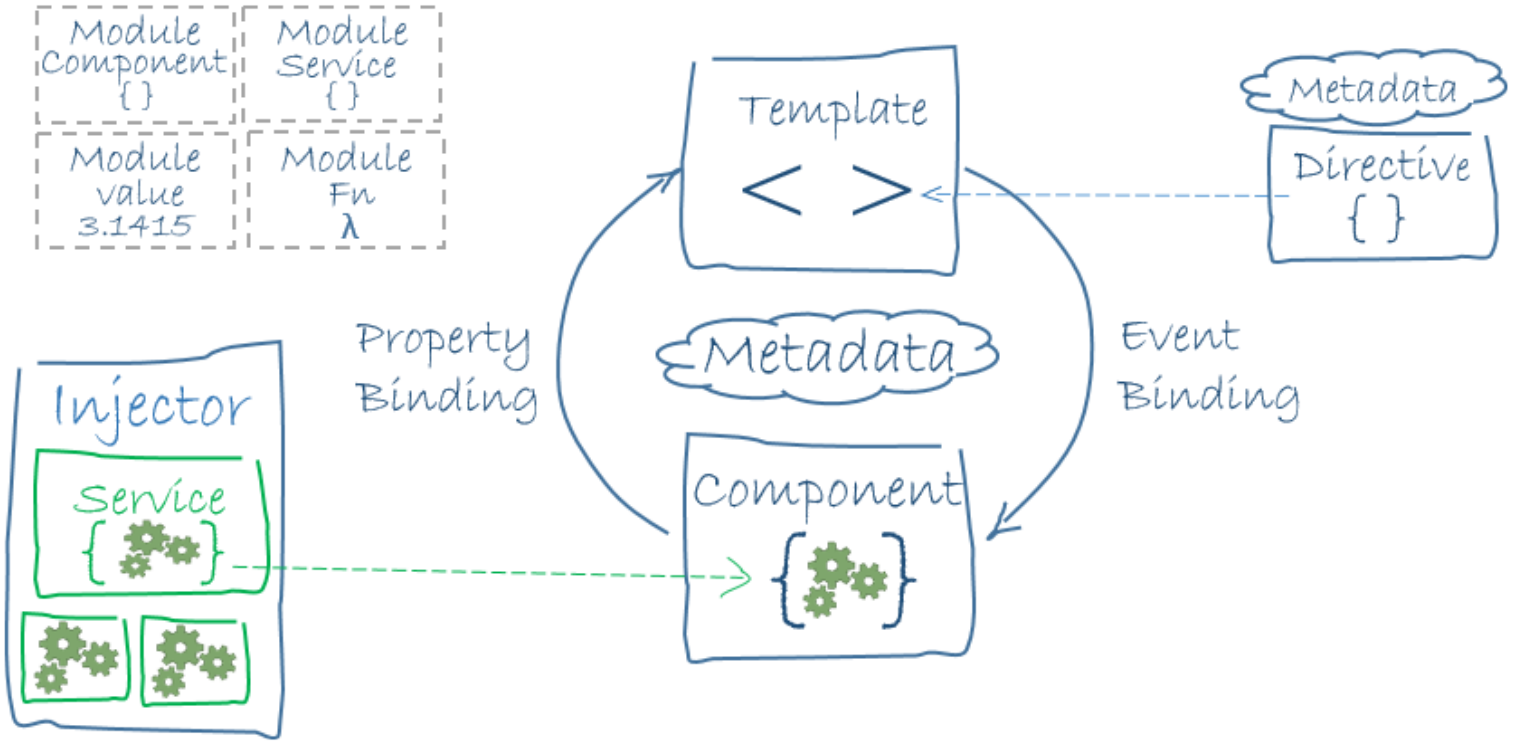
- Typen und Typableitung, strict, null und undefined: Grundlage jeder Komponente
- ?. und ??: im Template, zum Beispiel customer()?.firstname
- Funktionen, Arrow-Funktionen, filter und map: Listen im Kurs verarbeiten
- Interfaces und Destructuring: Datenmodelle und Eingaben
- Spread: neue Objekte statt Veränderung, wichtig für Signals (Reaktivität)
- Union Types und Literal-Typen: Zustände wie loading und loaded
- Generics: signal<T>(), input<T>(), Observable<T>
- Klassen und Parameter properties: customer.model.ts, Konstruktor-Injection
- Dekoratoren und Module: @Component, import und export in jeder Datei



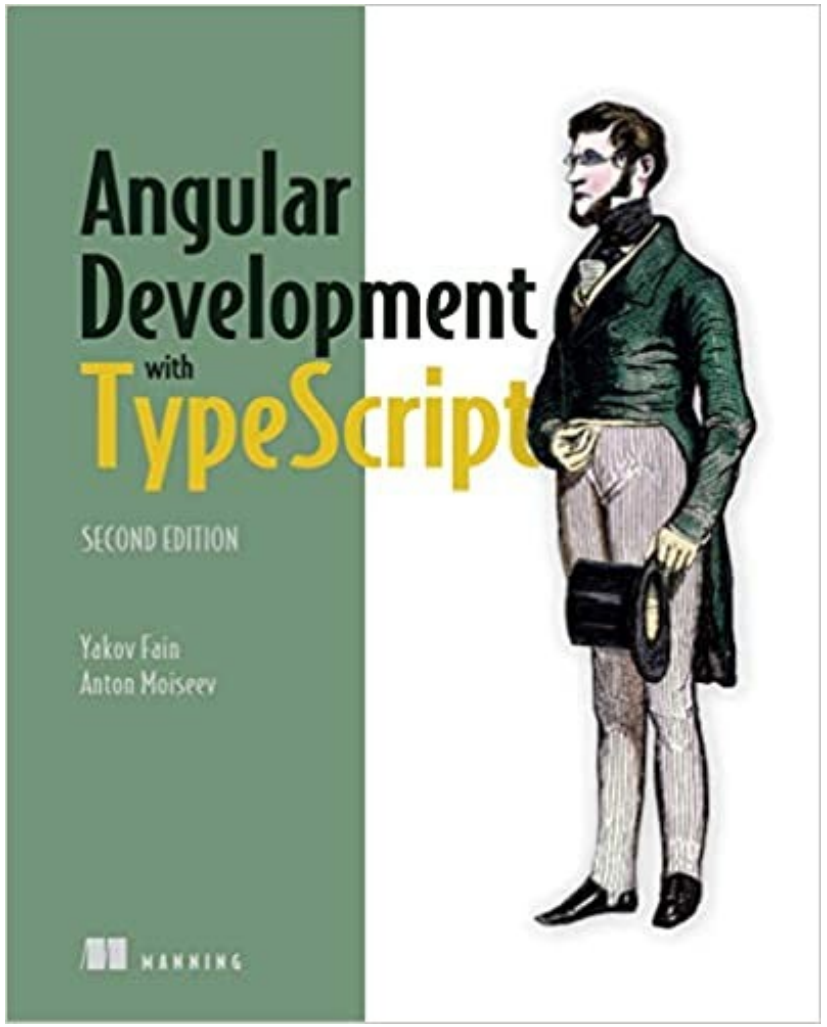
Weiter mit Übung 4

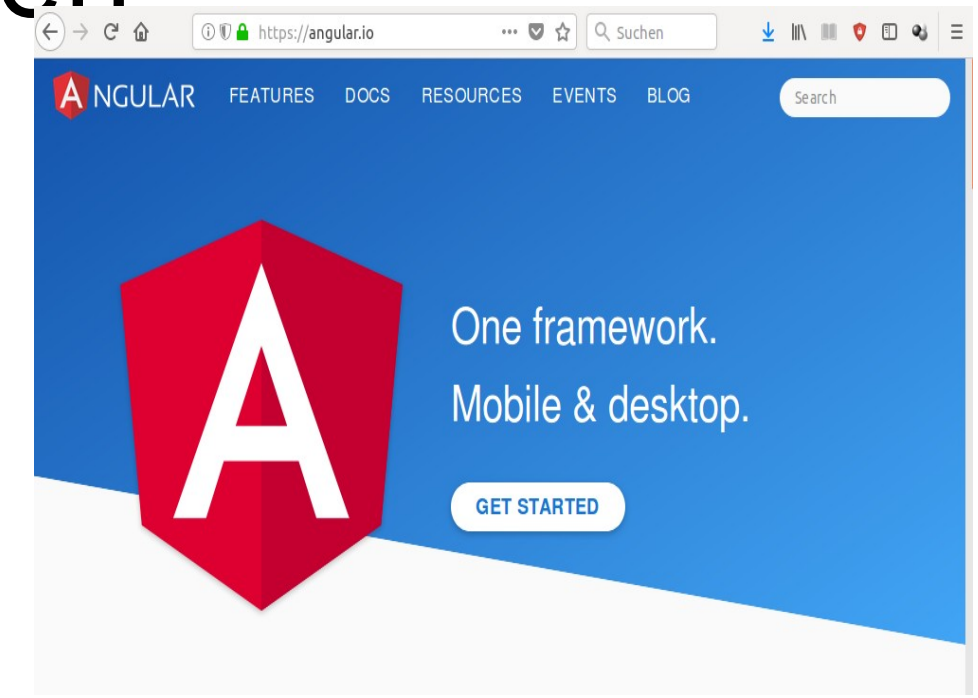

Angular

Architektur Überblick



Literatur und Quellen







Installation und Setup

JavaScript-Grundinstallation

node und npm sind auf einem Entwicklerrechner zu installieren

- Näheres hierzu im Anhang

Damit steht ein ausgefeilter Buildprozess zur Verfügung

- Verzeichnisstruktur und Projekt-Organisation
- Automatische Transpilation
- Browser-Update bei Änderungen an den Quellen

Ein spezieller Editor ist nicht notwendig

- Empfohlen wird Atom oder ähnliches

Installation des Angular Command Line Interfaces

- `npm install -g @angular/cli`
- Damit ist der Angular-Projektassistent `ng` installiert
  - Anlegen eines neuen Projekts
    - `ng new <projektname>`
  - Erzeugen neuer Angular-Programmteile
    - Services
    - Direktiven
    - …
- Freunde des Handmade Manifesto:
  - Das Anlegen eines Angular-Projektes kann natürlich auch ohne dieses Werkzeug erfolgen

Überblick über ng command line

- Development server
  - `ng serve startet einen Dev-Server (Vite/esbuild) auf http://localhost:4200/`
- Code Scaffolding
  - `ng generate component component-name (standalone ist Default)`
  - `directive|pipe|service|class|guard|interceptor|interface|enum|resolver`
- Build
  - `ng build (Production ist Default, Application Builder mit esbuild)`
- Unit Tests
  - `ng test – neue Projekte ab v21 mit Vitest, Bestandsprojekte mit Karma/Jasmine`
- End-to-End Tests
  - `ng e2e – Runner per ng add hinzufügen, z. B. Cypress oder Playwright`
- Updates
  - `ng update @angular/core @angular/cli – inkl. automatischer Migrationen`

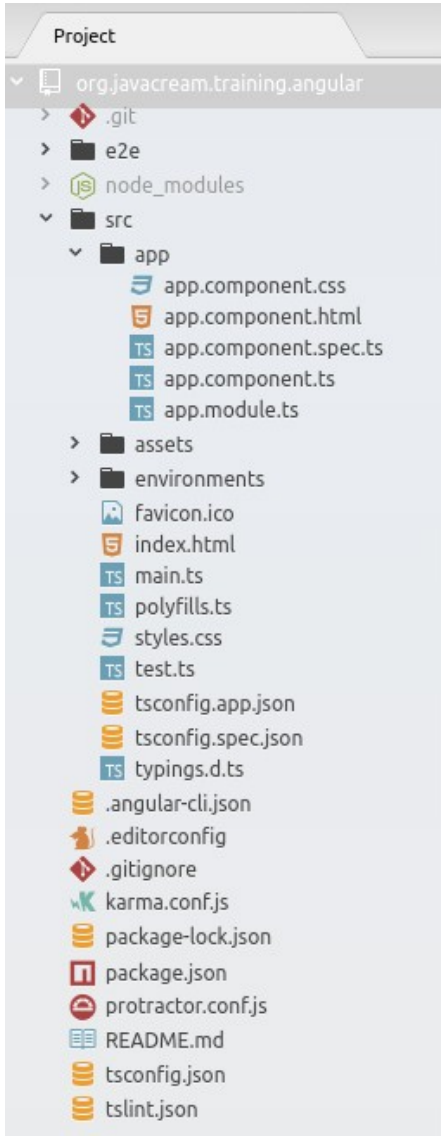


Ein erstes Projekt

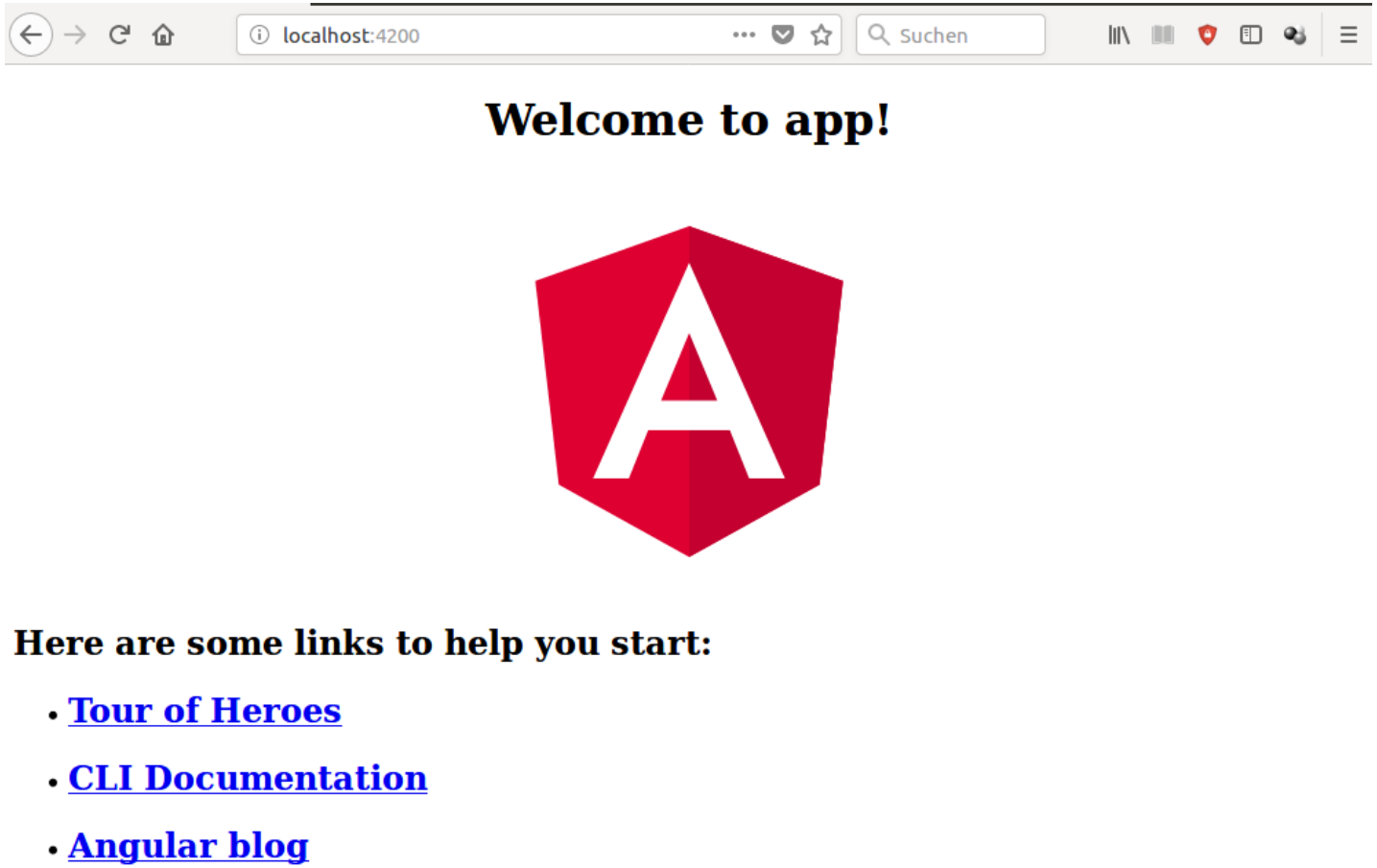
Anlegen des Projekts

- `ng new org.somepackage.angular`
- Damit wird ein node-Projekt erzeugt
- Ebenso werden automatisch alle Abhängigkeiten installiert
  - Insbesondere TypeScript

Projektstruktur



Die Anwendung unter localhost:4200



Übung 4: Erstelle Initiales Projekt

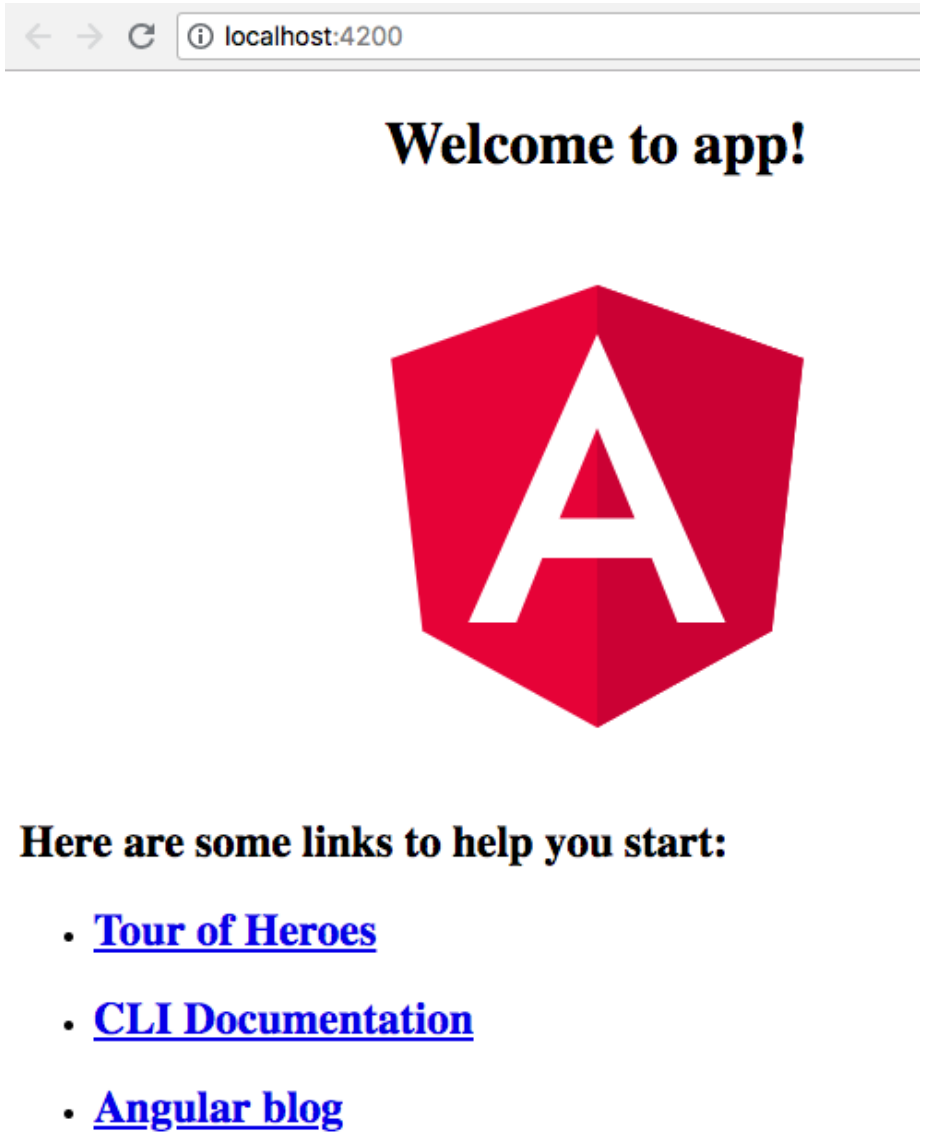
- Öffne ein Terminal
- Installiere, falls noch nicht geschehen, das Angular CLI global (siehe *Benötigte Software*)
- Navigiere zu einem Ordner in dem das neues Projekt anzulegen werden soll
- Lege eine neues Projekt an: ng new invoice-app
- Wechsel in das erzeugte Verzeichnis invoice-app
- Führe ng serve aus
- Öffne Anwendung im Browser unter *http://localhost:4200*



Reload der App im Hintergrund bei Change

- ng serve –o (observe)
- Beispiel:
  - Ändern des Seitentitels sowie der Willkommens-Nachricht
    - *src/index.html*
    - *src/app/app.component.html*
    - *src/app/app.component.ts*

Die aktualisierte Seite



Projektüberblick

Standalone Components

- Komponenten, Direktiven und Pipes sind standardmäßig standalone
  - standalone: true muss seit Angular 19 nicht mehr angegeben werden
- Kein NgModule zur Deklaration nötig
  - Jede Komponente deklariert ihre Abhängigkeiten selbst
- imports: [] direkt in der @Component-Annotation
  - Andere Komponenten, Direktiven, Pipes (z. B. RouterLink, FormsModule)
  - Für @if / @for ist kein CommonModule nötig
- Vorteile
  - Weniger Boilerplate, besseres Tree-Shaking, einfaches Lazy Loading
- NgModules: siehe Legacy-Kapitel am Ende (Übung 30)

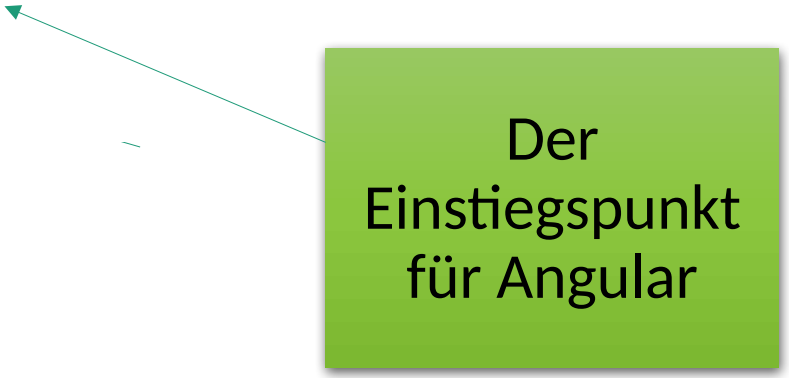
Übersicht der Dateien eines Projekts

- `src/index.html`
  - Die Hauptseite
- `src/main.ts`
  - *Start der Anwendung per bootstrapApplication – ersetzt app.module.ts. Module siehe Projekt 25 (Legacy)*
- `src/app/app.component.ts`
  - Definition des dynamischen Anteils einer Komponente mit TypeScript
- `src/app/app.component.html`
  - Definition des HTML-Templates einer Komponente
- `src/app/app.component.css`
  - Stylesheet der Komponente
- `src/app/app.component.spec.ts`
  - Definition von Unit-Tests (Vitest bzw. Jasmine/Karma)

Referenz auf Bootstrap Component in Index-Seite

index.html

```
1   <!doctype html>
2   <html lang="en">
3   <head>
4     <meta charset="utf-8">
5     <title>org.Javacream.Training.Angular</title>
6     <base href="/">
7
8     <meta name="viewport" content="width=device-width, initial-scale=1">
9     <link rel="icon" type="image/x-icon" href="favicon.ico">
10  </head>
11  <body>
12    <app-root></app-root>
13  </body>
14  </html>
15
```



Component-Annotation

- Eine Component ist eine TypeScript-Klasse mit der `@Component`-Annotation
  - Annotationen enthalten Metadaten, die der Klasse zugeordnet werden
  - Diese müssen von einem Framework interpretiert werden
- Annotationen sind damit so etwas wie statische Eigenschaften einer Klasse
  - Allerdings sind diese syntaktisch nicht Bestandteil der Klassendeklaration
  - Damit ergibt sich eine sauber Trennung
- Eine typische Component definiert
  - Den Selector als Bezug zum Template
  - Die Lokation des Templates
- Vollständige Liste unter
  - *`https://angular.io/api/core/Component`*

Aufgabe der Component

- Die Eigenschaften der Component-Klasse dienen als Schnittstelle zum HTML-Code
  - Werte werden über Interpolation oder Direktiven gebunden
  - Funktionen dienen als Event-Handler
- Bei Bedarf implementiert die Component-Klasse Lifecycle-Funktionen
  - `ngOnChanges`
  - `ngOnInit`
  - *`https://angular.io/guide/lifecycle-hooks`*

Beispiel: Component Klasse

```
import {ApplicationRef, Component, OnDestroy, OnInit} from "@angular/core";

@Component({
   selector: “app-root",
   templateUrl: "./app.component.html",
   styleUrls: ["./app.component.scss"],
   animations: [fadeAnimation]
})
export class AppComponent implements OnInit, OnDestroy {

   public title = "theFrontend";
```

TypeScript-Import der Angular-Core-Bibliothek

Referenz auf den Einstiegspunkt

Referenz auf Template und CSS

Definition einer Variablen und Wert-Zuweisung

Das HTML-Template einer Component

app.component.html

```
1   <!--The content below is only a placeholder and can be replaced.-->
2   <div style="text-align:center">
3     <h1>
4       Welcome to the great {{ title }}!
5     </h1>
6     <img width="300" alt="Angular Logo"
•     src="
•     3IDQwLjl6IiAvPgogIDwvc3ZnPg==">
7   </div>
8   <h2>Here are some links to help you start: </h2>
9   <ul>
10    <li>
11      <h2><a target="_blank" rel="noopener" href="https://angular.io/t
12    </li>
13    <li>
14      <h2><a target="_blank" rel="noopener" href="https://github.com/a
15    </li>
16    <li>
17      <h2><a target="_blank" rel="noopener" href="https://blog.angular
18    </li>
19  </ul>
```

Referenzieren der Component-Variable mit Interpolation

Kompletter Ablauf

- Ablauf insgesamt
  - Aufruf bootstrapApplication mit Component (AppComponent)
  - Komponenten-Klasse hat Decorator
  - Komponenten-Decorator hat Feld selector
  - Selector wird in DOM aus index.html gesucht
  - Komponente wird instanziiert

Bootstrapping einer Standalone-App

```
// src/main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideZonelessChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

bootstrapApplication(AppComponent, {
  providers: [
    provideZonelessChangeDetection(),
    provideRouter(routes),
    provideHttpClient()
  ]
}).catch(err ⇒ console.error(err));
// Kein AppModule mehr nötig – Legacy: siehe Übung 30
```

Einfache Erweiterungen

Mehrere Komponenten

- Auf einer Seite können auch mehrere Anwendungen dargestellt werden
- Diese sind komplett voneinander isoliert
  - Der Datenaustausch zwischen verschiedenen Komponenten wird später beschrieben

Export von Benutzerdefinierten Typen

- Eine Komponente kann auch eigene Datentypen definieren und Exportieren
  - Ein Export ist nicht notwendig, so lange der Typ nicht anderen Komponenten bekannt gemacht werden muss
- Ebenso können die Eigenschaften der Komponente komplexe Objekte sein
- Der Zugriff auf die Eigenschaften des komplexen Objekts erfolgen über die normale JavaScript-Syntax
  - Also über den `.`-Operator
  - Aber Vorsicht!
    - Die Deklarativen unterstützen nicht die komplette Syntax!
      - `if`
      - `for`
      - Deklarationen

Auflistungen

- Innerhalb der `{{}}`-Interpolation wird das `for`-Konstrukt nicht unterstützt
- Allerdings gibt es eine spezielle `forEach`-Anweisung

```
<ul>
      @for(person of people; track person.id){
       <li >
              <span>{{person.firstname}}</span> {{person.lastname}}
       </li>
      }
</ul>
```

UI-Programmierung

Data Binding

- Template nicht statisch sondern dynamisch
- Komponenten-Klasse stellt Daten bereit
  - Felder / Instanz Variablen
- Template zeigt Daten an
  - Interpolation mit {{ expression }}
  - Gibt Ergebnis der Expression als String aus
  - Zugriff nur auf Instanz-Variablen der eigenen Klasse
  - Template-Variablen

Die ngModel-Direktive

- Bisher wurden die Werte der Komponente mit der `{{}}`-Interpolation dargestellt
- Dieses Binding ist dynamisch (!)
  - Änderungen der Werte werden in der Oberfläche dargestellt
- Eigenschaften der Komponente können auch bidirektional gebunden werden
- Dazu dient die Direktive `ngModel`
  - Notwendig hierzu ist der Import von FormsModule aus @angular/forms im imports-Array der Komponente

Data Binding: Beispiel

```
import { Track } from "./../tracks/track.model";
@Component({
  selector: "music-app",
  template: `
    <h1>Music App</h1>
    Music App has {{tracks.length}} tracks to play  `
})
export class MusicAppComponent {
  public tracks: Track[];
  constructor() {
    this.tracks = [
    new Track("Sound of Silence"), new Track("I Wish"),
    new Track("Over My Shoulder"), new Track("Take Five")
   ];
 }}
```

Übung 5: Databinding

- Erstelle ein Datenmodell
  - Lege eine neue Datei src/app/customer/customer.model.ts an.
  - Erstelle und exportiere darin ein Klasse Customer mit id, firstname, lastname, street, zip, city.
  - Vergebe für jeden erzeugten Customer eine eindeutige ID.
- Erweitere AppComponent
  - Ersetze im Template den dynamischen Titel gegen statisches HTML (<h1>Invoice Manager App</h1>).
  - Importiere in der AppComponent Klasse die Klasse Customer.
  - Stelle für das Template eine Liste mit Cutomer bereit (Feld + Initializierung im Konstruktor).
  - Zeige im Template die Länge der Liste an.

Direktiven

- Allgemeines Konzept
  - Komponenten
  - Strukturelle Direktiven
  - Attribut-Direktiven
- *Direktiven* meint meist strukturelle und Attribut-Direktiven
  - Wie Komponenten Teil des UI
  - Unterschied zu Komponenten: kein eigenes Template

Direktiven

- Direktiven sind Erweiterungen von HTML, die vom Angular Framework gesucht und interpretiert werden
  - Während des Bootstrap-Vorgangs wird die Startseite der Anwendung nach Direktiven analysiert
  - Erweiterung durch
    - HTML-Elemente
    - Attribute
- Was genau beim Auftreten einer Direktive passiert hängt von der Direktiven-Implementierung ab
  - `ngModel`
    - Data Binding
  - `@for`
    - Iteration

Beispiel

```
@for (track of tracks; track track.id) {
  <li>{{track.name}}</li>
} @empty {
  <p>Keine Tracks gefunden.</p>
}
```

Direktiven des Angular-Frameworks

- Angular unterscheidet die folgenden Direktiven
  - Components sind Direktiven!
    - sie werden an ein HTML-Element gebunden
  - Structural Directives
    - Modifizieren das DOM der Anwendung
    - `for` oder `if` sind Beispiele hierfür
  - Attribute Directives
    - Ändern die Darstellung oder das Verhalten eines HTML-Elements
    - `ngStyle` als Beispiel
- Vollständige Liste unter *`https://angular.io/api?status=stable&type=directive`*

Built-in Control Flow: @if, @for, @switch

```
@if (customers.length > 0) {
  <p>{{ customers.length }} Kunden</p>
} @else {
  <p>Keine Kunden</p>
}
@for (customer of customers; track customer.id) {
  <li>{{ customer.firstname }}</li>
} @empty {
  <li>Keine Kunden gefunden.</li>
}
@switch (customer.status) {
  @case ('active') { <span>aktiv</span> }
  @default { <span>inaktiv</span> }
}
// Kein Import von CommonModule nötig, bessere Typprüfung und Performance
```

Noch ein paar Beispiele

```
@if (track.playable) {
  <button>play</button>
} @else {
  <span>nicht verfügbar</span>
}
```

Früher (Legacy, CommonModule nötig)

```
<div *ngIf="track.playable">...</div>
```

Attribut Direktiven

- Keine Veränderung der DOM Struktur

```
<form #form="ngForm">
        <input type="email" name="mail" ngModel/>
</form>
```

Übung 6

- Erweitere das Template von AppComponent
  - Zeige eine Liste aller Customer an



Child Components

Divide and conquer



Goals

- Listen Darstellung in eigene Komponente auslagern
  - Komplexere Darstellung ermöglichen
  - Wachsendes UI aufteilen
  - Wiederverwendbarkeit erreichen

Verschachtelung

- Gesamtes UI ist aus Baum von Komponenten aufgebaut
  - "Anwendung" ist Top-Level-Komponente
  - Komponenten können geschachtelt werden
- Vater- und Kind-Komponente sind isoliert
  - Kein Zugriff auf Daten des anderen
  - Daten müssen an Kind übergeben werden

Komponente als Direktive einbinden

- Komponenten sind auch Direktiven (eine Art)
- Direktiven (alle Arten) müssen bekannt gemacht werden
  - Per imports: [] in der @Component-Annotation
- Nur bekannt gemacht Direktiven werden ausgeführt

Mittels ng-generate

- Angular CLI legt neue Komponenten an
  - Klasse + alle benötigte Annotation
  - Html-Template
  - Leeres Styling
  - Test
- ng generate component *packetname/component-name*

Übung 7: Komponenten schachteln

- Erstelle eine Komponente CustomerListComponent
  - Generieren per ng generate component customer/customer-list
- Binde die neue Komponente ein
  - Importiere CustomerListComponent im imports-Array von AppComponent
  - Verwende CustomerListComponent im Template von AppComponent

Input einer Component

- Components können über das Template verschachtelt werden
  - Innerhalb des Templates wird ein Root-Element einer weiteren Angular-Component definiert
- Der Kind-Controller kann Input-Parameter definieren
  - Dazu wird eine Eigenschaft mit der `input`-Signal versehen
- Diese werden im Template gesetzt

Beispiel: Input einer Child-Component

```
<ul>
 @for(person of people; track person.id) {
   <li><people-detail [person]="person"></people-
detail></li>
 }
</ul>
```

```
@Component({
  selector: 'people-detail',
  imports: [],
  templateUrl: './people-detail.html',
  styleUrl: './people-detail.css',
})
export class PeopleDetail {
  person = input.required<Person>();

}
```

```
<h3>{{person().firstname}}
{{person().lastname}}</h3>
```

AppComponent:
Neues Root-Element für die Kind-Component

PeopleDetail
Component: Angabe des Root-Elements

Deklaration einer Input-Eigenschaft

Inputs

- Daten können an Komponente übergeben werden
  - *Property Binding* über [propertyName]="expression"
  - [propertyName] Property in der "inneren" Komponente *release-card*
  - expression Property in der "äußeren / aufrufenden" Komponente
- Data-Binding in Angular immer unidirektional
  - Objekt veränderbar
  - Referenz bleibt außen immer gleich

Übung 8: Inputs

- Erweitere die Komponente CustomerListComponent
  - Nimm als Input ein Array von Customer Objekten entgegen
  - Verschiebe die bisherige Darstellung aus AppComponent in das Template von CustomerListComponent
    (Text und Liste, nicht die h1 Überschrift)
- Passe die Verwendung von CustomerListComponent in AppComponent an
  - Lösche den entsprechenden Teil des Templates
  - Übergebe app-customer-list die Daten per Property-Binding

Events

Output aus Components

Event Binding

- Data-Binding unidirektional
  - Daten von außen nach innen
  - Wie kommen Informationen von innen nach außen?
- Komponenten können Events nach außen geben
  - *Event Binding* mit (eventName)="statement"
  - Zuweisung oder Funktionsaufruf
  - Zugriff auf das Event immer über $event möglich

Event Handler

- Im Gegensatz zum Standard-Eventmodell im Browser können beliebige Funktionen mit Parametern als Handler aufgerufen werden
- Die Event-behandelnde Funktion ist Bestandteil der `Component`

Event Handler: Beispiel

```
export class UiAppComponent {
  //…
  selectionList: string[] = ["Item1", "Item2", "Item3"];
  setItem(item: string) {
    this.data.boundData = item;
  }
}
<ul>
  @for (item of selectionList; track item) {
    <li (click)="setItem(item)">{{item}}</li>
  }
</ul>
```

Übung 9: Events

- Reagiere in CustomerListComponent auf Klicks auf die Listen-Elemente
  - Rufe eine Methode (listItemClicked) in der Component-Klasse auf
  - Übergebe das Dom-Event ($event vom Typ MouseEvent) und das Training des geklickten Listen-Eintrags
  - Logge den Namen des Customer und das Event auf die Konsole
    z.B. mit console.log(event, customer.firstname)

Output -- Events aus Komponenten

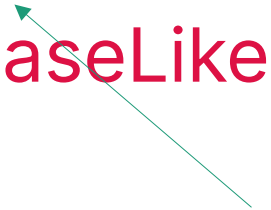
- Komponenten können Events nach außen werfen
- Eigenschaft wird mit der Funktion output<T>() angelegt
  - Typ-Parameter gibt Typ des Events an
  - Rückgabetyp ist OutputEmitterRef<T>
- Events werden über emit(wert) gefeuert
- Legacy: @Output() mit new EventEmitter<T>()

Output Beispiel

```
import { Component, input, output } from "@angular/core";
@Component({
  selector: "release-card", template: `<h3>{{release().name}}</h3>`
})
export class ReleaseCardComponent {
  release = input.required<Release>();
  // we use domain object as event, real world: ReleaseLikedEvent
  releaseLiked = output<Release>();
  likeRelease() {
    this.releaseLiked.emit(this.release());
  }
}
```

Nutzende Komponente

```
<div class="releases">
        @for (release of artist.releases; track release.id) {
    <release-card
        [release]="release"
        (releaseLiked)="onReleaseLiked($event, release)">
    </release-card>
    }
    </div>
```



Methode im aufrufendem Controller
NICHT in der Release-Card Komponente

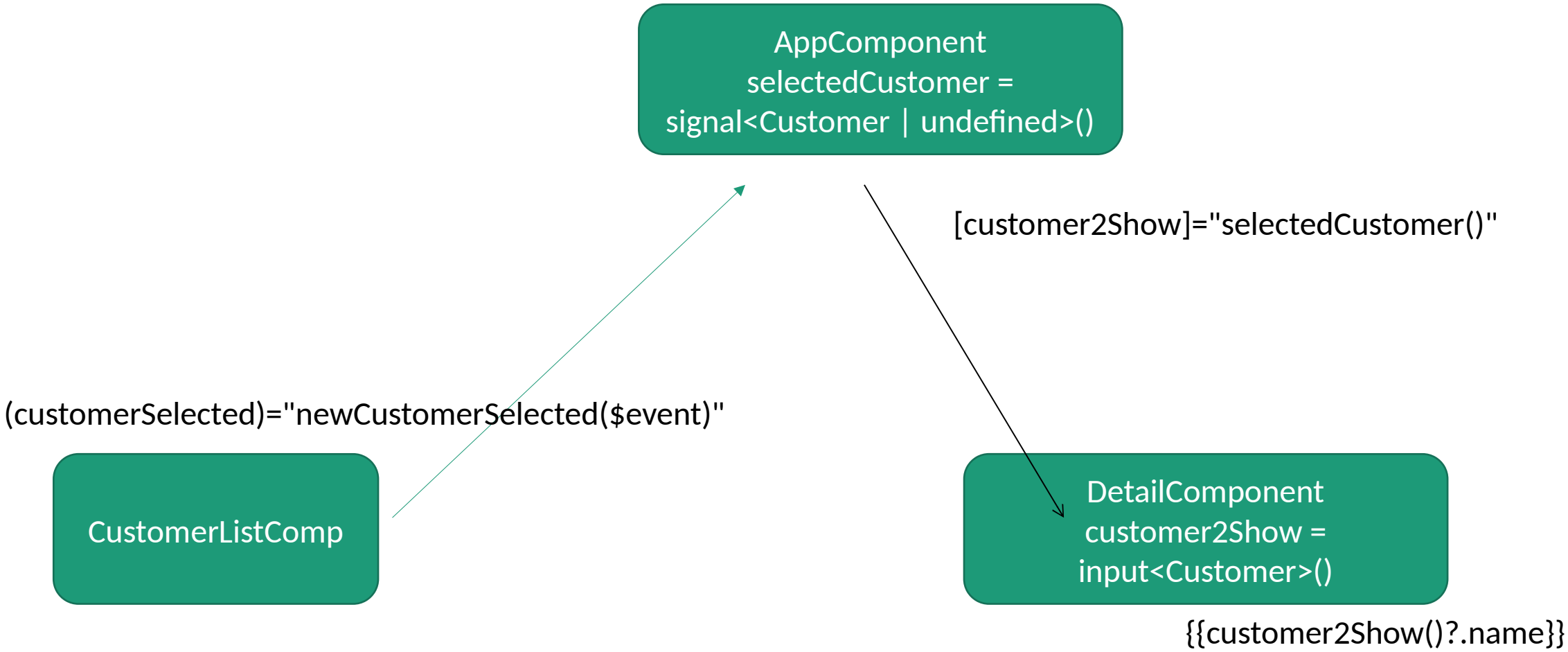
Lokaler State mit Signals

```
import { Component, computed, signal } from '@angular/core';
export class AppComponent {
  selectedCustomer = signal<Customer | undefined>(undefined);
  hasSelection = computed(() => this.selectedCustomer() !== undefined);

  newCustomerSelected(customer: Customer) {
    this.selectedCustomer.set(customer);   // oder .update(old => ...)
  }
}
<!-- Template: Signals werden wie Funktionen gelesen -->
<customer-details [customer]="selectedCustomer()" />
// Signals + zoneless: Angular weiß genau, was sich geändert hat
```

Übung 10: Output

- Erweitere CustomerListComponent
  - Erstelle in CustomerListComponent ein Output customerSelected = output<Customer>()
  - Emitte in CustomerListComponent#listItemClicked customerSelected ein Event
  - Erstelle in AppComponent ein Event-Binding für customerSelected an CustomerListComponent
  - Speichere in AppComponent den zuletzt selektierten Customer in einem signal()
- Erstelle eine Komponente CustomerDetailsComponent
  - Generieren per ng generate component customer/customer-details
  - Erstelle einen Signal-Input customer = input<Customer>() und zeige im Template dessen Properties an
  - Importiere CustomerDetailsComponent im imports-Array von AppComponent
- Verbinde die beiden Komponenten in AppComponent
  - Binde CustomerDetailsComponent in AppComponent ein (parallel zu CustomerListComponent)
  - Reiche das selektierte Customer an CustomerDetailsComponent weiter ([customer]="selectedCustomer()")
  - Zeige in AppComponent die Detail-Ansicht nur an, wenn ein Customer selektiert wurde (NPE vermeiden)


AppComponent
selectedCustomer =
signal<Customer | undefined>()
[customer2Show]="selectedCustomer()"
(customerSelected)="newCustomerSelected($event)"
CustomerListComp
DetailComponent
customer2Show =
input<Customer>()
{{customer2Show()?.name}}

Theorie: Warum Komponenten aufteilen?

- Eine Komponente sollte genau einen Grund zur Änderung haben
- Zu groß: schwer zu verstehen, schwer zu testen, jede Änderung ist riskant
- Zu fein: viele Dateien, viel Durchreichen von Daten, der Überblick geht verloren
- Ziel: so teilen, dass Änderungen lokal bleiben
- Dieser Teil ist reine Theorie
- Praxis dazu: Übungen 7 bis 10 (Komponenten schachteln, Inputs, Events, Outputs)

Theorie: Kriterien für die Aufteilung

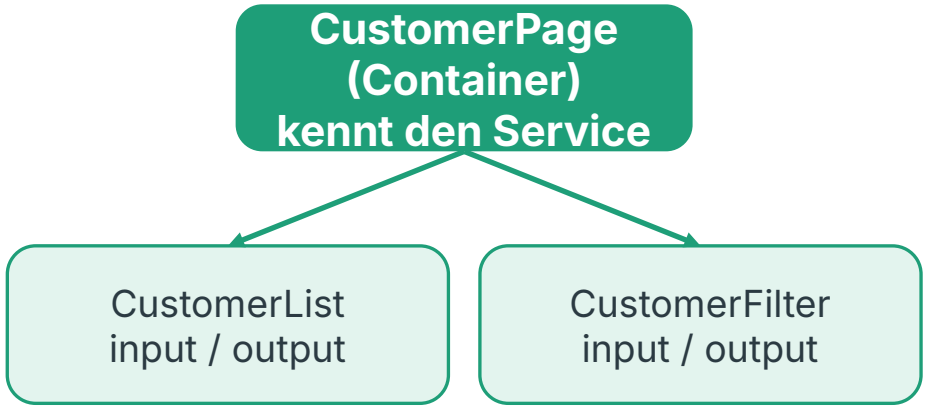
- Verantwortung: mehr als eine Sache? Beispiel: Liste lädt Daten, filtert und zeigt an
- Wiederverwendung: Taucht das Muster mehrfach auf? Beispiel: CardComponent
- Änderungsgrund: Ändern sich Teile aus verschiedenen Gründen? Beispiel: Filter und Anzeige
- Lesbarkeit: Passt das Template auf einen Bildschirm? Warnzeichen: über 100 Zeilen
- Testbarkeit: Braucht ein Test viel Aufbau für wenig Prüfung? Beispiel: 5 Services
- Rendering: Soll ein Teil anders aktualisiert werden? Beispiel: Kopfbereich und 1000 Zeilen

Theorie: Typische Fehlformen

- God Component: alles in einer Komponente (Daten, Logik, Layout)
- Prop Drilling: Daten werden durch viele Ebenen durchgereicht
  - Beispiel: 4 Ebenen, obwohl nur die letzte die Daten braucht
  - Besser: Service oder Content Projection
- Zu feine Teilung: eine Komponente pro span-Element
- Verdeckte Kopplung: Kind greift auf Eltern zu, per inject(ParentComponent) oder über das DOM

Theorie: Container und Darstellungskomponenten

- Container („smart“): kennt Services, lädt Daten, ist selten wiederverwendbar
  - Eingänge: Route und Services; Ausgänge: Aufrufe an Services
  - Test: Service durch einen Mock ersetzen
- Darstellung („dumb“): kennt keine Services, nur input() und output()
  - Oft wiederverwendbar
  - Test: Inputs setzen und Outputs prüfen



Theorie: Darstellungskomponente im Beispiel

```
// Darstellungskomponente: kennt weder Service noch Router
@Component({
  selector: 'customer-filter',
  template: `
    <input [value]="value()"
           (input)="changed.emit($any($event.target).value)">
  `
})
export class CustomerFilterComponent {
  value = input('');           // Eingang: aktueller Filtertext
  changed = output<string>();   // Ausgang: neuer Text nach einer Eingabe
}
```

Theorie: Schnittstellen sauber gestalten

- input.required<T>() für Pflichtdaten, statt im Template auf undefined zu prüfen
- Typen exportieren (z. B. Customer), kein any
- Outputs als Ereignis benennen: customerSelected statt onClick
- Eingaben im Kind nicht verändern; Änderungen als Output melden (oder model() bewusst nutzen)
- Kleine Schnittstelle: lieber 3 klare Inputs als ein riesiges Konfigurationsobjekt
- Kein Zugriff auf Interna des Kindes; wenn nötig, öffentliche Methode oder viewChild()

Theorie: Checkliste „Soll ich aufteilen?“

- 1. Kann ich in einem Satz sagen, was die Komponente tut – ohne das Wort „und“?
- 2. Gibt es Teile, die ich an anderer Stelle wiederverwenden würde?
- 3. Ändern sich Teile aus unterschiedlichen Gründen?
- 4. Ist die Schnittstelle zur Eltern-Komponente klein und typisiert?
- 5. Kann ich die Komponente isoliert testen?



Expression Language

Was wir fast alles schon kennen

Interpolation	{{ expression }}
Property Bindings	[propertyName]="expression"
Event-Bindings	(eventName)="statement"
Two-Way-Data-Binding	[(ngModel)]="assignable expression"
In Control Flow	@if(expression)

Expression

Template Operatoren

- Pipe Operator |
- Elivs Operator als Safe Navigation ?.

Evaluation Kontext

- Globaler Namensraum nicht zugreifbar
- this --> Instanzvariablen der @Component Klasse
- Eigene Template Variablen

Was geht nicht

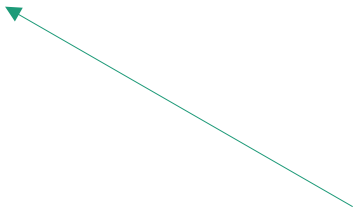
- new Operator
- Bit-Operatoren | und &
- Inkrement bzw. Dekrement

Best Practice

Halte Expressions **einfach und lesbar**
Komplexe Logik → in den Component (am besten als **computed Signal**)

Safe Navigator

- {{bean?.name}}



Falls bean null oder undefined dann keine Exception sondern Leerstring

- Auch mehrfach
  - {{bean?.complexProperty?.name}}



Performanc e Impact

Schlecht, wird bei jeder Change Detection gecalled

```
@if (isUserActive(user)) { ... }
```

Besser

```
// Im Component
activeUser = computed(() =>
this.isUserActive(this.currentUser()));
// Im Template
@if (activeUser()) { ... }
```

One Way Property Bindings bei Images

```
<img [src]="release.cover">
<button [disabled]="!track.playable"></button>
```

Event Binding

- One-Way nach oben im Komponenten-Baum
- $event als Handle auf das Event-Objekt
- Ziele
  - Standard-DOM-Events
    - <input type="text" (keyup)="logInput($event.target.value)">
  - Custom-Events an Komponenten
    - <release (releaseLiked)="highlight($event)"></release>

Style Bindings

- Class-Bindings
  - <div class="selected"
  - <div [class.selected]="isSelected(track)">
- Style-Bindings
  - <img [style.visibility]="release.cover ? 'visible' : 'hidden'">
- Alternativ: [class]="{ selected: isSelected(track) }" bzw. [style]-Bindings (ngClass/ngStyle nicht mehr nötig)

Data Binding in Formularen

- Ziel:
  - Formularfelder an Properties binden
- Manuelles Synchronisieren würde bedeuten
  - <input [value]="bean.name"
  - (input)="bean.name=$event.target.value">
- Zwei-Wege-Data-Binding mit ngModel
  - <input [(ngModel)]="bean.name">

- *Merke: Banana in a Box [()]*

Deferrable Views: @defer

```
@defer (on idle) {
  <customer-details [customer]="selectedCustomer()" />
} @placeholder {
  <div>Lade Kundendetails...</div>
} @loading (minimum 500ms) {
  <p>Lädt...</p>
} @error {
  <p>Fehler beim Laden</p>
}
// Trigger: on idle | on viewport | on interaction | on hover | on timer(2s) | when bedingung
// Die Komponente landet in einem eigenen Bundle und wird erst bei Bedarf geladen
```

Übung 11: Bindings auf Images und mehr

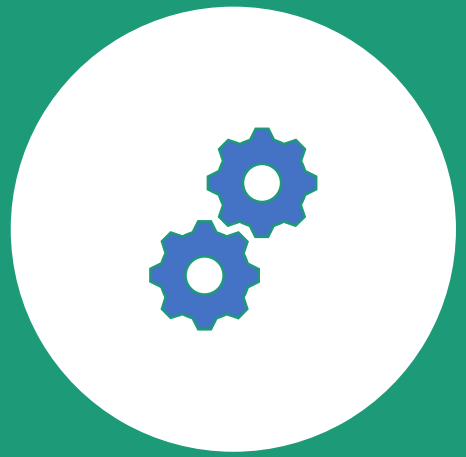
- Erweitere das Datenmodell
  - Füge der Klasse Customer eine optionale Eigenschaft imageUrl: string hinzu
  - Nehme imageUrl im Konstruktor als Parameter entgegen und setzte das Feld
- Passe AppComponent an
  - Kopiere aus der Musterlösung (11_image) die Bilder aus src/assets/images/customer
  - Setze beim Erzeugen der Testdaten bei manchen Customer ein Bild.
    Pfad z.B. assets/images/customer/carl.svg
- Erweitere die Darstellung in CustomerDetailsComponents
  - Zeige das Bild mit einer festen Größe an, falls vorhanden
  - Blende das Bild aus, falls kein Bild vorhanden
  - Optional: Lade CustomerDetailsComponent in AppComponent per @defer (on idle) mit @placeholder

Übung 12: Class Binding

- Erweitere das Datenmodell
  - Lege in Customer ein Feld discontinued: boolean an, das per Konstruktor gesetzt wird
  - Erzeuge in AppComponent mindestens ein Customer, an den nichts mehr geliefert wird
  - Setze für diesen Customer discontinued auf true
- Passe die Darstellung in CustomerListComponent an
  - Setze an nicht länger gewünschte Customer eine Klasse
  - Stelle nicht länger gewünschte Customer anders dar
    (über Klasse an Element und CSS in customer-list.component.css, z.B. ausgegraut oder durchgestrichen)
- Hebe in der Liste das selektierte Customer hervor
  - Speichere in CustomerListComponent den zuletzt selektierten Customer in einem signal()
  - Prüfe am <li> ob der aktuelle Customer das selektierte ist und setze ensprechend eine Klasse

Übung 13: Formulare

- Biete in der Detail-Ansicht die Möglichkeit an, den Namen eines Customers zu verändern
  - Importiere FormsModule aus @angular/forms im imports-Array von CustomerDetailsComponent
  - Erstelle ein input mit ngModel



Services

Eigenschaften eines Services



Services sind TypeScript-Klassen



Diese werden ausschließlich vom Angular-Framework instanziert

Diese Aufgabe übernimmt der Angular-Context



Benötigt eine Component den Zugriff auf einen Service, so wird dieser automatisch gesetzt

Dieses Verfahren heißt Dependency Injection



Insgesamt stellt Angular mit dem Service-Mechanismus ein Dependency-Injection-Framework zur Verfügung

Services? Was

Sehr allgemeines Konzept

- Irgendwelche Funktionalität kapseln
- Nicht UI spezifisch
- Implementierungsdetails verstecken

Einsatzmöglichkeiten

- Höhere Abstraktion schaffen (REST)
- Integration anderer Bibliotheken (WebSocket)
- Datenhaltung, Datenzugriff, Caching, Logik, ...

Können Abhängigkeit zu anderen Services haben

Aufgaben eines Services

- Standard-Services des Angular-Frameworks stellen nützliche Routinen zur Verfügung
  - Beispielsweise wird die Client-Server-Kommunikation über den `HttpClient`-Service realisiert
- Eigene Services werden in der Anwendung häufig benutzt, um in einer sauberen Architektur ein Datenmodell zu definieren
  - Dieses wird dann von allen Components konsistent benutzt
  - Services ermöglichen damit eine Inter-Component-Kommunikation, die unabhängig von Input-Parametern sind
    - Wesentlich leichter wartbar

Ein simpler Service

- Die Erzeugung eines Services kann das Angular-CLI übernehmen
  - ng generate service <ServiceName>
- Ein Service selbst ist eine @Injectable-Klasse
  - @Injectable({ providedIn: 'root' }) – anwendungsweiter Singleton, tree-shakeable
- Nutzung in jeder anderen Angular-Klasse per inject() (oder Konstruktor)
  - Components
  - Andere Services, Guards, Interceptors
- Alternativen zur Registrierung
  - providers in bootstrapApplication (main.ts)
  - providers der @Component-Annotation
  - https://angular.dev/guide/di

Methode in Services

```
import {Artist} from "./artist.model";


export class ArtistService {
    getAllArtists(): Artist[] {
            return [];
    }
    }
```

API Stabilität

- Service versteckt Details über Datenhaltung vor Komponenten
- API stabil, Datenquelle austauschbar
- Best Practice: Mock-Daten parallel zu Service in eigene Datei ablegen

```
import {Artist} from "./artist.model";
import {ARTISTS} from './artist.mock';

export class ArtistService {
    getAllArtists(): Artist[] {
        return ARTISTS;
    }
}
```

Objektnetze in OO- Systemen

- Aufbau der Netze komplex.
- Testbarkeit der Komponenten wird eingeschränkt.
- Änderbarkeit und Wartbarkeit nur bedingt gegeben.
- Favor Delegation over inheritance.
- Verschiedene Möglichkeiten Netze aufzubauen.

Assignment by Third Party

```
public static void main(String[] args)
{
    MyService service = new MyService();
    service.setDAO(new MyDAO());
}
```

Creating itself - DIRECT

```
public MyService()
{
	this.myDao = new MyDao();
}
```

Factory- Pattern

```
public MyService()
{
  this.myDao = Factory.getMyDao();
}
```

Dependency Injection

- Objekte werden zur Laufzeit „injiziert“.
- Objekte werden passiv, statt sie zum Beispiel mit einer Factory aktiv zu erzeugen.
- Abhängigkeiten sind explizit an der Schnittstelle durch entsprechende set-Methoden zu erkennen.
- Es können beliebige andere Objekte zugewiesen werden(z.B. Mocks / an Bedürfnisse angepasste Objekte).
- Sehr flexibel wenn Objekte nur noch von Interfaces abhängen.
- Externe Konfiguration der Beziehungen zwischen Objekten.
- Objekte bekommen ihre abhängigen Objekte statt sie selbst zu suchen.

Beispiel: Service

```
1  import { Injectable } from '@angular/core';
2
3  @Injectable()
4  export class PeopleModelService {
5
6    constructor() { }
7    people: Person[] = [new Person("Gärtner", "Hans")
8
9  }
```

@Injectable macht diese Klasse dem Angular-Context bekannt

```
providers: [
  PeopleModelService
],

export class ServiceAppComponent {
  constructor(readonly peopleModel: PeopleModelService ){

  }
  people:Person[] =  this.peopleModel.people
}
```

Im Modul oder der `@Component`-Annotation

Dependency Injection in einer Component

DI Konfig

- Woher weiß Angular was zu injizieren ist?
- Muss konfiguriert werden: Providers
  - Injector → Provider → Instance
- Konfiguration direkt am Service

```
@Injectable({ providedIn: 'root' })
export class TrackService { }
```

- oder global in main.ts

```
bootstrapApplication(AppComponent, { providers:
[ TrackService ] });
```

- Wie sieht ein Provider aus?

Wie geht DI in Angular

- Direkte Abhängigkeit
  - einfach nur die Klasse angeben: [ConsoleLogger]
  - Kurz für [{ provide: ConsoleLogger, useClass: ConsoleLogger}]
- Allgemein: Provider-Objekt
  - provide: Unter welchem Namen steht es zur Verfügung → Klasse oder InjectionToken
  - Ein Rezept, wie es erzeugt werden soll → useClass, useValue, useFactory oder useExisting

```
bootstrapApplication(AppComponent, {
  providers: [
    { provide: ConsoleLogger, useClass: ConsoleLogger }, // same as shortcut
    { provide: Logger, useExisting: ConsoleLogger } // different token, same instance
  ]
});
```

Moderne Dependency Injection mit inject()

```
import { Component, inject } from '@angular/core';

@Component({ selector: 'app-root', templateUrl: './app.component.html' })
export class AppComponent {
  private readonly customerService = inject(CustomerService);
  customers = this.customerService.getAll();
}
// Früher: constructor(private customerService: CustomerService) {}

// inject() funktioniert auch in Funktionen (Injection Context):
export const authGuard: CanActivateFn = () ⇒ inject(AuthService).isLoggedIn();
// Vorteile: weniger Boilerplate, einfache Vererbung, funktionale Guards & Interceptors
```

Kontextbasierte Injektion

- Injektoren bilden einen Baum
  - Jede Komponente ein Child-Injector vom Injector der Vater-Komponente
  - Instanzen sind Singletons in einem Injektor
- Provider für Child-Injektoren können an Komponenten definiert werden
  - Child-Injector kann Provider des Vaters überschreiben / ergänzen
  - providers Property in @Component Decorator
- Einsatzmöglichkeiten
  - Auflösen von Namenskonflikten
  - Unterschiedliche Konfiguration (z.B. Log-Level)

Übung 14

- Erstelle einen Service CustomerService
  - Generieren per ng generate service customer/customer
- Erstelle Mock-Daten
  - Erstelle eine Datei src/app/customer/customer.mock.ts
  - Exportiere darin ein Array von Customer (kopieren aus AppComponent)
- Verwende im Service die Mock-Daten
  - Importiere die Mock-Daten aus customer.mock
  - Erstelle in CustomerService eine Methode getAll(): Customer[]
  - Gebe darin die Mock-Daten zurück
- Verwende den Service noch nicht!

Übung 15

- Lasse CustomerService in AppComponent injizieren
  - Stelle den CustomerService über @Injectable({ providedIn: 'root' }) zur Verfügung (Default von ng generate)
  - Injiziere CustomerService in AppComponent per inject(CustomerService)
- Verwende CustomerService in AppComponent um die Daten zu beziehen
  - Lösche den Code zum Erzeugen der Test-Daten in AppComponent
  - Rufe CustomerService#getAll auf um Daten zu bekommen

Komponenten Lebenszyklus

- Komponenten haben einen Lebenszyklus
  - **Aufruf des Konstruktors**
  - **ngOnChanges**
  - **ngOnInit**
  - ngDoCheck
  - ngAfterContentInit
  - ngAfterContentChecked
  - ngAfterViewInit
  - ngAfterViewChecked
  - [ngDoCheck und ggf. ngOnChanges]*
  - **ngOnDestroy**

Was soll das?

- Bestimmte Informationen stehen erst später zur Verfügung
  - ID des Trainings beim Routing
  - Kind-Komponenten
- Instanziierung und Initialisierung trennen
  - Gut für Testbarkeit

- Achtung: keine Lifecycle-Hooks für Services

Best Practices: OnInit

```
import { Component, OnInit, inject } from "@angular/core";
@Component({
  selector: "music-app", template: `<h1>Music App</h1>`,
  providers: [ ArtistService ]
})
export class MusicAppComponent implements OnInit {
  private readonly service = inject(ArtistService);
  public artists: Artist[] = [];
  ngOnInit() {
    this.artists = this.service.getAll();
  }
}
```

Übung 16: OnInit

Initialisiere die Daten in AppComponent im ngOnInit Hook statt im Konstruktor

- Implementiere das Interface OnInit aus @angular/core
- Verschiebe das Laden der Daten aus dem Konstruktor in die Hook-Methode

Pipes

Daten für UI aufbereiten

Allgemein

- Daten-Transformation für das UI
  - Kleine, wiederverwendbare Operationen für das UI
  - An Funktionale Programmierung angelehnt
    - Daten und Konfiguration per Parameter
    - Pipe verändert ursprünglichen Wert nicht
- Pipes werden an Expression mit | angehängt
  - {{ *property* | json }}
- Pipes können per : konfiguriert werden
  - {{ release.releaseDate | date:'dd.MM.yyyy' }}
- Pipes können verkettet werden
  - {{ release.releaseDate | date | uppercase }}

Was sind Pipes?

- Pipes formatieren das Ergebnis beispielsweise einer Interpolation
  - `{{dateOfBirth | date}}`
    - Hier wird das Geburtsdatum als Datum formatiert dargestellt
  - Pipes sowie der Operator | sind aus der Linux-Welt übernommen
- Vordefinierte Pipes umfassen
  - `date[:format[:timezone[:locale]]]`
  - `currency[:currencyCode[:display[:digitInfo[:locale]]]]`
  - `lowercase`
  - Vollständige Liste und Dokumentation unter *`https://angular.io/api?status=stable&type=pipe`*

Decimal Pipe

- Formatiertung von Zahlenwert
- Parameter: Anzahl Nachkommastellen

  {{ track.size | decimal:2 }}

Date Pipe

- Für Datum-String, Date-Object, Timestamp-Number
- Nimmt als Parameter Shortcut (z.B. short) oder Format-String entgegen
- Kurzschreibweise: Zusammenspiel mit locale des Browsers

```
{{ release.releaseDate | date:'short' }}
{{ release.releaseDate | date:'dd.MM.yyyy' }}
```

Übung 17: Pipe

- Erweitere das Datenmodell
  - Lege in Customer ein Feld / Konstruktor-Parameter lastOrderDate vom Typ Date an
- Passe die Mock-Daten an
  - Setze für jeden Customer ein unterschiedliches Datum (new Date(year, month - 1, day))
  - Für mind. 1 Customer weniger als 1 Woche zurück
  - Setze für nicht länger gewünschte Customer natürlich ein Datum in der weiteren Vergangenheit
- Erweitere das Template von CustomerDetailsComponent
  - Gib das Datum lastOrderDate formatiert aus
  - Probiere verschiedene Formattierungen aus (Pattern, Shortcuts)

Pipes sind Klassen

- Pipes sind Klassen
  - Implementieren Interface PipeTransform aus @angular/core
  - Werden mit @Pipe Decorator aus @angular/core markiert

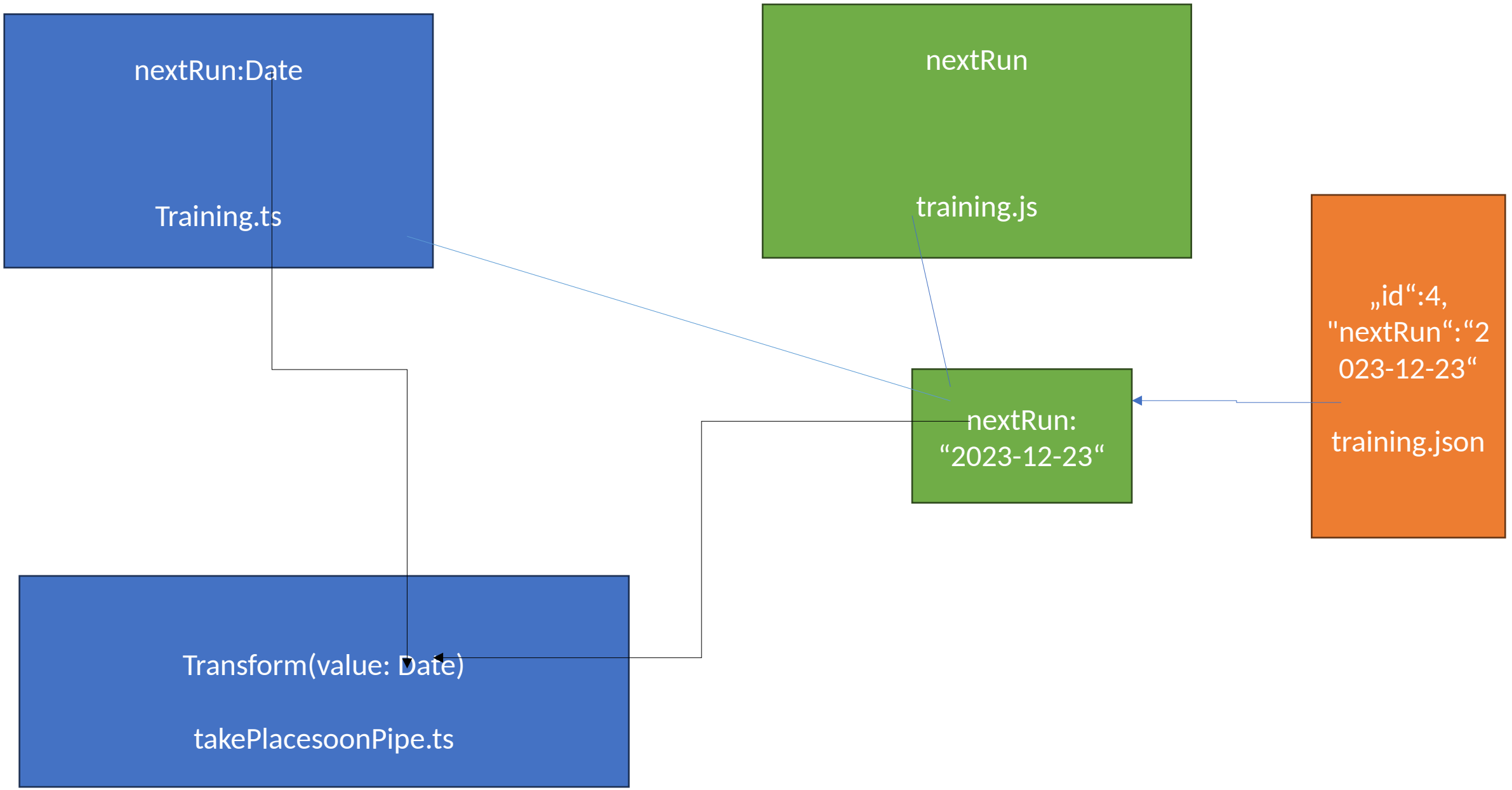
```
import {Pipe, PipeTransform} from "@angular/core";
@Pipe({ name: "hasLikes" })
export class HasLikesPipe implements PipeTransform {
        transform(withLikes: {likes :number}): boolean {
        return withLikes.likes > 0;
        }
}
```

Verwendung von Pipes in Komponenten

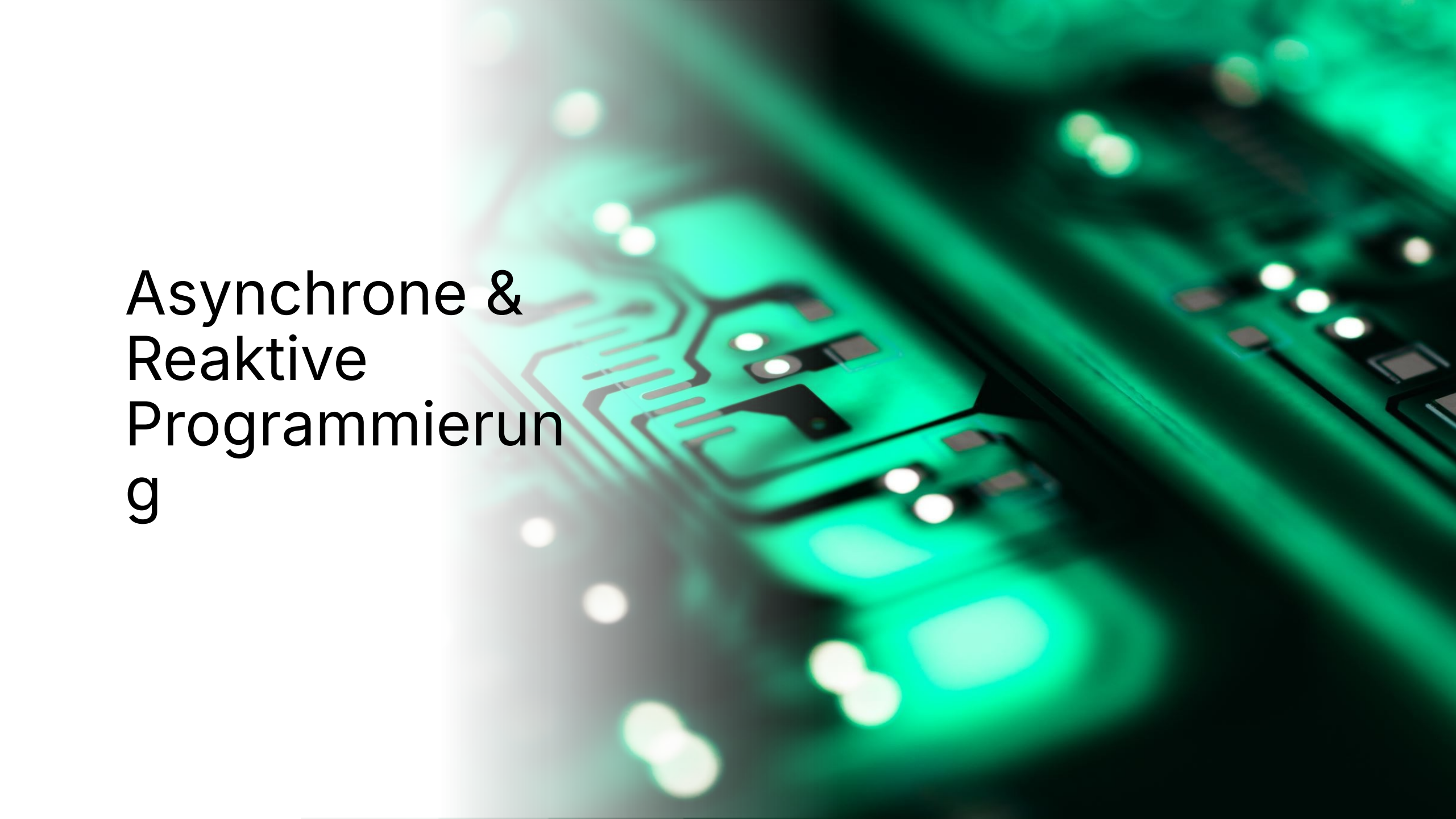
```
import { DatePipe } from '@angular/common';
import { HasLikesPipe } from './has-likes.pipe';
@Component({
  selector: 'release-card',
  imports: [ HasLikesPipe, DatePipe ],
  template: `@if (release() | hasLikes) { <span>❤</span> }`
})
export class ReleaseCardComponent { ... }
// Pipes sind standalone – Import direkt in der verwendenden Komponente
// Legacy: declarations/exports im NgModule (siehe Anhang)
```

Übung 18: Eigene Pipe

- Schreibe eine eigene Pipe currentActiveCustomers
  - Generieren per ng generate pipe customer/current-active-customers-pipe
  - Implementiere PipeTransform mit der Methode transform(date: Date, daysInPast = 7): boolean
  - Gib true zurück, falls der Cutomer innerhalb der letzten x Tage bestellt hat (Parameter soon)const timeDiff = Date.now() - date.getTime().
  - const maxTimeDiffInMS = daysInPast * 1000 * 60 * 60 * 24;
  - const isActive = timeDiff < maxTimeDiffInMS;
- Wende die currentActiveCustomers Pipe auf das lastOrderDate Datum in CustomerDetailsComponent an (Pipe im imports-Array importieren)
  - Verpacke die Ausgabe des lastOrderDate Datums in einen div-Container
  - Setze an diesem div eine Klasse, wenn der Customer aktiv ist ([class.active]="...")
  - Übergebe z.B. 5 (Tage) als Parameter daysInPast an currentActiveCustomers
  - Stelle über CSS mithilfe der gesetzten Klasse Customer, die active sind, anders dar (z.B. rote Schrift)


nextRun:Date
Training.ts
nextRun
training.js
„id“:4,
"nextRun“:“2
023-12-23“
training.json
nextRun:
“2023-12-23“
Transform(value: Date)
takePlacesoonPipe.ts

Asynchrone & Reaktive Programmierung



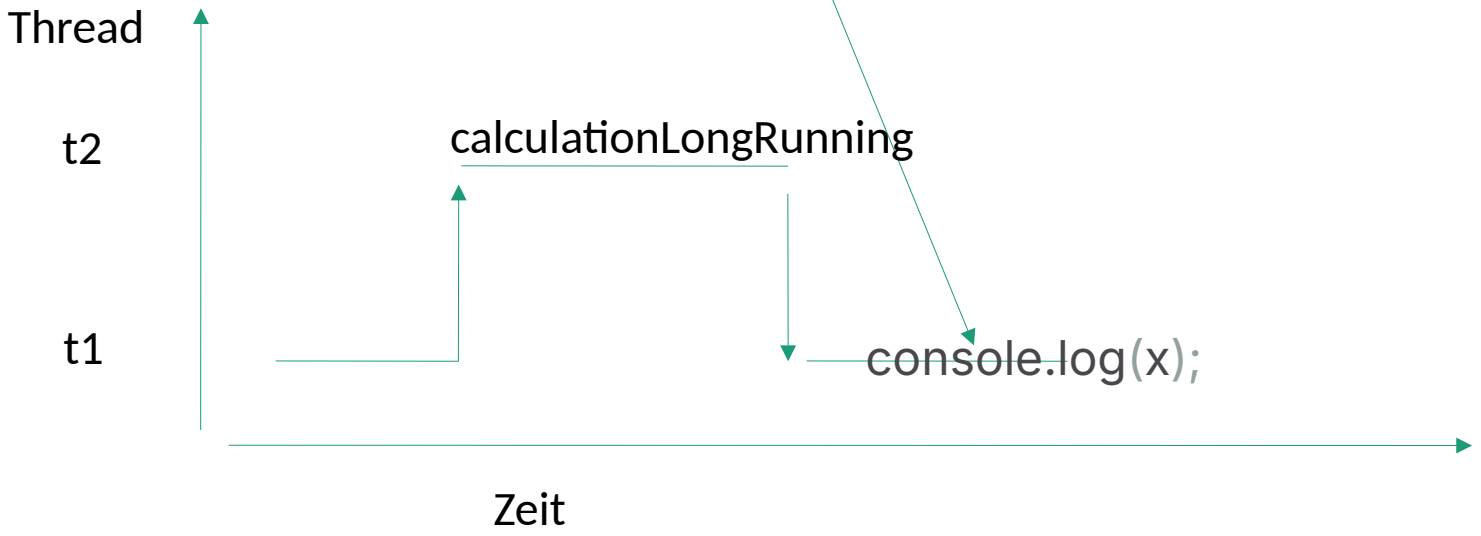
Welche Eigenschaft hat die Variable X

```
var x = 1;
console.log(x);
```

```
var x = [1, 2, 3];
x.forEach(alert);
```

Und hier?

```
asyncFunction(function callback(x) {
        console.log(x);
});
```



Threading im Browser

- Javascript per Definition Single-Threaded
- Event-Loop Thread mit Queue, die vom Interpreter abgearbeitet wird
- Beispiel: Ajax Aufruf
  - JavaScript startet HTTP Request
  - Interpreter wartet nicht bis Server geantwortet hat, UI würde einfrieren
  - Interpreter arbeitet andere Sachen ab (Event-Loop)
  - Nach Response, wird weiterer Code in Event-Loop-Queue eingestellt
  - Code läuft erst wenn in Event-Loop an der Reihe

Callback API

- Asynchroner Funktion wird Callback als zusätzlicher Parameter mitgegeben
- Callback wird aufgerufen sobald Ergebnis bereit steht
- Probleme
  - Keine standardisierte API (Wie werden Fehler behandelt?)
  - Mehrmalige oder spätere Nutzung kompliziert ("Callback-Hell")
  - Abhängigkeit von mehreren asynchronen Ergebnissen sehr schwierig zu handhaben

Lösung sind Promises

- Stellvertreter für Ergebnis, das zunächst unbekannt ist (z.B. http-Aufruf)
- Asynchronen Funktionen geben Promise zurück, kein Callback als Parameter
  - Können erfüllt oder abgelehnt werden
  - Bieten API für success- und error-Funktionen
- Promise kann genau ein Mal aufgelöst werden
  - 🚫 *Nicht geeignet für mehrere asynchrone Werte*

Promise Beispiel

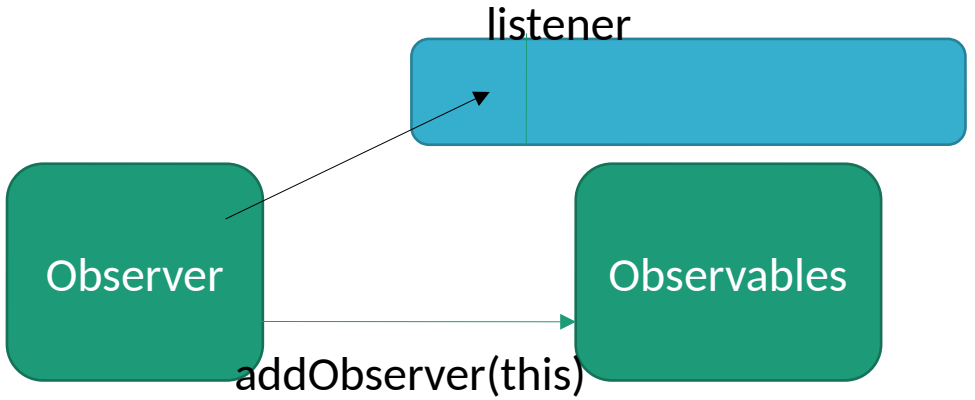
```
// call to async function returns a promise
const promise = doSomethingAsynchronous();
promise.then(
        (result) ⇒ { return filter(result) }, // success handler
        (error) ⇒ { } // error handler (optional)
).then((result) ⇒);
```

Observable als Promise++

- Observables sind Streams von Events und Daten
  - Aber auch Fehlern

- --a---b-c---d---X|
  - -- Zeitline
  - | Endsignal
  - a,b .. Ereignisse

- Schönere und interaktive Visualisierung
- https://rxmarbles.com




```
for(l in listener){
  l.notify(oldWeight,
this.weight)
}
```

Methoden für Zwischenverarbeitung

- Häufig genutzte Methoden:
  - map für Transformation
  - debounce für Verzögerung
  - filter zum Aussortieren
- Jede Transformation gibt wieder Observable zurück

--a---b-c---d---|

map to alphabet index

--1---2-3---4---|

filter > 2

--------3---4---|

Observables sind lazy

- Observables brauchen Abnehmer
  - forEach(dataHandler): Promise
  - subscribe(dataHandler, errorHandler?): Subscription
  - Handler: Funktionen mit Seiteneffekten
- Intermediate Transformationen werden nur ausgeführt wenn Abnehmer da ist

--a---b-c---d---|

map to alphabet index

--1---2-3---4---|

filter > 2

--------3---4---|

subscribe console.log

Intermediate vs Terminal

intermediate	terminal
filter	count
map	foreach
sorted	sum
distinct	min/max
limit	reduce
peek	findFirst

Unsubscribe Nicht vergessen

- subscribe() gibt Subscription zurück
- Subscription hat unsubscribe() Methode
- Wenn completed, werden alle intern alle Handler gelöscht

Nutze ngOnDestroy für unsubscribe

Modern: takeUntilDestroyed() aus @angular/core/rxjs-interop oder toSignal() – unsubscribe automatisch

Automatisches Unsubscribe

```
{{ aPromise | async }}
{{ anObservable | async }}
<trainingList [trainings]="releaseObservable$ | async"></
trainingLis
```

Training[]

Observable<Training[]>

Alternative ohne async-Pipe: toSignal() (siehe RxJS Interop)

Reactive Extensions – RxJS

- Entwickelt von MS für verschiedene Sprachen
- Implementierung von Observables
  - Wird in Angular verwendet
  - http://reactivex.io/

RxJS Interop: toSignal()

```
import { toSignal } from '@angular/core/rxjs-interop';

export class AppComponent {
  private readonly customerService = inject(CustomerService);
  // Observable → Signal, Subscription wird automatisch beendet
  customers = toSignal(this.customerService.getAll(), { initialValue: [] as
Customer[] });
}
<!-- Template: keine async-Pipe nötig -->
<customer-list [customers]="customers()" />
// Gegenrichtung: toObservable(signal)
// RxJS für asynchrone Streams (HTTP, Events) – Signals für den Template-State
```

Übung 19: Async CustomerService

- Verpasse CustomerService eine Observable basierte API
  - Lege im Service ein ReplaySubject als Feld an
  - Rufe im Konstuktor am Subject next mit den Mock-Daten auf
  - Lasse getAll das Observable<Customer[]> des Subject zurückgeben
- Passe alle Aufrufe an (toSignal(), alternativ subscribe oder async-Pipe)
  - Eigentlich nur:
    - AppComponent: customerMockList = toSignal(customerService.getAll(), { initialValue: [] })
    - app.component.html: <customer-list [customers]="customerMockList()">

Routing und Navigation

Arbeitsweise

- Routen definieren Pfade der Anwendung
- Jede Route verbindet einen Pfad mit einer Component
  - Auch Redirects können definiert werden
  - Die Pfade unterstützen Platzhalter
- Die Routen werden in app.routes.ts definiert und per provideRouter(routes) in main.ts für die gesamte Applikation bereitgestellt
  - dazu werden Klassen importiert
  - `import { Routes, provideRouter } from '@angular/router'`
- Routen-Pfade müssen nicht disjunkt sein
  - Der erste Treffer eines Pfads wird benutzt
  - Damit sind in der Routen-Definition speziellere Pfade vor allgemeinen zu platzieren

Features von Routen

- Deep Linking über URL
- Browser Historie (Vor / Zurück)
- Links über symbolische Namen, nicht über hart codierte URLs
- Pfad- oder Hash-Navigation
- Hierarchischer Aufbau, nicht zwingend absolute Routen

Routendefinition

- Route
  - Relativer path (ohne führenden Slash)
  - component für die Anzeige

```
import { Routes } from '@angular/router';


export const routes: Routes = [
{ path: 'release', component: AllReleasesComponent },
{ path: 'artist', component: AllArtistsComponent }
];
```

Ablauf

- URL des Browsers wird überprüft / auf Änderungen überwacht
- Passende Route wird gesucht
- Komponente wird angezeigt

Verlinken von Routen

- Dazu gibt es die `routerLink`-Direktive
- Die anzuzeigende Seite wird in einem `router-outlet`-Element angezeigt

```
@Component({
        selector: "music-app",
        template: `
        <h1>Music App</h1>
        <router-outlet></router-outlet>`
})
export class MusicAppComponent {}
```

Default / Fallback Route

- Keine gültige Route gefunden?
  - Einstieg über URL ohne Route
  - ungültige Route
- Fallback konfigurierbar
  - path mit **
  - redirectTo: auf anderen Pfad
  - Achtung: Reihenfolge der Routen spielt eine Rolle,
    - **Fallback als Letzte**

Routenkonfiguration

- provideRouter(routes, ...features) erzeugt die Provider für den Router-Service
- Aufruf einmalig in bootstrapApplication (main.ts)
- Features als optionale Funktionen
  - withComponentInputBinding() – Routen-Parameter als Inputs
  - withHashLocation(), withDebugTracing(), withPreloading(...)
- Lazy Loading per loadComponent / loadChildren
- Legacy: RouterModule.forRoot / forChild im AppModule (siehe Anhang)

Beispiel: Routen-Definition

```
// app.routes.ts
export const routes: Routes = [
  { path: 'path1', component: Component1 },
  { path: 'lazy', loadComponent: () => import('./lazy.component').then(m =>
m.LazyComponent) },
  { path: '', redirectTo: '/index', pathMatch: 'full' },
  { path: '**', component: PageNotFoundComponent }
];
// main.ts
bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes, withDebugTracing()) // <-- debugging purposes only
  ]
});
```

Location-Strategy

- LocationStrategy festlegen
  - PathLocationStrategy (default)
    - Bessere SEO Indizierung
  - HashLocationStrategy: default AngularJS 1
    - Simpler
- Verwendung von PathLocationStrategy
  - <base href="/"> als erstes Element im head der index.html
- import { provideRouter, withHashLocation } from '@angular/router';
- import { routes } from './app/app.routes';
- bootstrapApplication(AppComponent, {
- providers: [provideRouter(routes, withHashLocation())]
- });

Übung 20: Routen für Listenansicht

- Bereite die Anwendung auf Routing vor
  - Erstelle eine Datei src/app/app.routes.ts
  - Definiere und exportiere eine Konstante des Typs Routes (export const routes: Routes = [])
  - Registriere die Routen in main.ts per provideRouter(routes)
  - Importiere RouterOutlet im imports-Array von AppComponent
- Erstelle eine Route für die Listen-Ansicht
  - Füge in app.routes.ts eine Routen-Konfiguration für path: "customer" und component: CustomerListComponent hinzu
  - Erstelle auch eine Redirect-Route für path: "**“ mit redirect auf customer
  - Lösche CustomerListComponent und CustomerDetailsComponent aus dem Template von AppComponent
  - Füge in das Template von AppComponent ein router-outlet ein
- Passe CustomerListComponent an die Veränderungen an
  - Daten können nicht mehr per Input übergeben werden, da keine direkte Verwendung
  - Lade in CustomerListComponent die Daten vom Service (inject() + toSignal())
  - Kein ngOnDestroy nötig – toSignal() beendet die Subscription automatisch
  - Lösche das Laden der customers aus AppComponent

Pfadparameter für dynamische Information

Problem:

- Routen Konfiguration zur Entwicklungszeit
- Somit: Keine statische Route pro Datensatz möglich

Lösung:

- Dynamische Routen mit Parametern
- Routen Konfiguration kann in Pfadvariablen enthalten

Parameter vs URL-Parameter

- Parameter in Route für Pflichtparameter
- URL-Parameter für optionale Parameter

Beispiel Pfadparameter

```
{
    path: 'artist/:id',
    component: ArtistDetailsComponent
    }
```

Details zu den Routen

- Modern: Routen-Parameter, Query-Parameter und Route-Data direkt als Input der Komponente
  - `provideRouter(routes, withComponentInputBinding())`
  - `id = input<string>() in der Ziel-Komponente`
- Alternativ: ActivatedRoute per inject() – Zugriff auf die Routen-Definition, die zum Aufruf geführt hat
  - `url, paramMap, queryParamMap, data, …`
- Router-Events werden an interessierte Listener delegiert
  - `NavigationStart, RoutesRecognized, NavigationEnd, …`
- Details unter https://angular.dev/guide/routing

Beispiel: Component Input Binding

```
// main.ts
provideRouter(routes, withComponentInputBinding())
// app.routes.ts
{ path: 'artist/:id', component: ArtistDetailsComponent }
// artist-details.component.ts
export class ArtistDetailsComponent {
  private readonly service = inject(ArtistService);
  id = input<string>();          // wird aus :id befüllt
  artist = computed(() => ...);   // reagiert auf Änderungen von id()
}
```

Weiterverarbeitung von Routenparametern

```
export class ArtistDetailsComponent {
  private readonly service = inject(ArtistService);
  id = input.required<string>();
  artist = toSignal(
    toObservable(this.id).pipe(
      map(id ⇒ parseInt(id)),
      switchMap(id ⇒ this.service.getById(id))
    )
  );
}
// Legacy: this.route.params.pipe(map(...), switchMap(...)).subscribe(...)
```

Navigation im html

```
1.<a [routerLink]="['/artist']">Artists</a>
2.<a [routerLink]="['/artist', artist.id]">Some Artist</a>
3.<a [routerLink]="[artist.id]">Artist with ID</a>
```

- Absolute Links mit führendem Slash (1,2)
- Zur aktuellen Route relative Links ohne führenden Slash (3)

Navigation im Code – Component Klasse

```
private readonly router = inject(Router);
save() {
  // Navigate back to list
  this.router.navigate(['artist']);
}
```

Übung 21: Routing mit Parametern

- Erstelle eine Route für die Detail Ansicht
  - Konfiguriere eine Route für /customer/:id die auf die CustomerDetailComponent verweist
  - Erweitere CustomerService um eine Methode getById : Observable<Customer>
  - Lese in CustomerDetailsComponent den Routen-Parameter per Signal-Input id = input<string>() aus (withComponentInputBinding() in main.ts)
  - Hole das Customer für die Anzeige per ID vom Service
- Verlinke die Listen-Ansicht mit der Detail-Ansicht
  - Importiere RouterLink im imports-Array von CustomerListComponent
  - Füge den Listen-Elementen einen routerLink hinzu
  - Navigiere im routerLink zur Detail-Ansicht für das Element
- Verlinke die Detail-Ansicht mit der Listen-Ansicht
  - Füge im Template von CustomerDetailsComponent einen Link zurück zur Liste ein
  - Verwende einen Button mit einem Click-Handler
  - Navigiere im Handler per router.navigate (Router per inject())



Formulare und Eingabefelder

Bearbeiten von Daten

Rückblick auf Standard html

- Formular (form) einfacher Aufbau
  - Mit Feldern (input, select, ..)
  - FieldSet für visuelle Gruppierung
- Validierung mittels HTML 5 (teilweise)
- Posten der Daten mittels Pagereload oder AJAX

Formulare inAngular

Arten

- Template-Driven
- Model-Driven

Interne Forms API

- Binding (bzw. ChangeDetection)
- Validierung
- Aggregation

Formulare, Gruppen, Felder

- Ein Formular = Gruppe von Feldern + Submit-Aktion
- Der Zustand der Gruppe hängt von den enthaltenen Feldern ab
  - z. B. Gruppe ungültig, falls mindestens ein Feld ungültig ist
- Ein Feld ist ein Wert + weitere Zustände: dirty, touched, valid
- Zwei Arten in Angular: Template-Driven Forms (Hauptpfad), Reactive Forms (Wahlthema)

Template-Driven Forms: FormsModule und ngModel

- Template-Driven Form: das Formular wird im Template beschrieben, die Klasse bleibt schlank
- FormsModule aus @angular/forms im imports-Array der Komponente importieren
- Jedes Feld bekommt die Direktive ngModel und ein name-Attribut
  - Innerhalb eines form-Tags ist das name-Attribut Pflicht, sonst gibt es einen Fehler
- Template-Referenz #f="ngModel" gibt Zugriff auf den Zustand eines Feldes
  - z. B. f.invalid, f.dirty, f.touched, f.errors
- Angular legt das Formular (NgForm) und die Felder automatisch im Hintergrund an
- Im Hauptpfad des Kurses bleiben wir bei dieser einfachen Variante

customer-form.html

```
<form>
  <input name="firstname"
    [(ngModel)]="draft.firstname"
    #f="ngModel" required>
  @if (f.invalid && f.dirty) {
    <small>Pflichtfeld</small>
  }
</form>
```

Template-Driven Form: Aufbau im Template

```
@if (draft(); as c) {
  <form #customerForm="ngForm">
    <input name="firstname" [(ngModel)]="c.firstname">
    <input name="city" [(ngModel)]="c.city">

    <p>Formular: {{ customerForm.valid ? 'gültig' : 'ungültig' }}</p>
  </form>
}
<!-- c: bearbeitbare Kopie des Kunden, das Original bleibt unverändert -->
<!-- #customerForm="ngForm": Zugriff auf das ganze Formular, z. B. .valid -->
```

Bearbeitungskopie mit linkedSignal

```
// Der geladene Kunde (nur lesen!)
customer = computed(() ⇒ {
  const id = this.id();
  return id ? this.customerService.getById(id) : undefined;
});

// Bearbeitbare Kopie: folgt dem Original, kann aber überschrieben werden
draft = linkedSignal(() ⇒ {
  const customer = this.customer();
  return customer ? { ...customer } : undefined;
});
// Wechselt customer (z. B. andere ID), entsteht draft neu aus dem Original
```

Template-Driven oder Reactive Forms?

- Template-Driven: Beschreibung im Template, wenig Code, gut für einfache Formulare
- Reactive Forms: Beschreibung in der Klasse (FormGroup, FormControl), mehr Kontrolle
- Reactive Forms lohnen sich bei dynamischen Feldern, vielen Abhängigkeiten, komplexen Regeln
- Beide Varianten kennen dieselben Zustände: valid, dirty, touched, errors
- Beide Varianten verwenden dieselben Validatoren
- Im Hauptpfad: Template-Driven Form in Übung 22
- Reactive Forms: Wahlthema am Ende des Kurses, mit Wahlübung Z1



Validierung

Prüfen der Formulareingaben

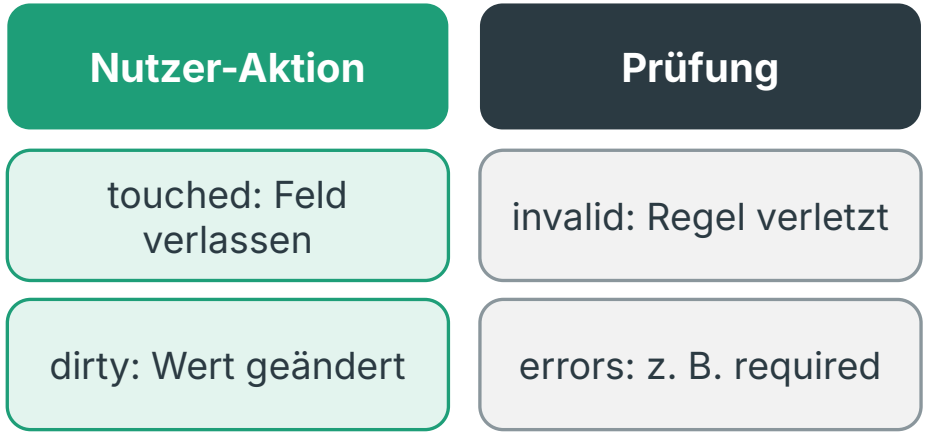
- 2-fache Validierung ist nicht redundant
- Client-seitig Validierung für User Experience
- Server-seitig Validierung für Sicherheit
- Validatoren
  - Built-in (required, pattern, minlength, maxlength, email)
  - Custom
- Verwendung
  - Template-Driven Forms: mittels HTML-Attributen bzw. Direktiven (Hauptpfad)
  - Reactive Forms: mit Hilfe der Forms API (Wahlthema)

Validierung mit HTML-Attributen

- Template-Driven Forms prüfen Eingaben über HTML-Attribute am Feld
- required: Pflichtfeld
- pattern="[0-9]*": Eingabe muss zum regulären Ausdruck passen (z. B. nur Ziffern für eine PLZ)
- minlength und maxlength: Mindest- und Höchstlänge
- email: Eingabe muss wie eine E-Mail-Adresse aussehen
- Angular wertet diese Attribute mit eingebauten Validator-Direktiven aus
- Eigene Prüfregeln (Custom Validators) sind möglich und werden als Direktive angelegt
- Die Prüfung im Client verbessert die Bedienung, die Prüfung im Server bleibt Pflicht

Feldzustände: dirty, touched und invalid

- Jedes Feld hat unabhängige Zustände:
  - untouched / touched: das Feld wurde noch nicht / schon einmal verlassen
  - pristine / dirty: der Wert ist noch unverändert / wurde verändert
  - valid / invalid: der aktuelle Wert ist gültig / ungültig
- errors enthält die konkrete Fehlerursache, z. B. required oder pattern
- Angular setzt passende CSS-Klassen: ng-valid, ng-invalid, ng-dirty, ng-touched
- Fehlermeldungen erst zeigen, wenn der Benutzer das Feld verändert hat (dirty)
- Das ganze Formular ist ungültig, sobald ein Feld ungültig ist



Fehlermeldung und Formularzustand

```
<input name="firstname" [(ngModel)]="c.firstname" required #firstname="ngModel">
@if (firstname.invalid && firstname.dirty) {
  <p class="error">Bitte einen Vornamen eingeben.</p>
}

<input name="zip" [(ngModel)]="c.zip" pattern="[0-9]*" #zip="ngModel">
@if (zip.invalid && zip.dirty) {
  <p class="error">Die PLZ darf nur aus Ziffern bestehen.</p>
}

<p>Formular: {{ customerForm.valid ? 'gültig' : 'ungültig' }}</p>
<!-- Übung 23: <button type="submit" [disabled]="customerForm.invalid"> ⟶
```

Übung 22: Kunde bearbeiten (Template-Driven Form)

- Öffne das Projekt 22_forms und arbeite in CustomerDetailsComponent
- 1. FormsModule im imports-Array importieren (für das Datumsfeld auch DatePipe)
- 2. Bearbeitbare Kopie als linkedSignal anlegen: draft = linkedSignal(() ⇒ ({ ...customer }))
- 3. Felder mit name-Attribut und [(ngModel)] an die Kopie binden
- 4. Validierung per HTML-Attribut: Vor- und Nachname required, PLZ pattern="[0-9]*"
- 5. Fehlermeldung je Feld mit @if (feld.invalid && feld.dirty)
- 6. Formularzustand ausgeben: #customerForm="ngForm" und customerForm.valid
- Musterlösung: 22_forms (baut auf 21_dynamic_routes auf)



Modulare wiederverwendbare Komponenten

Kind Komponenten

Best Practice

- Smarte Komponenten und einfache (dumme) Komponenten
  - Smarte Komponenten wissen etwas über Umgebung
    - z.B. Abhängigkeiten von Services
  - Dumme Komponenten haben nur Inputs und Outputs
    - z.B. keine Abhängigkeiten von Services
- Bessere Wiederverwendbarkeit bei dummen Komponenten

- In unserer Beispielanwendung Formular aus Detailkomponente auslagern. **Wie läuft die Interaktion topDown und bottomUp**

Rückblick: Input Output

- Vater zu Kind über Property-Binding
  - Daten von oben nach unten übergeben
  - Kind: Signal-Input input() – Reaktion auf Änderungen per computed() / effect() statt ngOnChanges
- Kind zu Vater über Event-Binding mit output()
  - Events von unten nach oben reichen

viewChild() – Signal Query

- Injection von Kindkomponente zur einfacheren Interaktion

```
export class ParentComponent {
  listComponent = viewChild(ListComponent);   // Signal<ListComponent | undefined>
  refresh() { this.listComponent()?.reload(); }
}
```

- Legacy: @ViewChild(ListComponent) private listComponent: ListComponent;

Kommunikation über ThirdParty

- Kommunikationskanal
  - Service stellt Observables bereit
- Auch andere Kommunikationsstrukturen möglich z.B. 1:n
- Vaterkomponente kann als Provider vom Service dienen (Scoped Service Instanziierung)

Beispiel Kommunikations-Service

```
@Component({ selector: 'list', ... })
class ListComponent {
  private readonly com = inject(Communication);
  constructor() {

this.com.detailToList.pipe(takeUntilDestroyed())
      .subscribe((update: DetailUpdate) =>
console.log("update received"));
  }
  doSomething() {
    this.com.listToDetail.next(new ListUpdate());
  }
}
```

```
@Injectable()
class Communication {
  listToDetail = new
Subject<ListUpdate>();
  detailToList = new
Subject<DetailUpdate>();
}
@Component({
  imports: [ListComponent,
DetailComponent],
  template: `<list /><detail />`,
  providers: [Communication]
})
class MasterDetailComponent {}
```

Übung 23

- Stelle in CustomerService eine Methode update bereit
  - Rufe die Methode beim Speichern des Formulares in CustomerDetailsComponent auf
  - Ersetze in der Methode das ursprüngliche Customer-Objekt durch das neue
  - Feuere einen neuen Datensatz am internen Subject
- Zeige in AppComponent immer den letzt aktiven Customer an
  - Implementiere in CustomerService die Logik (Methode getNext(): Observable<Customer>)
  - Nutze dafür das Observable von getAll
  - Nutze eine flache Kopie des Arrays zum Sortieren (array.slice().sort)
  - Hole in AppComponent das Observable von getNext und wandle es per toSignal() in ein Signal um
  - Zeige im UI den Namen des nächsten Customer an

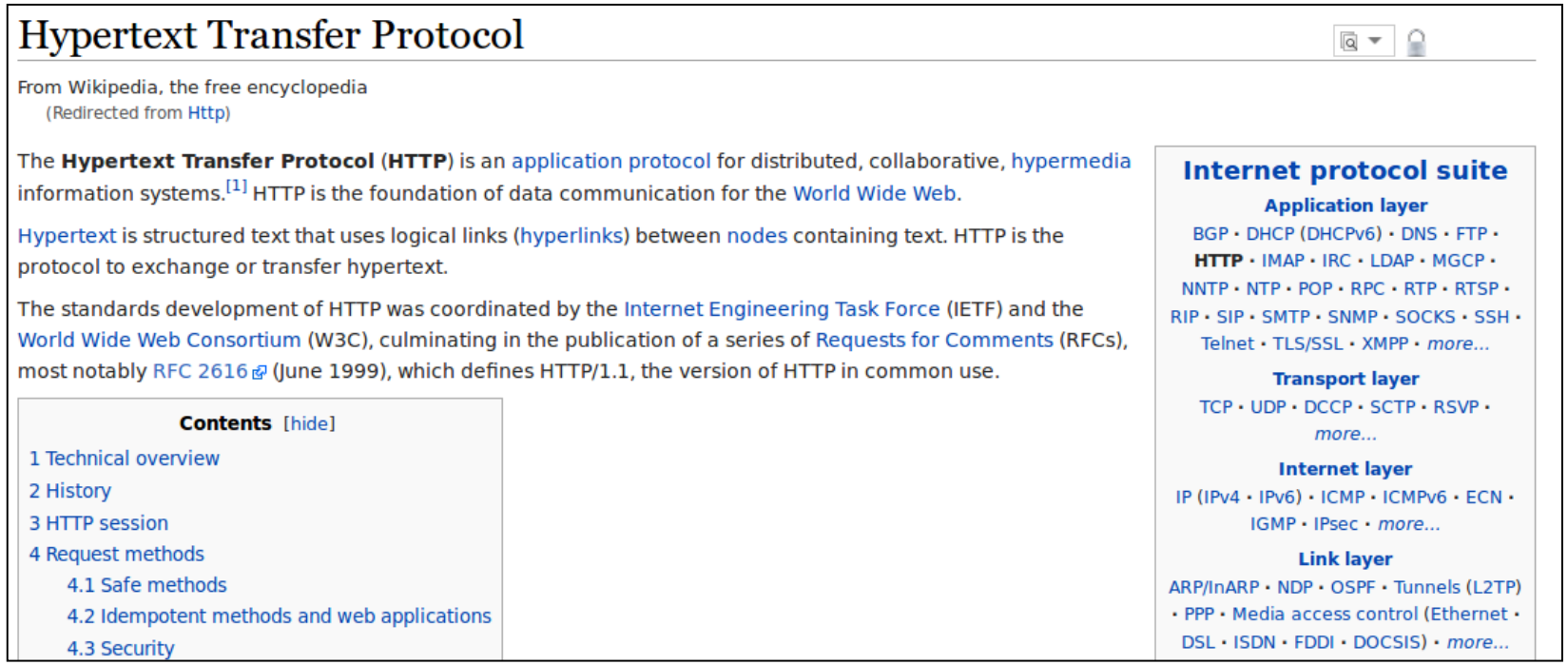


Remote-Kommunikation


Exkurs REST

Das http-Protokoll

- Eine umfassende Spezifikation des w3w-Konsortiums
  - Siehe http://en.wikipedia.org/wiki/Http



Hypertext Transfer Protocol

From Wikipedia, the free encyclopedia
(Redirected from Http)

The **Hypertext Transfer Protocol** (**HTTP**) is an application protocol for distributed, collaborative, hypermedia information systems.[1] HTTP is the foundation of data communication for the World Wide Web.

Hypertext is structured text that uses logical links (hyperlinks) between nodes containing text. HTTP is the protocol to exchange or transfer hypertext.

The standards development of HTTP was coordinated by the Internet Engineering Task Force (IETF) and the World Wide Web Consortium (W3C), culminating in the publication of a series of Requests for Comments (RFCs), most notably RFC 2616 (June 1999), which defines HTTP/1.1, the version of HTTP in common use.

Contents [hide]

1 Technical overview
2 History
3 HTTP session
4 Request methods
  4.1 Safe methods
  4.2 Idempotent methods and web applications
  4.3 Security

Internet protocol suite

Application layer
BGP • DHCP (DHCPv6) • DNS • FTP • **HTTP** • IMAP • IRC • LDAP • MGCP • NNTP • NTP • POP • RPC • RTP • RTSP • RIP • SIP • SMTP • SNMP • SOCKS • SSH • Telnet • TLS/SSL • XMPP • *more...*

Transport layer
TCP • UDP • DCCP • SCTP • RSVP • *more...*

Internet layer
IP (IPv4 • IPv6) • ICMP • ICMPv6 • ECN • IGMP • IPsec • *more...*

Link layer
ARP/InARP • NDP • OSPF • Tunnels (L2TP) • PPP • Media access control (Ethernet • DSL • ISDN • FDDI • DOCSIS) • *more...*

Elemente der http-Spezifikation

- Definition von URIs
  - Pfad
  - Parameter
- http-Request und http-Response
  - Daten-Container mit Header und Body
  - Encodierung
- Umfassender Satz von Header-Properties
  - Content-Length
  - Accepts
  - Content-Type

Elemente der http-Spezifikation II

- http-Methoden
  - PUT
  - GET
  - POST
  - DELETE
  - OPTIONS
  - HEAD
- Statuscodes für Aufrufe
  - 404: „Not found“
  - 204: „Created“
  - …

MimeTypes

- Definition der Datentypen des Internet
  - Nicht zu verwechseln mit einem XML-Schema
  - Ein MimeType ist „nur“ eine strukturierte Zeichenkette
  - Eigene Erweiterungen sind möglich

REST und http

- REST hat mit http prinzipiell nichts zu tun
  - REST ist eine abstrakte Architektur
  - http ist ein konkretes Kommunikationsprotokoll
- Aber
  - http passt als Kommunikations-Protokoll der „Referenz-Implementierung“ Internet natürlich perfekt zum REST-Stil

GET

- Liefert die Ressource

DELETE

- Löscht die Ressource

PUT

- Erzeugt aktualisiert die Ressource
- Client spezifiziert resourceIdentifier
- PUT /resources/<newResourceId> HTTP/1.1

POST

- Erzeugt ein neues Objekt unterhalb (Collection) der spezifizierten Ressource
- POST /resources HTTP/1.1

Der http Client

Der Service HttpClient

- HttpClient ist ein Service
  - und steht damit via Dependency Injection zur Verfügung: inject(HttpClient)
- provideHttpClient() in main.ts registrieren
  - optional withFetch(), withInterceptors([...])
- Das API
  - umfasst die Standard-http-Methoden
  - ist asynchron konzipiert (Observables)

```
this.http.get<Data>('/endpoint').subscribe(data => {
  this.results = data.jsonAttribute;
});
```

Json Daten extrahieren

```
private readonly http = inject(HttpClient);
getAll(): Observable<Release[]> {
  return this.http.get<{ data: Release[] }>('api/release').pipe(
    map(response ⇒ response.data),
    catchError(this.handleError)
  );
}
// JSON-Parsing übernimmt HttpClient automatisch
```



Exception Handler

```
private handleError(error: HttpErrorResponse) {
    let message;
    if (error.message) {
            message = error.message;
    }
    else if (error.status) {
            message = '${error.status} - ${error.statusText} ';
    }
    else {
            message = 'Server error‘;
    }

    // Rethrow in intermediate Methods
    return throwError(() ⇒ new Error(message));
    }
```

Header

```
const headers = new HttpHeaders({ 'Content-Type': 'application/json' });
return this.http.post<{ data: MyObject }>('api/objects', object, { headers }).pipe(
  map(response ⇒ response.data),
  catchError(this.handleError)
);
// JSON.stringify übernimmt HttpClient automatisch
```



Übung 24: Starten des Servers

- Zip-Datei *server-js.zip* entpacken
- Konsole öffnen und in Server Verzeichnis wechseln
- Installieren per npm install
- Starten über node server.js
- Backend läuft unter localhost:3000
- dev.proxy.json: { "/api": { "target": "http://localhost:3000" } }
- Dev-Server mit Proxy starten (CORS) ng serve --proxy dev.proxy.json

Übung 24: Endpunkte

- GET
  - api/customer (GET_ALL)
  - api/customer/:id (GET_BY_ID)
- POST api/customer (INSERT)
- PUT/PATCH/DELETE api/customer/:id (UPDATE, PARTIAL_UPDATE, REMOVE)

Übung 24: http

- Verwende in CustomerService das Backend (inject(HttpClient)) statt der Mock-Daten
- Registriere provideHttpClient() in main.ts
  - Dev-Server mit Proxy starten
  - Lade die Liste initial vom Backend
  - Achtung: Date Felder vom Server werden serialisiert als String gesendet
    - Müssen noch in Date umgewandelt werden
  - Schicke die geänderten Daten in update per put zum Backend
  - Lade (der Einfachheit halber) nach einem update die Liste neu vom Backend
  - Musterlösung: 23_communication (enthält bereits die HTTP-Anbindung)

Ausblick: Resource API

```
export class CustomerDetailsComponent {
  private readonly customerService = inject(CustomerService);
  id = input.required<string>();
  // lädt automatisch neu, sobald sich id() ändert
  customer = httpResource<Customer>(() => `api/customer/${this.id()}`);
  // Alternative mit bestehendem Observable-Service
  customer2 = rxResource({
    params: () => this.id(),
    stream: ({ params }) => this.customerService.getById(params)
  });
}
<!-- Template -->
@if (customer.isLoading()) { <p>Lade...</p> }
@if (customer.hasValue()) { <p>{{ customer.value().firstname }}</p> }
// Resource APIs sind teilweise noch experimentell
```

Eigene Direktiven

- Können simpel erstellt werden
- `ng generate directive <directive-name>`
- Die Direktive ist wiederum eine annotierte TypeScript-Klasse
  - Der `selector` ist das neue HTML-Attribute oder -Element
  - Das DOM-Element, das manipuliert werden soll, wird per inject(ElementRef) bereitgestellt

Eigene Directive: Beispiel

- Ein Beispiel ist unter https://angular.dev/guide/directives/attribute-directives zu finden
- import { Directive, ElementRef, inject } from '@angular/core';
- @Directive({
- selector: '[appHighlight]'
- })
- export class HighlightDirective {
- private readonly el = inject(ElementRef);
- constructor() {
- this.el.nativeElement.style.backgroundColor = 'yellow';
- }
- }
- Verwendung: HighlightDirective im imports-Array der Komponente

Funktionale Route Guards

```
export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);
  if (authService.isLoggedIn()) { return true; }
  router.navigate(['/login']);
  return false;
};
// Registrierung in den Routen:
{ path: 'customer', component: CustomerListComponent, canActivate: [authGuard] }
```

Funktionale HTTP-Interceptors

```
export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  console.log(`[HTTP] ${req.method} ${req.url} gestartet`);
  return next(req);
};
// Registrierung in main.ts:
provideHttpClient(withInterceptors([loggingInterceptor]))
```

Vorteile: keine Klasse/Interface nötig, inject() funktioniert auch hier, mehrere Guards/Interceptors einfach als Array kombinierbar

Übung 25: Funktionale Guards & Interceptors

Erstelle einen AuthService mit Login-Status als Signal
Erstelle den funktionalen Guard authGuard (CanActivateFn)
Schütze die Routen customer und customer/:id mit canActivate: [authGuard]
Erstelle eine einfache LoginComponent
Erstelle den funktionalen Interceptor loggingInterceptor (HttpInterceptorFn)
Registriere ihn über provideHttpClient(withInterceptors([...]))
Musterlösung: 25_httpInterceptorGuard (baut auf 23_communication auf)

Resource API (httpResource & rxResource)

- Deklaratives asynchrones Datenladen in Angular 21
- Direkte Integration von HTTP-Requests in die Signal-Welt:
- • `httpResource('/api/customer'): Gibt reaktives Resource-Objekt zurück`
- • Liefert fertige Signals: .value(), .isLoading(), .error(), .status()
- • Kein manuelles .subscribe(), keine explizite async-Pipe erforderlich
- rxResource für dynamische Parameter und RxJS-Streams:
- • Reagiert automatisch auf Signal-Änderungen (z. B. params: () ⇒ this.id())
- • Automatisches Request-Cancelling bei schnellen Parameter-Änderungen
- • Unterstützt Neuladen per .reload() und manuelle Mutation per .set()/.update()

Beispiel: httpResource & rxResource

- `// Liste laden mit httpResource in CustomerListComponent:`
- `customerResource = httpResource<Customer[]>('/api/customer');`
- customers = this.customerResource.value;
- isLoading = this.customerResource.isLoading;

- `// Detailansicht mit dynamischer ID per rxResource:`
- id = input<string>();
- customerResource = rxResource({
- params: () ⇒ this.id(),
- stream: ({ params }) ⇒ this.customerService.getById(params!)
- });
- customer = this.customerResource.value; // Reagiert sofort auf id-Wechsel

Übung 26: Resource API

- **Öffne das Projekt 26_resourceApi**
- `1. Kundenliste auf httpResource umstellen:`
- `    • Verwende httpResource in CustomerListComponent für das Laden der Kunden`
- • Zeige im Template Ladezustand (@if isLoading()) und Fehler an
- **2. Detailansicht auf rxResource umstellen:**
- • Binde das Laden der Kundendetails an das Signal-Input id per rxResource
- • Entferne manuelle effect()-Aufrufe für das Nachladen bei ID-Änderung
- **3. Teste das Verhalten im Browser:**
- • Wechsle zwischen verschiedenen Kunden und beobachte die reaktive Aktualisierung

Bild-Optimierung mit NgOptimizedImage

- Direktive NgOptimizedImage aus @angular/common für Web-Performance
- Vermeidung von Cumulative Layout Shift (CLS):
  - Verpflichtende Angabe von width und height (oder fill-Modus)
  - Browser reserviert sofort den passenden Platz im Layout
- Automatische Lade-Optimierung:
  - Standardmäßig Lazy Loading (Bilder laden erst, wenn sie ins Sichtfeld scrollen)
  - priority-Attribut für wichtige Hero-/LCP-Bilder (Largest Contentful Paint)
  - Automatisches Erzeugen von srcset für hochauflösende Displays (Retina)
- Integration mit CDNs (Cloudinary, Imgix) über provideImageLoader()

Beispiel: NgOptimizedImage

- `// 1. Direktive in Standalone-Komponente importieren:`
- import { NgOptimizedImage } from '@angular/common';

- @Component({
- standalone: true,
- imports: [NgOptimizedImage],
- template: `
- <!-- Feste Dimensionen verhindern Layout Shift —>
- <img [ngSrc]="customer().imageUrl" width="200" height="200" priority>
- `
- })
- export class CustomerDetailsComponent {}

Übung 27: NgOptimizedImage

- **Öffne das Projekt 27_ngOptimizedImage**
- **1. NgOptimizedImage in CustomerDetailsComponent einbinden:**
- • Importiere NgOptimizedImage aus @angular/common im imports-Array
- • Ersetze [src] durch [ngSrc] für das Kundenbild
- • Setze explizite Dimensionen width="200" und height="200"
- • Ergänze das Attribut priority für das Detailbild
- **2. Überprüfe das Verhalten:**
- • Öffne die Browser-Entwicklertools (Konsole & Network-Tab)
- • Beobachte, dass Angular keine CLS-Warnungen mehr meldet

Signal Queries: viewChild & contentChild

- Signal-basierte DOM- und Komponenten-Abfragen ab Angular 17.2+
- Ersatz für klassische Dekoratoren (@ViewChild, @ViewChildren, @ContentChild):
- - viewChild('refName'): Greift auf ein Element/Komponente im eigenen Template zu
- - contentChild('refName'): Greift auf projizierten Content (<ng-content>) zu
- - viewChildren('refName') / contentChildren(...): Liefert ein Array von Treffern
- Vorteile der Signal-Varianten:
- - Typsicher: Liefert ein Signal<T | undefined> oder Signal<T> bei viewChild.required()
- - Kein Lebenszyklus-Zwang: Kein Warten auf ngAfterViewInit nötig
- - Reaktiv: Kann direkt in computed() oder effect() verwendet werden

Beispiel: DOM-Zugriff mit viewChild()

```
// Template mit Referenzvariable #searchInput:
// <input #searchInput type="text" placeholder="Kunde suchen...">
// <button (click)="resetSearch()">Zurücksetzen</button>

export class CustomerListComponent {
  // Signal Query auf das native Input-Element:
  searchInput = viewChild<ElementRef<HTMLInputElement>>('searchInput');

  resetSearch(): void {
    // Direktes Auslesen und Zugriff auf native DOM-Methoden:
    const input = this.searchInput()?.nativeElement;
    if (input) {
      input.value = '';
      input.focus(); // Fokus zurück auf das Suchfeld setzen
    }
  }
```

Übung 28: viewChild()

- **Öffne das Projekt 28_viewChild**
- **1. Suchfeld in CustomerListComponent ergänzen:**
- • Füge ein Eingabefeld mit Template-Variable #searchInput ein
- • Deklariere searchInput = viewChild<ElementRef>('searchInput')
- **2. Filterlogik und Fokus-Steuerung umsetzen:**
- • Filtere die angezeigte Kundenliste dynamisch nach dem eingegebenen Namen
- • Erstelle einen Button 'Filter löschen', der den Text leert und per searchInput().nativeElement.focus() den Fokus setzt
- **3. Teste im Browser:**
- • Filtere nach einem Kunden und prüfe das Zurücksetzen und die Fokus-Platzierung

Komponenten-Komposition mit <ng-content>

- Content Projection ermöglicht flexible, wiederverwendbare UI-Container
- Prinzip: HTML/Komponenten werden von der Elternkomponente an die Kindkomponente übergeben
- Vorteile:
- - Höhere Wiederverwendbarkeit (Card, Modal-Dialog, Tabs, Accordion)
- - Trennung von Layout-Struktur und fachlichem Inhalt
- Multi-Slot Content Projection mit select-Attribut:
- - <ng-content select="[card-header]">: Platzhalter für spezifische Bereiche
- - <ng-content>: Default-Platzhalter für den gesamten restlichen Inhalt
- - <ng-content select="[card-footer]">: Platzhalter für Aktionen / Buttons
- Inhalte werden im Kontext der Elternkomponente evaluiert (Scope & Bindings)

Beispiel: Multi-Slot Content Projection

- `<!-- Template von CardComponent (app-card): -->`
-
- <ng-content select="[card-header]" />
- <ng-content />
- <ng-content select="[card-footer]" />
-

- `<!-- Verwendung in CustomerDetailsComponent: -->`
- <app-card>
- <h3 card-header>Kundendetails</h3>
- <form #form="ngForm"><!-- Eingabefelder im Body →</form>
- <div card-footer><button (click)="save()">Speichern</button>
- </app-card>

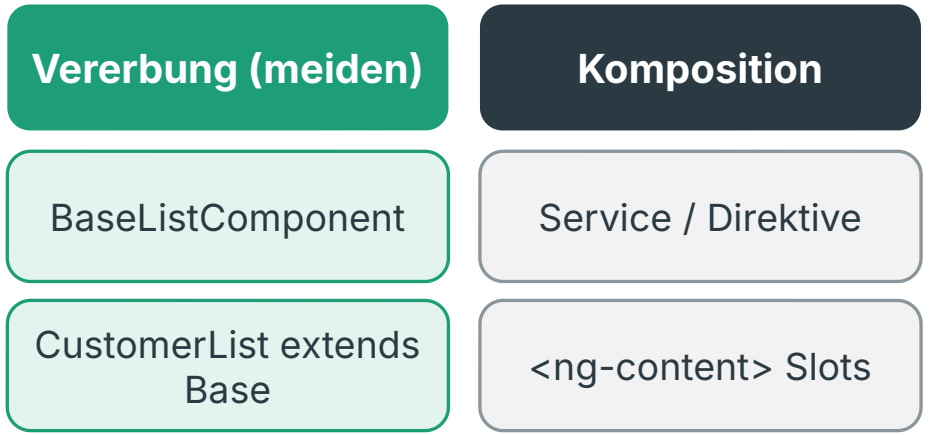
Übung 29: Content Projection

- **Öffne das Projekt 29_contentProjection**
- **1. Wiederverwendbare Card-Komponente erstellen:**
- - Erzeuge CardComponent in src/app/shared/card/
- - Definiere drei Slots mit <ng-content>: [card-header], Default und [card-footer]
- - Gib der Card ein ansprechendes Rahmen- und Schatten-Design per CSS
- **2. Card in CustomerDetailsComponent einbinden:**
- - Importiere CardComponent und platziere das Detailformular innerhalb von <app-card>
- - Weise Überschrift und Aktions-Buttons den entsprechenden Slots zu
- **3. Teste das visuelle Ergebnis im Browser**

Komponenten sinnvoll aufteilen

Komposition statt Vererbung

- Wiederverwendung über Komposition: <ng-content> (Übung 29), Direktiven, kleine Komponenten
- Vererbung zwischen Komponenten vermeiden
  - Gründe: versteckte Kopplung und Eigenheiten der Dekoratoren
- Gemeinsames Verhalten besser in Services oder Host-Direktiven (hostDirectives)



Trennung von Oberfläche und Fachlogik

- Ebene 1: Komponente – Darstellung und Eingaben
- Ebene 2: Service oder Store – Fachlogik und Zustand
- Ebene 3: API-Client (HttpClient) – Zugriff nach außen
- Abhängigkeiten zeigen nur nach unten: Komponente kennt Service, Service kennt Client
- Der Service kennt keine Komponente
- Reine Berechnungen (z. B. aktive Kunden) als pure Funktion oder Pipe, ohne Angular testbar
- Keine Fachregeln im Template: komplexe @if-Bedingungen in ein computed oder eine Pipe



Wartbarkeit: Merkmale einer stabilen Codebasis

- Namen sagen, was etwas ist (CustomerListComponent), nicht wie (Helper2)
- Dateistruktur nach Features (customer/, auth/, shared/), nicht nach Typ
- TypeScript strict, kein any ohne Begründung
- Kleine Funktionen und Komponenten, wenig Verschachtelung
- Automatische Prüfung: ESLint, Prettier und Tests in der Pipeline
- Entscheidungen festhalten: kurze Notiz (ADR), z. B. „Warum Store statt Service?“

Struktur und Testbarkeit hängen zusammen

- Kleine Komponenten mit klaren Inputs und Outputs: Test in etwa 5 Zeilen
- Viele versteckte Abhängigkeiten (inject überall, globale Zustände): Test braucht viel Mocking
- Faustregel: Wenn ein Test schwer zu schreiben ist, ist oft die Struktur das Problem
- Vertiefung im Kapitel zur Testbarkeit

Übung 31: Kundenliste aufteilen

- **1. CustomerFilterComponent: Eingabefeld und Löschen-Button, viewChild-Fokus wandert mit**
  - value = input(''), changed = output<string>(), cleared = output<void>()
- **2. CustomerListItemComponent erstellen**
  - customer = input.required<Customer>(), selected = input(false)
  - chosen = output<Customer>()
- **3. CustomerListComponent behält nur: Daten laden, Filterzustand, Auswahlzustand**
- **4. Das Verhalten der App darf sich nicht ändern (Smoke-Test im Browser)**
- **5. Reflexion: Welche Komponente ist jetzt leicht testbar? Welche Abhängigkeiten sind weg?**
- **Musterlösung: 31_komponenten_aufteilen (baut auf 29_contentProjection auf)**

Reaktivität und Datenfluss

Grundmodell: Quelle, Ableitung, Wirkung

- Quelle (signal): hält Zustand und wird gesetzt
- Ableitung (computed): berechnet aus den Quellen neue Werte
- Wirkung (Template oder effect): greift nach außen – Oberfläche, Speicher, Logging
- Reihenfolge: Quelle, dann Ableitung, dann Wirkung
- Angular verfolgt automatisch, wer welches Signal liest (Abhängigkeitsgraph)
- Nichts muss von Hand abonniert werden

Quelle, Ableitung, Wirkung

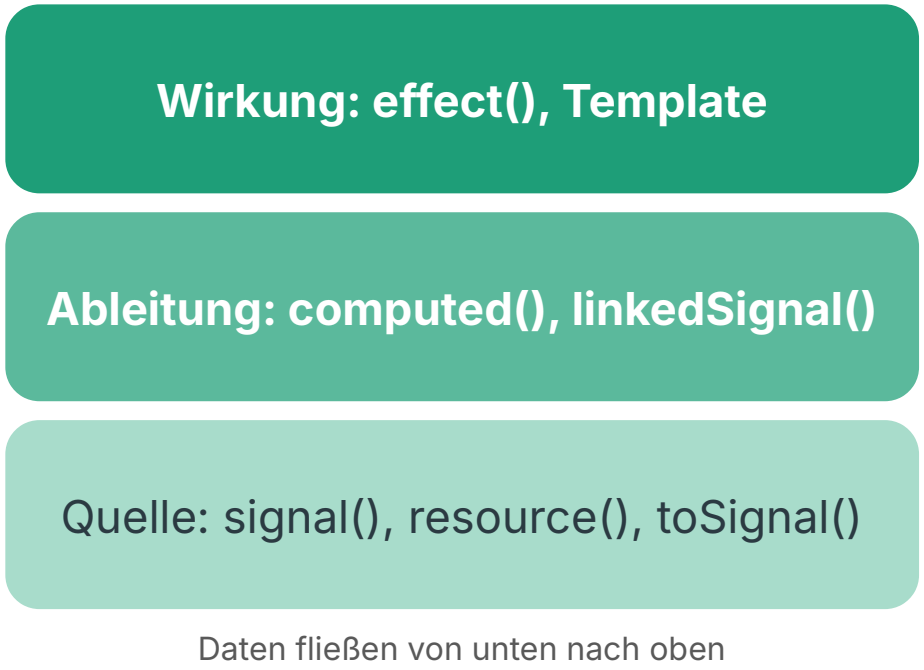
```
const filter = signal("");

const visible = computed(() =>
  customers().filter(c =>
    c.lastname.includes(filter())));

effect(() => {
  console.log(visible().length);
});
```

Landkarte der Reaktivitäts-Bausteine

- signal(): Zustand halten, schreibbar – z. B. Filtertext, Auswahl
- computed(): Wert ableiten, nur lesbar – z. B. gefilterte Liste, Summen
- linkedSignal(): abgeleitet, überschreibbar – z. B. Auswahl, die bei Listenwechsel zurückspringt
- effect(): Nebenwirkung – z. B. localStorage, Logging, Drittbibliothek
- resource() / httpResource(): asynchron laden – z. B. Daten vom Server
- toSignal() / toObservable(): Brücke zu RxJS – z. B. Router-Ereignisse



Lokaler und abgeleiteter Zustand

- Lokaler Zustand (Quelle): Benutzer oder ein Ereignis kann ihn direkt ändern
- Abgeleiteter Zustand: folgt zwingend aus anderem Zustand
  - Nicht speichern, sondern berechnen
- Regel: Single Source of Truth
  - Derselbe Fakt darf nur an einer Stelle gespeichert sein

Doppelter oder abgeleiteter Zustand

```
// Falsch: doppelter Zustand, muss von Hand synchron gehalten werden
customers = signal<Customer[]>([]);
filtered  = signal<Customer[]>([]);   // wird irgendwo "mitgepflegt"

// Richtig: abgeleitet, kann nicht auseinanderlaufen
filtered = computed(() ⇒ {
  const text = this.filterText().toLowerCase();
  return this.customers().filter(c ⇒ c.firstname.toLowerCase().includes(text));
});
```

Befund im Startprojekt der Übung 31

```
// Normale Methode: läuft bei jeder Change Detection neu
filteredCustomers() { ... }

// Funktion statt computed: kein Zwischenspeicher
customers = () ⇒ this.customerResource.value()?.data ?? [];

// Das funktioniert, aber:
//  - keine Zwischenspeicherung
//  - schwer zu verfolgen
//  - Abhängigkeiten sind nicht sichtbar
// Umbau zu computed ist Teil von Übung 32
```

computed im Detail

- Lazy: berechnet erst, wenn der Wert zum ersten Mal gelesen wird
- Memoized: liefert den gespeicherten Wert, solange sich die Abhängigkeiten nicht ändern
- Dynamische Abhängigkeiten: nur Signals, die beim letzten Lauf wirklich gelesen wurden, zählen
- Rein: keine Nebenwirkungen und kein Schreiben auf Signals (Angular wirft einen Fehler)
- Gleichheit: geändert nur bei anderem Wert oder anderer Referenz (Vergleich wie ===)

computed

```
const count = signal(2);

const double = computed(() => {
  console.log("rechne");
  return count() * 2;
});

double(); // rechnet, 4
double(); // aus dem Speicher
count.set(3);
double(); // rechnet neu, 6
```

linkedSignal: abgeleitet, aber änderbar

```
customers = input.required<Customer[]>();

// Auswahl springt auf das erste Element, sobald sich die Liste ändert,
// kann aber vom Benutzer überschrieben werden
selected = linkedSignal(() ⇒ this.customers()[0]);

onPick(c: Customer) {
  this.selected.set(c);
}

// Ersetzt das Muster "ein effect setzt ein Signal anhand eines anderen zurück"
```

Nebenwirkungen und effect()

- Nebenwirkung: verändert etwas außerhalb des Signal-Graphen
- Gute Gründe für einen effect:
  - Zustand in localStorage oder sessionStorage sichern
  - Logging und Analytics
  - Drittbibliothek (Diagramm, Karte, Canvas) mit Daten versorgen
  - Imperativer DOM-Zugriff, der nicht per Binding möglich ist

effect: Beispiele

```
constructor() {
  // Filtertext im Browser sichern, sobald er sich ändert
  effect(() => {
    localStorage.setItem('customerFilter', this.filterText());
  });

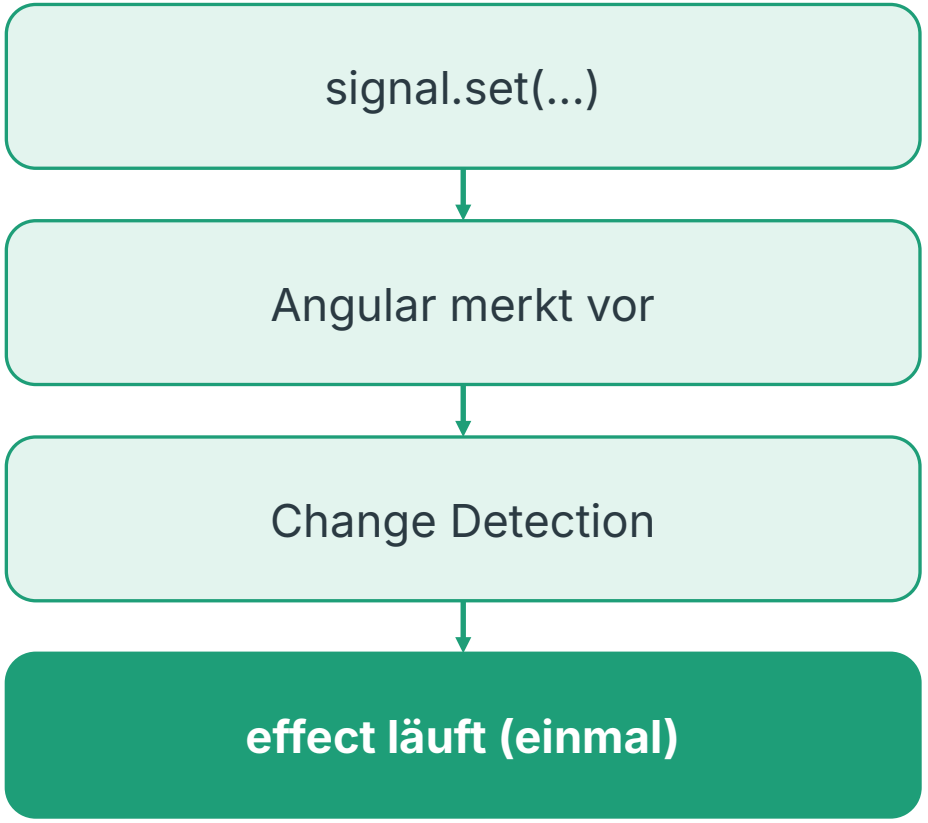
  // Aufräumen mit onCleanup, bevor der effect erneut läuft oder zerstört wird
  effect((onCleanup) => {
    const id = setInterval(() => this.refresh(), this.pollMs());
    onCleanup(() => clearInterval(id));
  });
}
// untracked(() => ...): Lesen ohne Abhängigkeit
// Ein effect muss im Injection Context entstehen (Konstruktor, Feldinitialisierer)
```

Wann kein effect?

- Wert aus anderen Signals berechnen: besser computed
- Signal bei Änderung einer Quelle zurücksetzen: besser linkedSignal
- Auf Klick oder Eingabe reagieren: besser Event-Handler, z. B. (click)
- Daten laden, wenn sich eine id ändert: besser resource oder rxResource
- Beim Start einmal etwas tun: besser Konstruktor oder afterNextRender
- Faustregel: Wenn der effect ein Signal schreibt, gibt es fast immer eine bessere Lösung
  - Technisch erlaubt (seit Version 19), aber ein Warnsignal

Wie ein effect ausgeführt wird

- Läuft mindestens einmal, danach bei jeder Änderung der gelesenen Signals
- Läuft asynchron, nicht sofort nach set(): Angular plant ihn bei der Change Detection ein
- Mehrere Änderungen hintereinander ergeben nur einen Lauf (Batching)
- Aufräumen über onCleanup
- Muss im Injection Context entstehen (Konstruktor, Feldinitialisierer) oder mit injector
- untracked(): Lesen ohne Abhängigkeit



Mehrere set() ergeben einen Lauf

Stolperfalle 1: Mutation statt neuer Referenz

```
list = signal<Customer[]>([]);

this.list().push(c);                // Falsch: Signal merkt nichts
this.list.update(l ⇒ [...l, c]);    // Richtig: neue Referenz

// Eintrag ändern, ebenfalls mit neuer Referenz
this.list.update(l ⇒ l.map(x ⇒ x.id === c.id ? { ...x, ...changes } : x));

// Signals vergleichen mit ===: gleiches Array bedeutet "keine Änderung"
// Gilt auch für Objekte in Signals und für input()-Werte
```

Stolperfalle 2: Zyklen und Ketten

```
// Falsch: der erste effect schreibt in b, der zweite liest b und schreibt in a ...
effect(() ⇒ this.b.set(this.a() * 2));
effect(() ⇒ this.a.set(this.b() + 1));   // Endlosschleife, schwer nachzuvollziehen

// Besser: eine Quelle und ein computed
a = signal(1);
b = computed(() ⇒ this.a() * 2);

// Schreiben auf Signals in einem computed ist verboten (Fehler NG0600)
```

Stolperfalle 3: Methoden im Template

```
<!-- Normale Methode: wird bei jeder Change Detection aufgerufen -->
{{ expensiveSum() }}

<!-- computed: rechnet nur neu, wenn sich Abhängigkeiten ändern -->
{{ total() }}

<!-- Ausnahme: reine, billige Aufrufe. Im Zweifel computed oder eine reine Pipe -->
```

Stolperfalle 4: Brücke zu RxJS

- toSignal(obs$) ohne initialValue liefert T | undefined – im Template behandeln
- toSignal abonniert automatisch und räumt auf (Injection Context nötig)
- Fehler im Observable werden beim Lesen des Signals erneut geworfen: vorher catchError
- Zeit und Ereignisströme (debounce, switchMap, Retry): RxJS bleibt die bessere Wahl
- Zustand: Signals

Brücke zu RxJS: Filter mit Verzögerung

```
// Signal in einen Datenstrom umwandeln, verzögern, wieder als Signal bereitstellen
filter$ = toObservable(this.filterText).pipe(
  debounceTime(300),
  distinctUntilChanged()
);
debounced = toSignal(this.filter$, { initialValue: '' });

// debounceTime(300): erst weiterreichen, wenn 300 ms keine neue Eingabe kam
// distinctUntilChanged(): gleiche Werte hintereinander ignorieren
```

Stolperfalle 5: zu grobe Signals

```
state = signal({
  customers: [],
  filter: '',
  selectedId: null,
  loading: false
});
// Jede Änderung an irgendeinem Feld invalidiert ALLE, die state() lesen

// Besser: mehrere kleine Signals, daraus computed für Ausschnitte
customers = signal<Customer[]>([]);
filter = signal('');
selectedId = signal<number | null>(null);
```

Nachvollziehbare Datenflüsse

- Eltern zu Kind: input() – Daten nach unten
- Kind zu Eltern: output() – Ereignisse nach oben
- Komponente, Service oder Store, Signal, alle Leser aktualisieren sich
- Regeln: eine Richtung, ein Besitzer pro Zustand
- Änderungen nur über benannte Methoden („Aktionen“)
- Debugging: Angular DevTools zeigen den Signal-Graph und die Werte
- Im Zweifel temporär effect(() ⇒ console.log(x()))

Entscheidungsbaum: Welcher Baustein?

- 1. Kommt der Wert vom Benutzer oder von einem Ereignis? Dann signal
- 2. Folgt er aus anderen Werten? Dann computed
  - Soll er überschreibbar sein: linkedSignal
- 3. Kommt er asynchron vom Server? Dann resource oder httpResource
- 4. Muss etwas außerhalb von Angular passieren? Dann effect
- 5. Geht es um Zeit und Ereignisströme (Debounce, Abbrechen, Wiederholen)? Dann RxJS
  - Am Rand der Komponente mit toSignal in ein Signal wandeln

Übung 32: Reaktivitäts-Fallen finden

- **Teil A: filteredCustomers() und customers in computed umbauen**
  - Prüfen: Ein console.log im computed erscheint nur bei geänderten Abhängigkeiten
- Teil B: Filtertext per effect in localStorage sichern und beim Start wiederherstellen
- Teil C: Patch von der nächsten Folie in CustomerListComponent einfügen
  - Je Zeile benennen: Was ist falsch? Welche Regel wird verletzt?
  - Beheben: (1) computed, (2) neue Referenz im Zustand, (3) computed
- Reflexion: Welche Fehler hätte ein Test gefunden, welche nur ein Review?
- **Musterlösung: 32_reaktivitaet (baut auf 31_komponenten_aufteilen auf)**

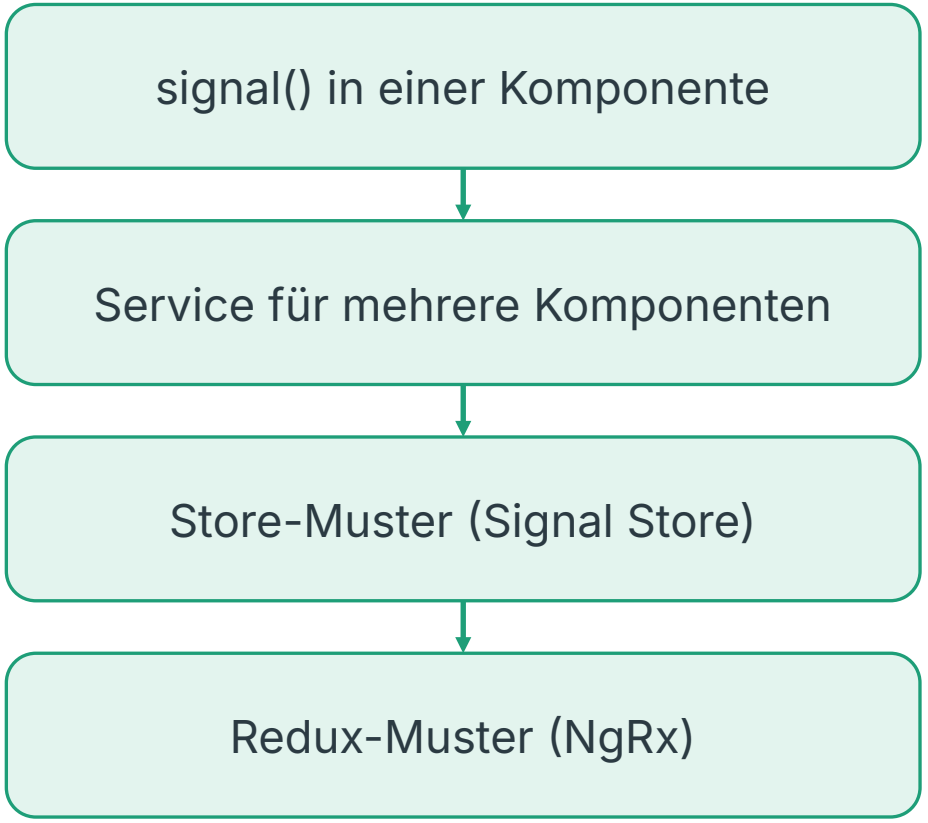
Übung 32: Patch für Teil C

```
// (1)
selectedName = signal('');
effect(() ⇒ this.selectedName.set(this.selectedCustomer()?.firstname ?? ''));
// (2)
addDemo() { this.customers().push(new Customer('Neu', 'Demo')); }
// (3)
total = signal(0);
effect(() ⇒ this.total.set(this.filteredCustomers().length));
// Aufgabe: Was ist je Nummer falsch? Welche Regel wird verletzt?
```

Anwendungsstruktur und Zustandsverwaltung

Wie Zustand wächst

- Phase 1: eine Komponente, 3 Felder – signal() genügt
- Phase 2: 3 Komponenten teilen sich Daten – Service
- Phase 3: Daten, Filter, Auswahl, Laden und Fehler – Store-Muster
- Phase 4: viele Teams, Nachvollziehbarkeit nötig – Redux-Muster (NgRx)
- Nicht jede Anwendung braucht Phase 4
- Zu früh strukturieren kostet Boilerplate, zu spät strukturieren kostet Umbau



Nur so weit gehen, wie nötig

Anwendung strukturieren: Ordner und Zuständigkeiten

```
src/app/
  customer/              ← Feature
    customer-list/       ← Container
    customer-details/
    ui/                  ← Darstellungskomponenten
    customer.store.ts    ← Zustand und Fachlogik
    customer.api.ts      ← HTTP-Zugriff
    customer.model.ts
  auth/
  shared/                ← nur wirklich Allgemeines (Card, Pipes)

// Ein Feature kennt andere Features nur über Schnittstellen (Service, Route)
// Abhängigkeitsrichtung: Features nutzen shared, nie umgekehrt
```

Drei Schichten: Zustand, Logik, Darstellung

- Zustand: Daten halten und ableiten – Signals im Store
- Logik: Regeln, Befehle, Laden – Methoden im Store oder Service
- Darstellung: Anzeigen und Eingaben weitergeben – Komponenten und Templates
- Komponenten lesen Signals und rufen Methoden auf
- Sie entscheiden nicht, wie der Zustand aussieht



Service mit Signals als einfacher Store

```
@Injectable({ providedIn: 'root' })
export class CustomerStore {
  private readonly api = inject(CustomerService);
  // privat schreibbar, öffentlich nur lesbar
  private readonly _customers = signal<Customer[]>([]);
  private readonly _filter = signal('');
  readonly customers = this._customers.asReadonly();
  readonly filter = this._filter.asReadonly();
  readonly filtered = computed(() => {
    const f = this._filter().toLowerCase();
    return this._customers().filter(c => c.firstname.toLowerCase().includes(f));
  });

  setFilter(text: string) { this._filter.set(text); }
  async load() { this._customers.set(await firstValueFrom(this.api.getAll())); }
}
```

Asynchronen Zustand modellieren

```
type LoadState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'error'; message: string }
  | { status: 'loaded'; data: T };

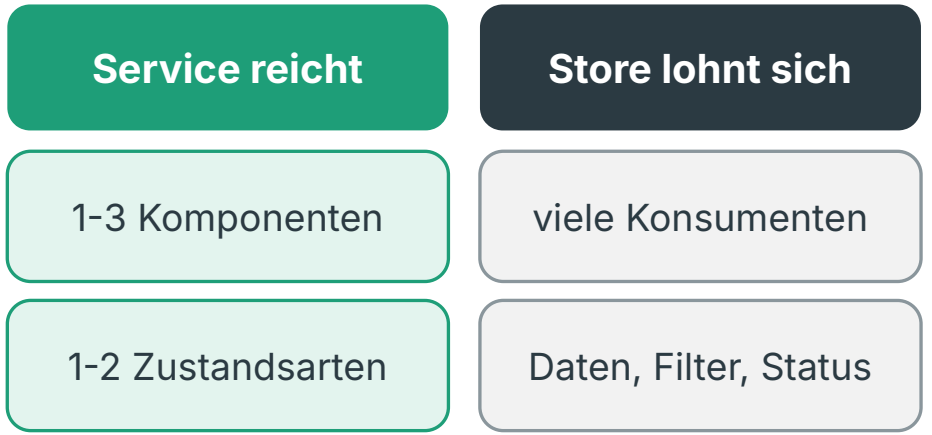
// Unmögliche Kombinationen (loading UND error) sind nicht darstellbar
// Alternative: resource liefert value(), isLoading(), error() fertig (Übung 26)
// Regel: Server-Daten (geladen, gecacht) von UI-Zustand (Dialog offen, Tab) trennen
```

Wachsende Zustandskomplexität beherrschen

- Normalisieren: Entitäten nach id in einer Map halten, Listen enthalten nur IDs
- Ableiten statt kopieren: ausgewählter Kunde = computed aus selectedId und customers
- Änderungen als benannte Befehle: addCustomer, renameCustomer statt überall set()
- Unveränderlich aktualisieren (neue Referenzen), siehe Kapitel Reaktivität
- Zustand klein halten: nur speichern, was nicht berechenbar ist

Wann strukturierten Zustand einführen? (1/2)

- Konsumenten: Service bei 1–3 Komponenten, Store bei vielen, verteilt über Features
- Zustandsarten: Service bei 1–2, Store bei mehreren zusammenhängenden (Daten, Filter, Status)
- Seiteneffekte: Service bei wenigen, Store bei vielen, die voneinander abhängen
- Nachvollziehbarkeit: Service, wenn unwichtig; Store, wenn Änderungen protokolliert werden



Wann strukturierten Zustand einführen? (2/2)

- Teamgröße: Service bei 1–3 Personen, Store bei mehreren Teams mit einheitlichen Mustern
- Wiederverwendung: Service im Einzelfall, Store bei gleichen Mustern in vielen Features
- Testaufwand: Service bei geringem Aufwand, Store bei gezielten Tests der Zustandslogik
- Ergebnis: Starte einfach (Service mit Signals)
  - Strukturiere erst, wenn ein Kriterium zum Problem wird

NgRx Signal Store: Muster als Bibliothek

```
export const CustomerStore = signalStore(
  { providedIn: 'root' },
  withState({ customers: [] as Customer[], filter: '' }),
  withComputed(({ customers, filter }) ⇒ ({
    filtered: computed(() ⇒ customers().filter(
      c ⇒ c.firstname.toLowerCase().includes(filter().toLowerCase()))),
  })),
  withMethods((store, api = inject(CustomerService)) ⇒ ({
    setFilter(filter: string) { patchState(store, { filter }); },
    async load() {
      patchState(store, { customers: await firstValueFrom(api.getAll()) });
    },
  })),
);

// In der Komponente:
// store = inject(CustomerStore);   → store.filtered(), store.setFilter(...)
```

Klassisches NgRx im Vergleich

- State (Zustand): im Signal Store withState
- Selector (Ableitung): im Signal Store withComputed
- Action (Absicht): im Signal Store eine Methode
- Reducer (Zustandsübergang): im Signal Store patchState in einer Methode
- Effect (asynchrone Nebenwirkung): im Signal Store eine Methode oder rxMethod
- Klassisches NgRx: mehr Zeremonie, dafür Action-Log, Time-Travel und strikte Trennung
- Sinnvoll bei sehr großen Anwendungen mit vielen Teams
- Vertiefung: folgendes Deck „State Management“

Typische Fehlentscheidungen

- Alles in den Store: UI-Details (Tab offen, Tooltip) gehören in die Komponente
- God Store: ein Store für die ganze Anwendung – besser einer pro Feature
- Doppelte Wahrheit: Daten im Store und zusätzlich kopiert in der Komponente
- Server-Cache mit Client-Zustand verwechseln
  - Server-Daten haben Alter, Invalidierung und Fehler: resource oder Caching-Strategie bedenken
- Framework-Entscheidung statt Problem: erst die Schwierigkeit benennen, dann das Werkzeug wählen

Testbarkeit des Zustands

- Store = Klasse oder Funktion mit klarer Schnittstelle: ohne Oberfläche testbar
- Test: Methode aufrufen, dann das Signal prüfen
- Kein fixture, kein DOM nötig
- Vertiefung im Kapitel zur Testbarkeit

Übung 33: CustomerStore einführen

- **1. CustomerService zur reinen API-Schicht machen, dann customer.store.ts anlegen**
  - Zustand: customers, filter, selectedId; abgeleitet: filtered, selected
  - Methoden: load(), setFilter(), select()
- **2. CustomerListComponent nutzt nur noch den Store (keine eigene Filterlogik)**
- **3. Der localStorage-Effect aus Übung 32 wandert in den Store – wohin gehört er?**
- **4. Zusatz: Variante mit signalStore aus @ngrx/signals (npm install @ngrx/signals)**
- **5. Reflexion: War der Store hier nach den Kriterien nötig? Begründung in 3 Sätzen**
- **Musterlösung: 33_state_store (baut auf 32_reaktivitaet auf)**

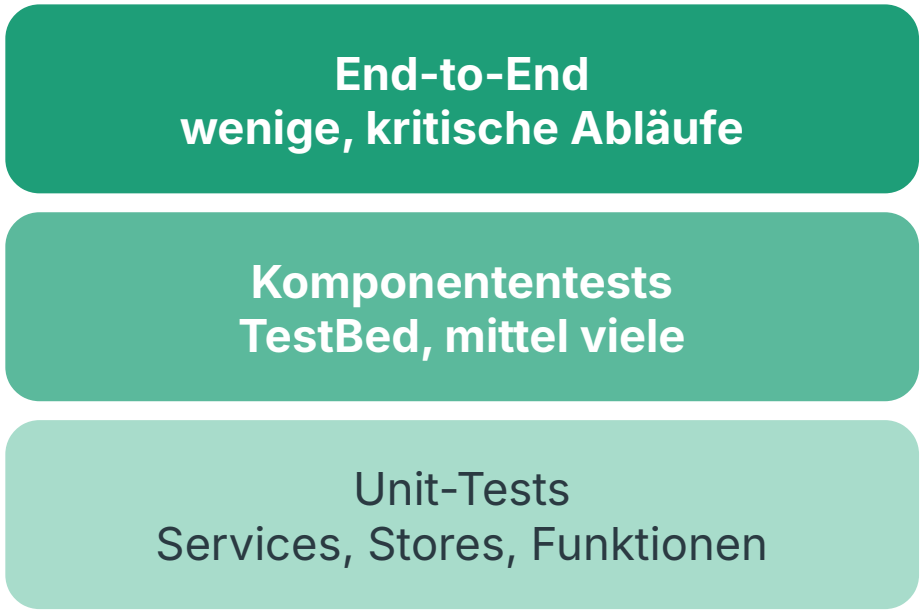
Qualitätssicherung: Testbarkeit und Tests

Wofür Tests?

- Sicherheitsnetz für Änderungen: Man kann umbauen, ohne Angst vor versteckten Fehlern
- Dokumentation: Ein Test beschreibt, wie sich der Code verhalten soll
- Frühes Feedback: Fehler fallen beim Entwickeln und in der Pipeline auf
- Tests sind Code und kosten Pflege: Schlechte Tests bremsen mehr, als sie helfen

Testpyramide mit Angular-Bezug

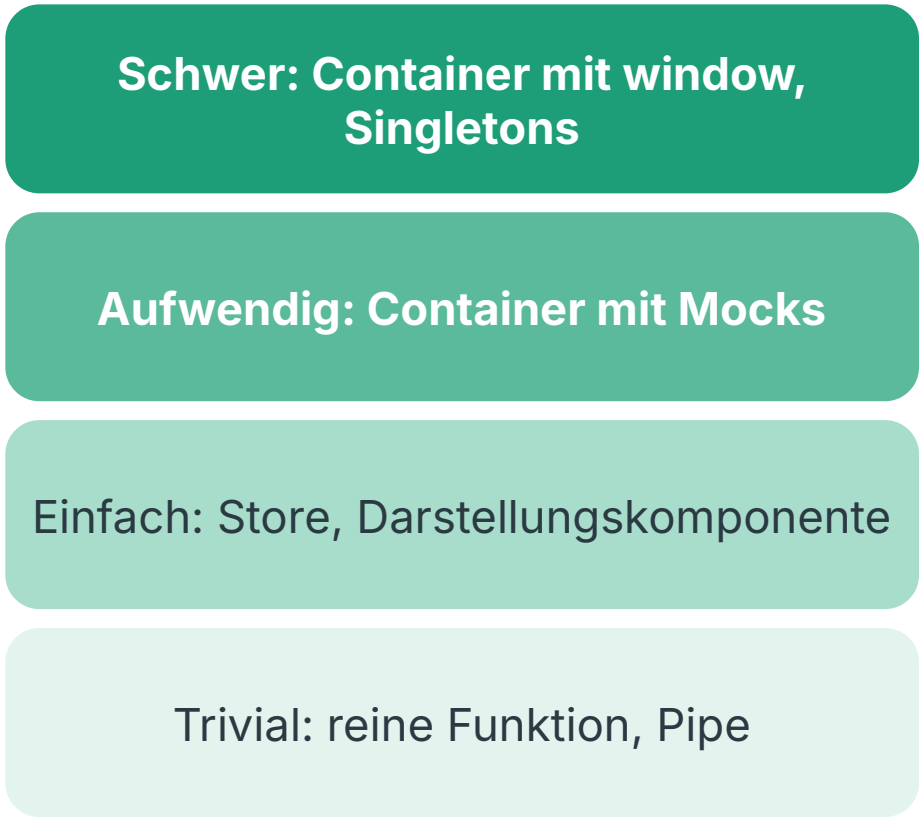
- Unten, viele: Unit-Tests für Services, Stores und reine Funktionen
- In der Mitte, mittel viele: Komponententests mit Template und Klasse (TestBed)
- Oben, wenige: End-to-End-Tests für kritische Abläufe (z. B. mit Playwright oder Cypress)
- Je mehr Logik außerhalb der Komponenten liegt, desto mehr schnelle Unit-Tests sind möglich



Unten viele, oben wenige

Struktur bestimmt Testbarkeit

- Reine Funktion oder Pipe: trivial zu testen, Eingabe hinein, Ausgabe prüfen
- Store oder Service mit Signals: einfach, Methode aufrufen und das Signal lesen
- Darstellungskomponente mit input und output: einfach, Input setzen und Output prüfen
- Container mit mehreren Services: aufwendig, es werden Ersatzobjekte (Mocks) nötig
- Komponente mit DOM- oder window-Zugriff und Singletons: schwer
- Vorher: Komponente ruft HttpClient selbst auf und filtert, nötig sind HTTP-Mocks und DOM
- Nachher: dieselbe Logik im Store, der Test hat etwa fünf Zeilen und braucht kein TestBed



Logik nach unten verlagern

Vitest-Grundlagen

```
import { describe, it, expect, vi, beforeEach } from 'vitest';

describe('CustomerStore', () ⇒ {
  let store: CustomerStore;
  beforeEach(() ⇒ { store = TestBed.inject(CustomerStore); });

  it('filtert nach Vornamen (Groß-/Kleinschreibung egal)', () ⇒ {
    store.setCustomers([new Customer('Hans', 'M'), new Customer('Lisa', 'S')]);
    store.setFilter('LI');
    expect(store.filtered().map(c ⇒ c.firstname)).toEqual(['Lisa']);
  });
});
```

Vitest: Werkzeuge und Aufbau

- vi.fn() erzeugt eine Mock-Funktion, die ihre Aufrufe aufzeichnet
- vi.spyOn() beobachtet eine echte Methode, ohne sie zu ersetzen
- vi.useFakeTimers() ersetzt echte Zeit durch steuerbare Zeit
- Aufbau: Arrange (vorbereiten), Act (ausführen), Assert (prüfen)
- Ein Test prüft ein Verhalten
- Testnamen beschreiben Verhalten („filtert …“), nicht Methodennamen

Arrange, Act, Assert

```
it("filtert nach Nachname", () => {
  // Arrange
  const store = createStore();
  // Act
  store.setFilter("Simp");
  // Assert
  expect(store.visible().length)
    .toBe(1);
});
```

Abhängigkeiten isolieren

- Stub oder Fake: einfache Ersatzimplementierung, z. B. ein Service mit festen Daten
- Mock (vi.fn): prüft Aufrufe, z. B. „wurde update mit diesem Wert aufgerufen?“
- Spy: echtes Objekt wird beobachtet, selten nötig, eher bei Altcode
- Echte Abhängigkeit: kein Ersatz bei reinen Funktionen und billigen Hilfsklassen
- Over-Mocking vermeiden: Wer alles ersetzt, testet nur die Ersatzobjekte

customer.service.spec.ts

```
const api = {
  getAll:
vi.fn().mockReturnValue(of([])),
};

const store = new CustomerStore(api);
store.load();

expect(api.getAll)
  .toHaveBeenCalledTimes(1);
```

Abhängigkeiten per Dependency Injection ersetzen

```
TestBed.configureTestingModule({
  providers: [
    { provide: CustomerService, useValue: { getAll: () ⇒ of([hans, lisa]) } },
  ],
});

const store = TestBed.inject(CustomerStore);

// Der Store bekommt jetzt den Ersatz statt des echten CustomerService.
// Kein Code im Store muss dafür geändert werden.
```

HTTP-Aufrufe testen

```
TestBed.configureTestingModule({
  providers: [provideHttpClient(), provideHttpClientTesting()],
});
const http = TestBed.inject(HttpTestingController);

service.getAll().subscribe(r ⇒ result = r);
http.expectOne('api/customer').flush({ data: [hans] });
http.verify();            // keine unerwarteten Requests
expect(result).toEqual([hans]);
```

Komponenten mit Signal-Inputs testen

```
const fixture = TestBed.createComponent(CustomerListItemComponent);
fixture.componentRef.setInput('customer', hans);   // Signal-Input setzen
await fixture.whenStable();                        // Rendering abwarten
expect(fixture.nativeElement.textContent).toContain('Hans');

const spy = vi.fn();
fixture.componentInstance.chosen.subscribe(spy);
fixture.nativeElement.querySelector('li').click();
expect(spy).toHaveBeenCalledWith(hans);
```

Komponententests: wichtige Regeln

- Signal-Inputs nie direkt zuweisen (component.customer = ...), das funktioniert nicht
- Stattdessen fixture.componentRef.setInput('name', wert) verwenden
- Die Anwendung läuft zoneless: nicht auf automatische Change Detection verlassen
- Nach Änderungen await fixture.whenStable() aufrufen, bevor das DOM geprüft wird
- Ausgänge (output) prüft man, indem man sich mit subscribe anhängt

Signals und Effects testen

```
// signal und computed: direkt testen, ohne TestBed

// effect: braucht einen Injection Context und einen "Tick"
TestBed.runInInjectionContext(() ⇒ { store = new CustomerStore(); });
store.setFilter('x');
TestBed.tick();                // führt anstehende Effects aus
expect(localStorage.getItem('customerFilter')).toBe('x');
```

Was macht einen guten Test aus?

- Er schlägt fehl, wenn das Verhalten kaputt ist, und nur dann
- Er ist unabhängig von Reihenfolge und anderen Tests
- Er ist schnell und deterministisch: keine echten Timer, kein Netz
- Er ist lesbar: Arrange, Act, Assert, sprechende Namen, kleine Testdaten
- Er prüft Verhalten, nicht Implementierungsdetails (private Felder, CSS-Klassen)
- Coverage ist ein Hinweis, kein Ziel: 100 % ohne Prüfungen beweisen nichts

Automatisch erzeugte Tests richtig einordnen

- Quellen: ng generate (Gerüste), Assistenten in der Entwicklungsumgebung, KI-Werkzeuge
- should create mit toBeTruthy: prüft nur, dass die Instanz entsteht
- Test wiederholt die Implementierung: Tautologie, Fehler fallen nicht auf
- Alles gemockt, nur Aufrufe geprüft: testet nicht das echte Verhalten
- Nur der Normalfall: leere Listen, null, Fehler und Grenzwerte fehlen
- Snapshots großer Templates: brechen oft und werden blind aktualisiert
- Test passt sich dem Fehler an: bestätigt falsches Verhalten

Prüfverfahren für generierte Tests

- 1. Lesen: Was behauptet der Test? Passt das zur Anforderung?
- 2. Kaputtmachen: Produktivcode absichtlich ändern, wird der Test rot? Sonst ist er wertlos
- 3. Randfälle ergänzen: leer, sehr groß, Fehlerfall, Sonderzeichen
- 4. Vereinfachen: Mocks reduzieren, Namen verbessern
- 5. Verantwortung: Wer den Test committet, haftet dafür, nicht das Werkzeug

Übung 34: Tests schreiben und bewerten

- **Öffne das Projekt 34_tests (Test-Runner: Vitest)**
- Teil A: Schreibe Tests für CustomerStore, ohne Oberfläche
  - Filter (Groß-/Kleinschreibung), Auswahl, leere Liste, Filter ohne Treffer
  - Ersetze CustomerService per providers durch ein Ersatzobjekt
- Teil B: Teste CustomerListItemComponent: Name wird angezeigt, Klick löst chosen aus
- Teil C: Bewerte die mitgelieferte, generierte Spec für CustomerStore (5 Schwächen)
  - Ändere in filtered includes zu startsWith: Welche Tests werden nicht rot?
  - Verbessere die Spec
- **Musterlösung: 34_tests (baut auf 33_state_store auf)**

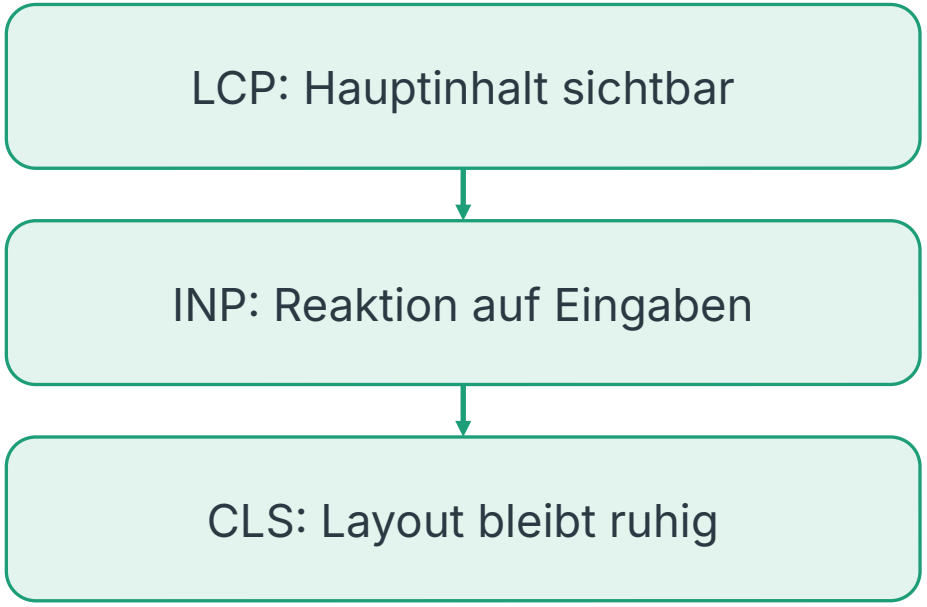
Performance und Laufzeitverhalten

Performance im Anwendungskontext

- Performance ist kein Selbstzweck, sondern Nutzererlebnis: Fühlt sich die App schnell an?
- Ladezeit: Wie schnell ist etwas sichtbar und bedienbar?
  - Einflüsse: Bundle-Größe, Netzwerk, Rendering
- Laufzeit: Wie flüssig reagiert die App?
  - Einflüsse: Change Detection, große Listen, Berechnungen
- Regel Nr. 1: Erst messen, dann optimieren
- Vorzeitige Optimierung macht Code komplizierter, ohne zu wirken

Messen: Kennzahlen

- LCP (Largest Contentful Paint): Wann ist der Hauptinhalt sichtbar?
- INP (Interaction to Next Paint): Wie schnell reagiert die App auf Eingaben?
- CLS (Cumulative Layout Shift): Springt das Layout beim Laden?
- Bundle-Größe: Wie viel Code muss der Browser laden?



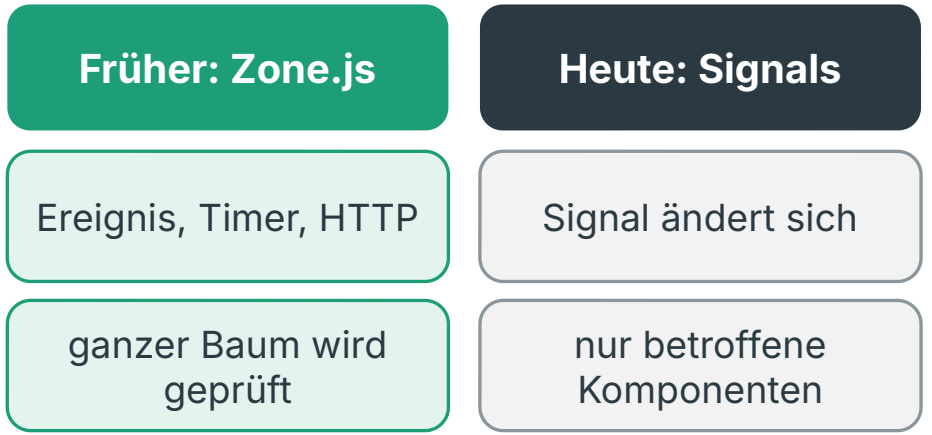
Core Web Vitals

Messen: Werkzeuge

- Chrome DevTools, Bereich Performance: Flamegraph und lange Aufgaben (Long Tasks)
- Lighthouse: Ladeverhalten und Kennzahlen (Core Web Vitals)
- Angular DevTools, Profiler: Welche Komponente wird wie oft geprüft?
- ng build --stats-json mit einem Analyse-Werkzeug: Was steckt im Bundle?
- Budgets in angular.json: Der Build schlägt bei zu großem Bundle fehl

Wie Angular rendert

- Früher mit Zone.js: Jedes Ereignis, jeder Timer, jede HTTP-Antwort prüft den ganzen Baum
- Heute zoneless mit Signals: Ändert sich ein Signal, werden nur betroffene Komponenten geprüft
- OnPush: Prüfung nur bei geänderten Inputs, Ereignissen im Template oder geänderten Signals
- Empfehlung: changeDetection: ChangeDetectionStrategy.OnPush für alle neuen Komponenten
- Das Startprojekt läuft bereits mit provideZonelessChangeDetection()



Struktur beeinflusst die Laufzeit

- Kleine Komponenten mit eigenen Signals ergeben kleine Aktualisierungsbereiche
- Eine große Komponente, die alles liest, wird bei jeder Änderung komplett neu bewertet
- Beispiel Invoice-App: Der Filtertext darf nur die Liste neu rendern, nicht den Kopfbereich
- Ein weiterer Grund für die Aufteilung in kleine Komponenten aus dem Kapitel zur Wartbarkeit

Zustandsstruktur und Rendering

```
// Schlecht: ein grobes Objekt-Signal, jede Änderung betrifft alle Leser
state = signal({ customers: [], filter: '', selectedId: null });

// Besser: feine Signals und computed, nur betroffene Leser werden aktualisiert
customers  = signal<Customer[]>([]);
filter     = signal('');
selectedId = signal<string | null>(null);
selected   = computed(() => this.customers().find(c => c.id === this.selectedId()));

// computed liefert bei gleichem Ergebnis dieselbe Referenz,
// nachfolgende Leser rechnen dann nicht neu.
```

@for und track

```
<!-- Schlecht: Objektidentität (so im Startprojekt) -->
@for (c of filteredCustomers(); track c) {
  <app-customer-list-item [customer]="c" />
}

<!-- Besser: stabile, eindeutige ID -->
@for (c of filteredCustomers(); track c.id) {
  <app-customer-list-item [customer]="c" />
}
```

track: Regeln

- Mit track c entstehen bei neu geladenen Objekten alle DOM-Elemente neu
- Zustand der Zeilen (Fokus, Animation) geht dabei verloren
- track $index nur verwenden, wenn die Liste statisch ist
- Faustregel: Eine stabile, eindeutige ID verwenden



Berechnungen im Template

{{ expensiveSum() }} <!-- Methode: wird bei jeder Change Detection aufgerufen -->

{{ total() }} <!-- computed: gecacht, rechnet nur bei geänderten Eingaben -->

{{ customer.lastOrderDate | daysAgo }} <!-- reine Pipe: gecacht je Eingabe -->

<!-- Vorsicht bei unreinen Pipes (pure: false) und async-Pipe in Schleifen -->

Große Listen

- track mit ID: weniger Neuaufbau der DOM-Elemente
- Virtual Scrolling (@angular/cdk/scrolling): nur sichtbare Zeilen sind im DOM
- Paginierung oder serverseitiges Filtern: weniger Daten werden übertragen
- Debounce für Filtereingaben: weniger Neuberechnungen
- Leichte Zeilenkomponenten mit OnPush: günstigeres Prüfen

Debounce für den Filter

```
// filterText ist ein Signal, das an das Eingabefeld gebunden ist
debouncedFilter = toSignal(
  toObservable(this.filterText).pipe(debounceTime(250)),
  { initialValue: '' }
);

// Die Liste liest debouncedFilter() statt filterText():
// Gefiltert wird erst, wenn 250 ms keine neue Eingabe kam.
```

Ladezeit: nur laden, was gebraucht wird

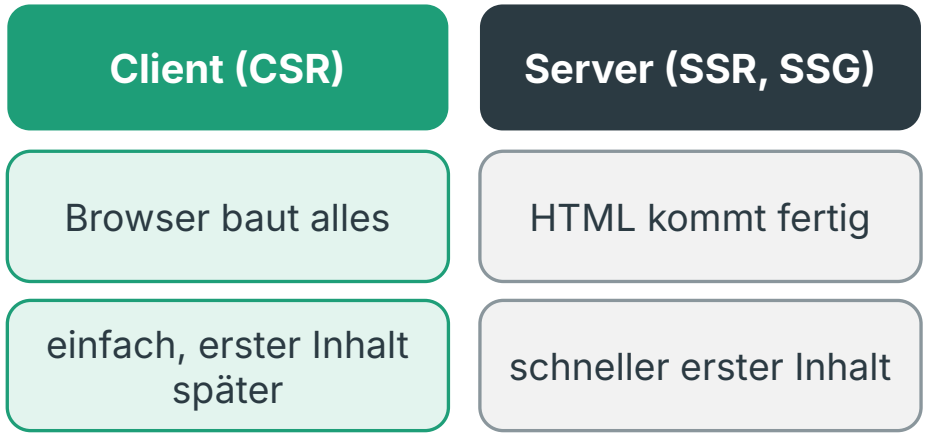
- Lazy Routes: loadComponent: () ⇒ import('./customer-details/...').then(...)
- @defer: schwere Teile erst bei Bedarf laden (on viewport, on interaction, on idle, when)
  - Mit @placeholder und @loading für die Übergangszeit
- NgOptimizedImage (Übung 27): priority für das Hauptbild, width und height gegen Layout-Sprünge
- Bundle-Hygiene: nur importieren, was man nutzt; Tree-Shaking; Budgets

app.routes.ts

```
{
  path: 'details/:id',
  loadComponent: () =>
    import('./customer-details')
      .then(m => m.CustomerDetails),
}
```

Rendering-Strategien im Überblick

- CSR (Client-Side Rendering): Der Browser baut alles, einfach, aber erster Inhalt kommt später
- SSR mit Hydration: Server liefert HTML, Browser aktiviert es, schnellerer erster Inhalt
- Prerendering (SSG): HTML entsteht beim Build, schnell für statische Seiten
- Interne Fachanwendungen: oft reicht CSR
- Öffentliche Seiten: SSR prüfen, siehe die Folie „Server-Side Rendering (SSR) & Hydration“



Typische Ursachen für Laufzeitprobleme

- 1. Methodenaufrufe im Template mit teurer Berechnung
- 2. @for ohne oder mit ungeeignetem track
- 3. Sehr große Listen ohne Virtualisierung oder Paginierung
- 4. Grobes Objekt-Signal, das alles ungültig macht
- 5. Zu viele Effects oder Effects, die Signals schreiben
- 6. Nicht abgemeldete Subscriptions, Timer und Event-Listener (Speicherlecks)
- 7. Zu große Bundles oder zu viele Bibliotheken beim Start
- 8. Ungeeignete Bildgrößen und -formate
- 9. Zu häufige HTTP-Aufrufe ohne Debounce oder Cache

Vorbereitung auf Skalierung

- Messbare Budgets (Bundle, LCP) in der Pipeline festlegen
- Performance-Tests bei Releases, z. B. mit Lighthouse CI
- Frühere Entscheidungen zahlen ein: kleine Komponenten, feine Signals, Lazy Loading
- Wiederholbar messen und Ergebnisse dokumentieren

Übung 35: Langsame Liste

- **Öffne das Projekt 35_performance (5000 Testkunden per Hilfscode, LOAD_TEST_SIZE: 0 = aus)**
- **1. Messe: Chrome DevTools (Eingabe im Filter) und Angular DevTools Profiler**
  - Notiere Dauer der Aktualisierung und Anzahl neu gerenderter Komponenten
- **2. Optimiere Schritt für Schritt und miss jeweils neu:**
  - a) track customer zu track customer.id b) OnPush für Liste und Listenelement
  - c) Debounce für den Filter (250 ms) d) Virtual Scrolling mit cdk-virtual-scroll-viewport
  - e) optional: CustomerDetailsComponent als Lazy Route (loadComponent)
- **3. Fülle eine Tabelle: Maßnahme, Wert vorher, Wert nachher, Fazit**
- **Musterlösung: 35_performance (baut auf 34_tests auf)**

Server-Side Rendering (SSR) & Hydration

- Modernes Rendering-Modell ab Angular 17+ (Standard bei ng new)
- Server-Side Rendering (SSR) mit @angular/ssr:
  - Initiales Rendern des HTML auf dem Node.js-Server
  - Schneller First Contentful Paint (FCP) und optimale Suchmaschinen-Indexierung (SEO)
- Non-Destructive Hydration:
  - Client-Browser zerstört nicht den Server-DOM, sondern übernimmt ihn nahtlos
  - Kein sichtbares UI-Flackern beim Starten der Angular-Anwendung
- Event Replay (Angular 18+):
  - Klicks und Eingaben vor Abschluss der Hydration werden zwischengespeichert
  - Nach Start der Anwendung werden die Aktionen automatisch abgespielt
- Setup in main.config.ts: provideClientHydration(withEventReplay())

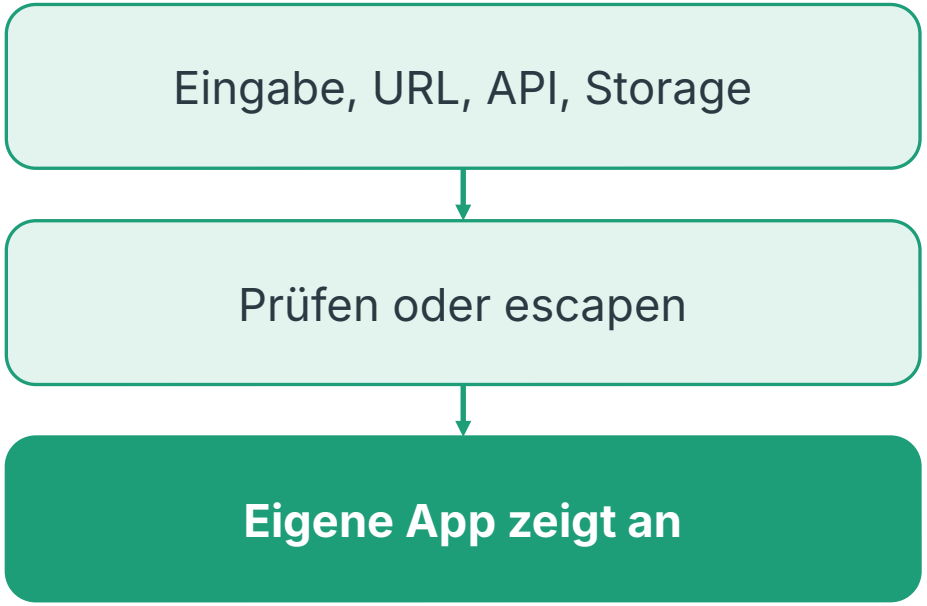
Sicherheit im Frontend

Grundsatz: Das Frontend ist nicht vertrauenswürdig

- Der Code läuft im Browser des Nutzers: Er kann ihn lesen, ändern und umgehen
- Keine Geheimnisse im Frontend: keine API-Keys mit Rechten, Passwörter, Signierschlüssel
- Berechtigungen werden im Backend durchgesetzt
  - Guards (Übung 25) verbessern die Bedienung, sie sind kein Schutz
- Validierung im Client ist Bedienkomfort, Validierung im Server ist Sicherheit

Vertrauensgrenzen

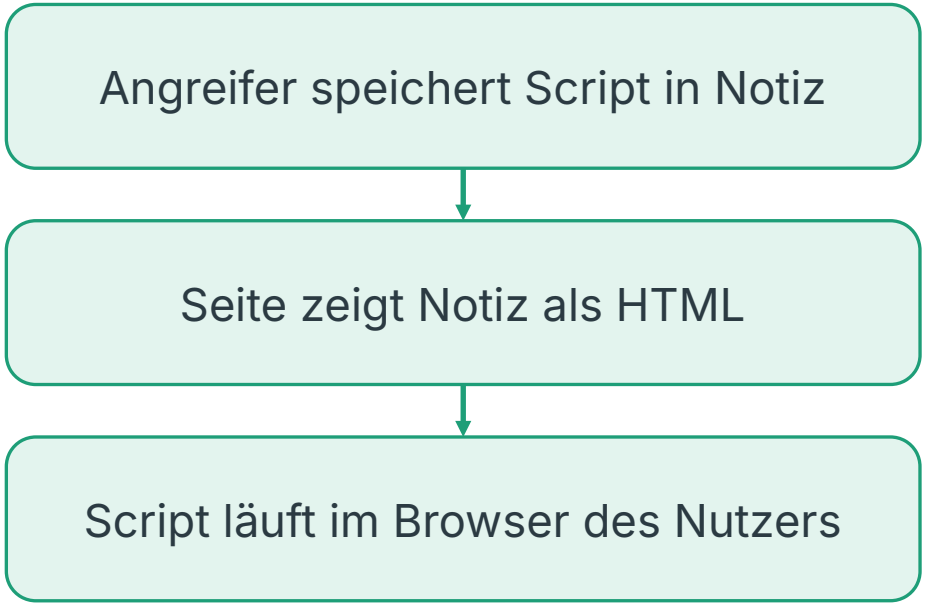
- Nicht vertrauenswürdig, immer prüfen oder escapen:
  - Eingaben im Formular; URL, Query- und Pfadparameter, Fragment
  - Antworten von APIs, auch vom eigenen Backend; localStorage, Cookies, postMessage
  - Daten von Drittanbietern und Dateiuploads
- Vertrauenswürdig: der Quelltext der eigenen App und fest einkompilierte Konstanten
- Alles von außen ist Eingabe, auch Daten aus dem eigenen Backend
- Angezeigte Daten müssen für ihren Kontext geschützt werden (HTML, URL, Style)



Alles von außen ist Eingabe

XSS: das Hauptrisiko

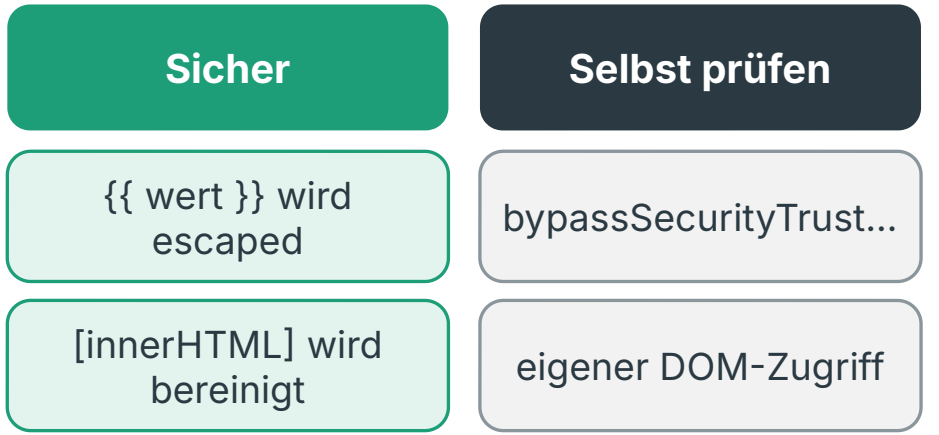
- Cross-Site Scripting (XSS): Ein Angreifer schleust Script in die Seite ein
- Das Script läuft mit den Rechten des Nutzers: Es kann Token auslesen und Aktionen auslösen
- Beispiel: Eine Kunden-Notiz enthält ein Bild mit Fehler-Ereignis:
  - <img src=x onerror="alert('XSS')">
- Zeigt die Seite die Notiz ungeschützt als HTML an, läuft das Script im Browser des Betrachters
- Hier genügt harmlos ein alert, in echten Fällen wäre der Schaden groß



Angular escaped und bereinigt standardmäßig

Was Angular standardmäßig tut

- Interpolation {{ }}: Der Wert wird als Text ausgegeben (escaped)
- [innerHTML]: Angular bereinigt das HTML (Sanitizing) und entfernt gefährliche Teile
  - Zum Beispiel script-Elemente und Event-Handler wie onerror
- [href] und [src]: gefährliche URLs wie javascript: werden entschärft (Präfix unsafe:)
- Templates werden beim Build kompiliert, Nutzereingaben werden nicht als Template ausgewertet



Interpolation und innerHTML im Vergleich

```
<!-- Sicher: Ausgabe als Text, die Notiz erscheint mit allen Zeichen -->
<p>{{ customer.note }}</p>

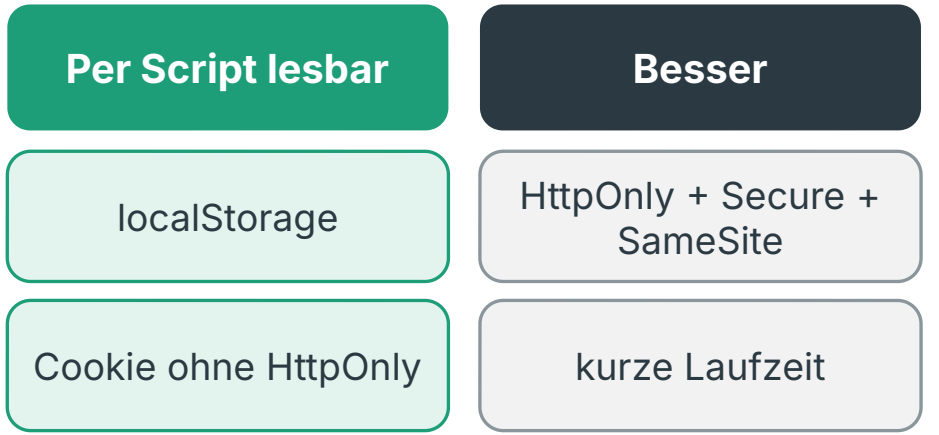
<!-- Bereinigt: Formatierung wie <b> bleibt, gefährliche Teile werden entfernt -->
<div [innerHTML]="customer.note"></div>
```

Wo Angulars Schutz ausgehebelt wird

- DomSanitizer: bypassSecurityTrustHtml/-Url/-Script/-ResourceUrl, Angular prüft nicht mehr
- Nur für selbst kontrollierte Inhalte, nie für Nutzereingaben
- Direkter DOM-Zugriff: nativeElement.innerHTML = …, document.write, eval, new Function
- Templates zur Laufzeit aus Strings bauen, wenn Nutzertext enthalten ist
- Drittbibliotheken, die HTML erzeugen (Markdown-Renderer, Editoren) ohne Sanitizing
- Regel: bypassSecurityTrust* ist ein Review-Alarm, jeder Einsatz braucht eine Begründung im Code

Authentifizierung und Token

- localStorage oder sessionStorage: bei XSS per Script auslesbar
- Cookie ohne HttpOnly: ebenfalls per Script auslesbar
- Cookie mit HttpOnly, Secure und SameSite: Script kommt nicht heran (CSRF beachten)
- Variable im Speicher: geht beim Reload verloren, schwerer zu stehlen
- Empfehlung: Sitzung über HttpOnly-Cookie, das das Backend setzt; Token mit kurzer Laufzeit
- Login bevorzugt über etablierte Bibliotheken oder Identity Provider statt Eigenbau
- Der AuthService aus Übung 25 (Signal isLoggedIn) ist nur eine Demo



CSRF und HTTP (1/2)

- CSRF: Eine fremde Seite löst mit dem Browser des Nutzers Aktionen im Backend aus
  - Der Browser sendet dabei die Cookies des Nutzers automatisch mit
- Gegenmaßnahmen: SameSite-Cookies und CSRF-Token
- HttpClient liest das Cookie XSRF-TOKEN und sendet es als Header X-XSRF-TOKEN
  - Nur bei veränderten Anfragen (z. B. POST, PUT, DELETE) an relative URLs
  - Das Backend muss das Cookie setzen und den Header prüfen

XSRF-Namen anpassen

```
// Standard: Cookie "XSRF-TOKEN", Header "X-XSRF-TOKEN"
provideHttpClient(
  withXsrfConfiguration({
    cookieName: 'MY-XSRF-COOKIE',   // Name des Cookies, das das Backend setzt
    headerName: 'X-MY-XSRF',        // Name des Headers, den das Backend erwartet
  })
);
```

CSRF und HTTP (2/2)

- CORS schützt nicht den Server
  - CORS regelt, welche Webseiten Antworten lesen dürfen
  - Nicht mit Absicherung verwechseln
- Immer HTTPS verwenden
- Keine sensiblen Daten in URLs (sie landen in Logs und im Referrer)

Weitere typische Risikoquellen (1/2)

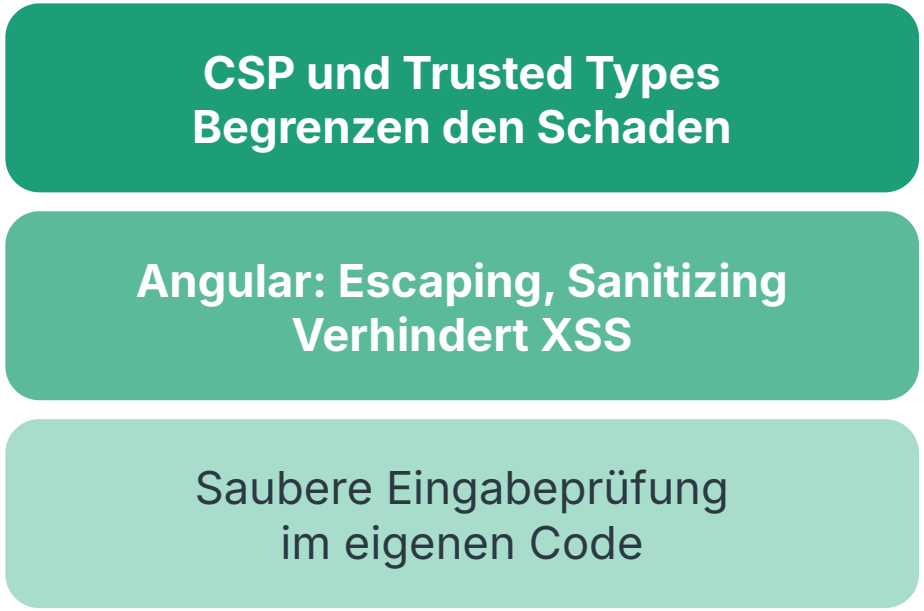
- Offene Weiterleitung: navigateByUrl mit ungeprüftem returnUrl-Parameter
  - Gegenmaßnahme: Ziel gegen eine Allowlist prüfen, nur interne Pfade
- Unsichere Abhängigkeiten: veraltete npm-Pakete mit Lücken
  - Gegenmaßnahme: npm audit, Updates, Lockfile, Werkzeuge für automatische Updates
- Geheimnisse im Bundle: API-Key in environment.ts
  - Gegenmaßnahme: Zugriff über das Backend, Keys mit minimalen Rechten

Weitere typische Risikoquellen (2/2)

- Zu viele Daten im Client: Das Backend soll nur erlaubte Felder liefern
- Sensible Daten in Logs: nichts Sensibles per console.log oder Interceptor ausgeben (Übung 25)
- Clickjacking: Seite in fremdem iframe, Schutz durch frame-ancestors oder X-Frame-Options
- Drittanbieter-Skripte (Tracking, Widgets): Notwendigkeit prüfen, Subresource Integrity, CSP

Content Security Policy und Trusted Types

- CSP ist ein HTTP-Header des Servers
  - Er legt fest, woher Skripte und Styles geladen werden dürfen
  - Er begrenzt den Schaden bei XSS
- Angular unterstützt strikte CSP (z. B. Nonce über CSP_NONCE) und Trusted Types
- Einordnung: zusätzliche Verteidigungsschicht (Defense in Depth)
- Sie ersetzt kein sauberes Escaping



Mehrere Schutzschichten

Umgang mit Daten und Benutzerinteraktionen

- Datensparsamkeit: nur anzeigen und speichern, was gebraucht wird (DSGVO)
- Kontextgerecht behandeln: Text per Interpolation, HTML per Sanitizing, URL durch Validieren
- Nicht auf Client-Validierung verlassen: im Backend wiederholen
- Dateiuploads: Typ und Größe prüfen, Inhalte nie ausführen
- Fehlermeldungen nicht zu detailreich, keine Stacktraces für Endnutzer

Verantwortungsvoller Einsatz im Team

- Sicherheit ist Teamaufgabe: Review-Checkliste und automatische Prüfungen (Lint, npm audit)
- Updates zeitnah mit ng update: Sicherheitsfixes kommen über Minor- und Patch-Versionen
- Dokumentierte Ausnahmen: Wo wird bypassSecurityTrust* oder innerHTML genutzt, und warum?
- Vorfälle offen und lernorientiert behandeln

Review-Checkliste Frontend-Sicherheit

- Wird bypassSecurityTrust* verwendet? Ist es begründet?
- Gibt es innerHTML, nativeElement.innerHTML, document.write oder eval?
- Werden URL-Parameter ungeprüft für Navigation oder Anzeige genutzt?
- Werden Tokens oder personenbezogene Daten geloggt oder im localStorage abgelegt?
- Stehen Geheimnisse in environment-Dateien oder im Bundle?
- Verlässt sich eine Funktion nur auf einen Guard statt auf das Backend?
- Wurden neue Abhängigkeiten geprüft (npm audit, Pflegezustand)?

Übung 36: XSS erleben und beheben

- **Öffne das Projekt 36_sicherheit (Lastgenerator aus Ü35 aus; Notiz kommt aus dem CustomerService)**
- **1. Customer erhält note?: string; Kunde 1: wichtig <img src=x onerror="alert('XSS')">**
- **2. Anzeige per Interpolation und per [innerHTML]: Ergebnis im Browser (Elements) prüfen**
- **3. Gegenbeispiel mit bypassSecurityTrustHtml: Was passiert? Wann wäre das akzeptabel?**
- **4. Login: returnUrl nur zulassen, wenn der Pfad mit /customer beginnt und nicht absolut ist**
  - Tipp: new URL(value, window.location.origin) und origin sowie pathname prüfen
- **5. loggingInterceptor darf Header wie Authorization nie ausgeben (Test oder Review-Kommentar)**
- **6. Gehe die Review-Checkliste für die ganze App durch und dokumentiere Funde**
- **Musterlösung: 36_sicherheit (baut auf 35_performance auf)**

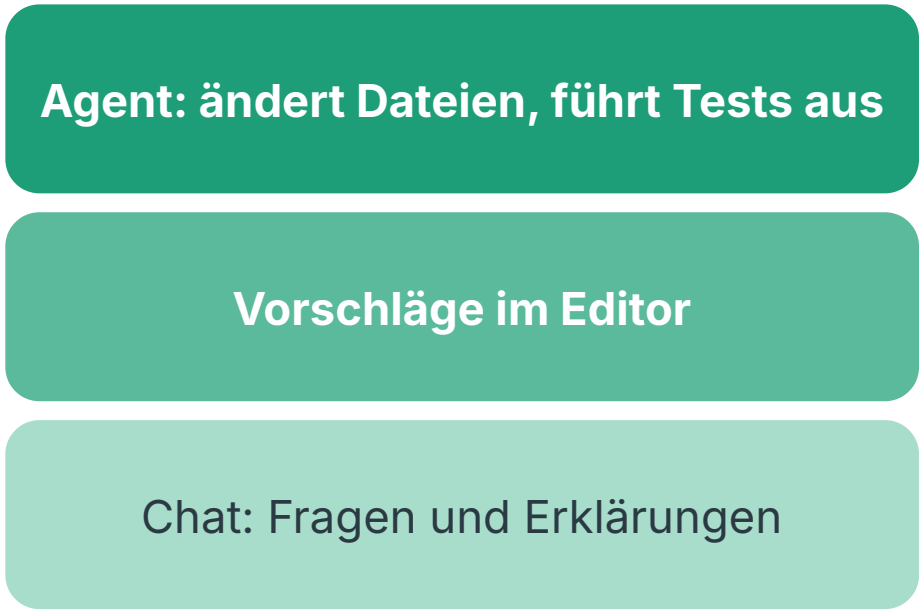
KI-gestützte Entwicklung

Wofür KI in der Entwicklung?

- Sehr gut geeignet: Gerüste, Standardcode, Umbenennungen
- Gut geeignet: fremden Code und Fehlermeldungen erklären lassen
- Gut, aber mit Review: Tests vorschlagen, Randfälle sammeln
- Gut, mit Tests: Umbauten in kleinen Schritten
- Nur als Diskussionspartner: Architekturentscheidungen, Fachlogik, Sicherheit
- Riskant: große Änderungen blind übernehmen
- KI ist ein schneller Assistent ohne Verantwortung: Die Verantwortung bleibt beim Entwickler

Formen der Unterstützung

- Chat: Fragen stellen, Code erklären lassen
- Vorschläge in der Entwicklungsumgebung: Ergänzung einzelner Zeilen oder Blöcke
- Agentische Werkzeuge: lesen das Projekt, ändern mehrere Dateien, führen Befehle oder Tests aus
- Je höher die Selbstständigkeit, desto höher der Prüfaufwand und der mögliche Schaden



Zusammenarbeit: Kontext ist alles

- Aufgabe klar und klein: „Erstelle eine Darstellungskomponente CustomerFilter mit input/output“
- Nicht: „Baue die App um“
- Projektregeln mitgeben: eine Datei im Repository mit Stil und Verboten (Name je nach Werkzeug)
- Beispiele zeigen: „Mache es analog zu CustomerListComponent“
- Aktuelle Angular-Konventionen angeben; Hinweise auf angular.dev prüfen

Beispiel: Projektregeln für Angular

Projektregeln
- Angular 21, TypeScript strict, zoneless, OnPush.
- Standalone-Komponenten (kein NgModule), inject() statt Konstruktor-Injection.
- Control Flow @if/@for (mit track auf ID), keine *ngIf/*ngFor.
- input()/output()/model(), keine Dekoratoren @Input/@Output.
- Zustand: signal/computed; effect nur für Nebenwirkungen außerhalb von Angular.
- Kein any, kein bypassSecurityTrust* ohne Rückfrage.
- Tests mit Vitest; neue Logik immer mit Tests.

Vorschläge bewerten: Review-Checkliste

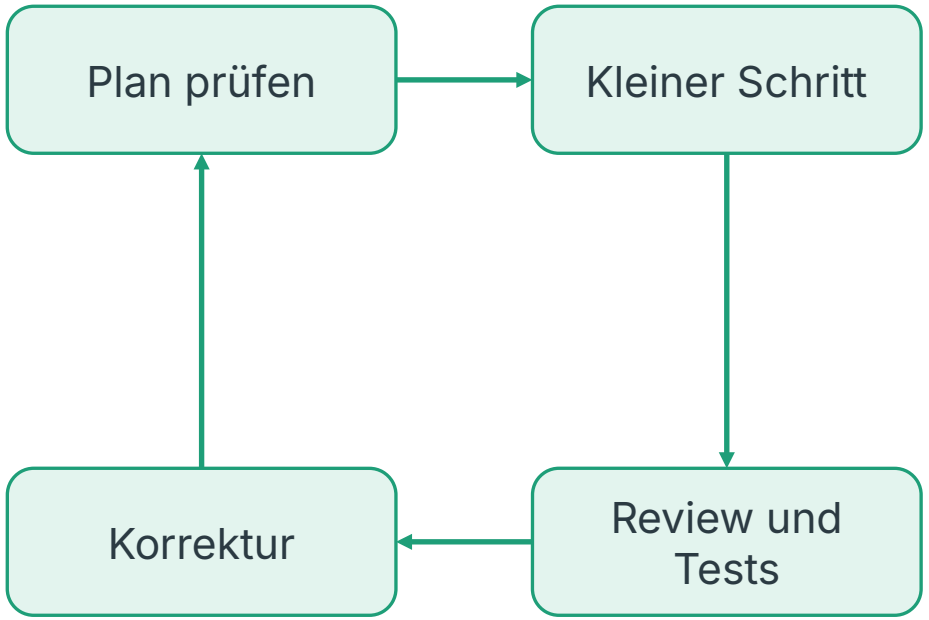
- 1. Läuft es? Build, Lint, Tests
- 2. Passt es zur Architektur? Abhängigkeitsrichtung, Feature-Struktur, keine neue Parallelwelt
- 3. Ist es modern? Keine veralteten APIs (*ngIf, @Input, NgModule), keine erfundenen Methoden
- 4. Ist es sicher? innerHTML, bypassSecurityTrust*, Tokens, Logging
- 5. Ist es reaktiv korrekt? Effects missbraucht? Werte verändert statt neu gesetzt?
- 6. Ist es getestet? Prüfen die Tests Verhalten oder nur toBeTruthy?
- 7. Verstehe ich es? Was ich nicht erklären kann, committe ich nicht

Typische KI-Fehler bei Angular

- Veraltete Muster: *ngFor, @Input(), Konstruktor-Injection, NgModule
- Erfundene oder falsche APIs: Methoden oder Optionen, die es nicht gibt
- effect für alles: Zustand synchronisieren, statt computed zu nutzen
- any und Typ-Casts: Fehler verschwinden, statt gelöst zu werden
- Unnötige Komplexität: zusätzliche Abstraktionen, Wrapper, Hilfsklassen
- Sicherheitsblindheit: innerHTML, Tokens im localStorage, ungeprüfte Weiterleitungen
- Tests, die nichts prüfen: toBeTruthy, Spiegelung der Mocks
- Stilbruch: andere Namens- und Ordnerkonventionen

Mehrrunden-Zusammenarbeit

- Ablauf: Ziel, Plan prüfen, kleiner Schritt, Review und Tests, Korrektur, nächste Runde
- Erst Plan, dann Code: Schritte besprechen, bevor Dateien geändert werden
- Kleine, überprüfbare Schritte, ein Commit pro Schritt
- Ergebnisse zurückspiegeln: Fehlermeldungen, Testausgaben und Befunde konkret nennen
- Kontext pflegen: Bei langen Sitzungen Regeln wiederholen oder neu mit Zusammenfassung starten
- Bei Abweichung vom Plan stoppen und korrigieren



Gute Prompts: ein Muster

Kontext: Angular 21 App, Feature customer, siehe customer-list.component.ts
Aufgabe: Extrahiere die Filter-UI in eine Darstellungskomponente
CustomerFilterComponent
Vorgaben: input()/output(), OnPush, keine Services, kein neues Paket
Abnahme: Verhalten unverändert; Test für den Output vorhanden; ng build läuft durch
Format: Zeige zuerst den Plan, dann die Änderungen als Diff

// Merkregel: Kontext, Aufgabe, Vorgaben, Abnahmekriterien, Format

Grenzen und Fehlannahmen (1/2)

- Annahme: „Die KI weiß, wie unser Projekt aussieht.“
  - Realität: Nur das, was im Kontext steht
- Annahme: „Wenn sie selbstsicher klingt, stimmt es.“
  - Realität: Sie klingt immer überzeugend, auch bei Unsinn (Halluzination)
- Annahme: „Es kompiliert, also ist es richtig.“
  - Realität: Falsches Verhalten, Sicherheitslücken und Performance-Probleme bleiben möglich

Grenzen und Fehlannahmen (2/2)

- Annahme: „Tests von der KI sind ein Qualitätsnachweis.“
  - Realität: Nur wenn sie Verhalten wirklich prüfen
- Annahme: „Die KI kennt die aktuelle Angular-Version.“
  - Realität: Ihr Wissensstand kann veraltet sein, deshalb die Dokumentation prüfen
- Annahme: „Die KI erspart mir das Lernen.“
  - Realität: Ohne Wissen kann ich Ergebnisse nicht beurteilen

Datenschutz, Lizenzen, Verantwortung

- Keine Geheimnisse, Zugangsdaten oder Kundendaten in Prompts oder Kontextdateien
- Firmenrichtlinien beachten: erlaubte Werkzeuge, Datenablage, Nachweispflichten
- Lizenzen: Herkunft generierten Codes kann unklar sein, bei längeren Übernahmen prüfen
- Nachvollziehbarkeit: Verantwortlich ist der Entwickler, der den Code committet
- Vorgeschlagene Pakete prüfen (Existenz, Pflege, Sicherheit): Namen werden manchmal erfunden

Was bleibt Entwicklerkompetenz?

- Anforderungen verstehen und präzisieren
- Architektur und Abwägungen (Aufteilung, Reaktivität, Zustandsverwaltung)
- Code lesen und beurteilen
- Testen, Messen, Absichern (Tests, Performance, Sicherheit)
- Verantwortung im Team und gegenüber den Nutzern

Übung 37: KI-Ergebnis reviewen

- **Öffne das Projekt 37_ki_review**
- Auftrag: „Ergänze Kunden-Export als CSV und eine Komponente CustomerExportButton“
  - Ohne erlaubtes Werkzeug: Datei ki-vorschlag.md aus der Aufgabe verwenden
- **1. Prüfe das Ergebnis mit der Review-Checkliste und dokumentiere mindestens 5 Befunde**
  - Absichtlich enthalten: *ngIf, any, effect schreibt Signal, innerHTML, toBeTruthy
- **2. Gib die Befunde in einem zweiten Prompt zurück („Behebe: …“) und prüfe erneut**
- **3. Auswertung: Welche Fehler hat die KI nicht selbst erkannt? Welche kamen neu hinzu?**
- **Musterlösung: 37_ki_review (baut auf 36_sicherheit auf)**

Übung 38: Abschlussprojekt Kunden-Dashboard

- **Öffne das Projekt 38_abschluss; Kleingruppen, KI-Unterstützung nach Wahl**
- Neues Feature: Seite /dashboard mit Kennzahlen
  - Anzahl Kunden, aktive Kunden der letzten 7 Tage, Kunden pro Stadt
- **1. Plan mit 5 Schritten schreiben und im Team abstimmen**
- **2. In kleinen Schritten umsetzen, je Schritt ein Commit**
- **3. Anforderungen: computed für Kennzahlen, CustomerStore, Lazy Route, mindestens 3 Tests**
  - Keine unsicheren Muster; Selbst-Review mit den Checklisten
- **4. Kurzvorstellung (5 Minuten je Gruppe): Was war gut, was musste korrigiert werden?**
- **Musterlösung: 38_abschluss (baut auf 37_ki_review auf)**

Rückblick-Übung: Technologieentscheidung

- Zeitpunkt: am Kursende, etwa 10 Minuten – jetzt ist das nötige Wissen vorhanden
- Szenario: Ein Versicherer plant ein Kundenportal
  - 8 Entwickler, 10 Jahre Laufzeit, 3 bestehende Backend-Teams
- Fragen: Passt Angular? Welche Teile aus dem Kurs sprechen dafür, welche Risiken gibt es?
- Ergebnis: je Gruppe (auch zu zweit oder allein) 3 Argumente dafür und 3 Risiken
- Es gibt keine richtige Antwort
- Ziel: Entscheidungskriterien ausdrücklich benennen, z. B. Skills, Lebensdauer, Teamgröße

Legacy: Angular Module (NgModule)

Was ist ein Modul

- Angular-spezifischer Code muss organisiert werden
  - Gleiche Grundgedanken und Aufbau wie bei ES6 Modulen
  - Kapselung & Wiederverwendbarkeit
  - Leichtes Einbinden von Bibliotheken
  - Genaue Steuerung was wo verwendet wird
  - Kompromiss zwischen
    "alles immer angeben müssen" (Angular < 2.0.0-rc.5) und
    "einfach rein schmeißen" (AngularJS 1)

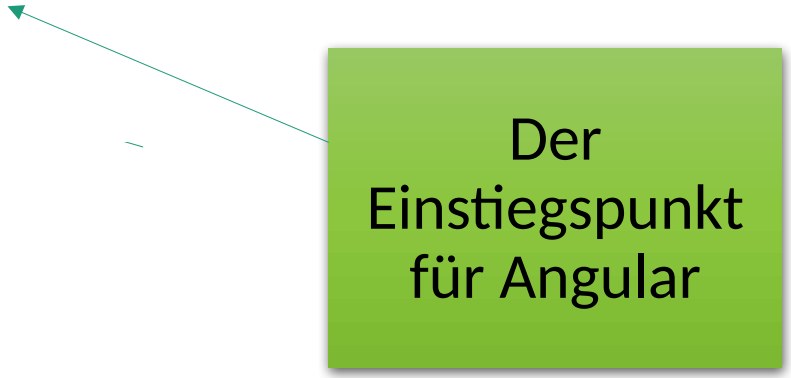
Übersicht der Dateien eines Moduls

- *src/index.html*
  - Die Hauptseite
- *src/app/app.module.ts*
  - Definition des Moduls
- *src/app/app.component.ts*
  - Definition des dynamischen Anteils einer Komponente mit TypeScript
- *src/app/app.component.html*
  - Definition des HTML-Templates einer Komponente
- *src/app/app.component.css*
  - Stylesheet der Komponente
- *src/app/app.component.spec.ts*
  - Definition von Karma-Tests

Referenz auf Module in Index-Seite

index.html

```
1   <!doctype html>
2   <html lang="en">
3   <head>
4     <meta charset="utf-8">
5     <title>org.Javacream.Training.Angular</title>
6     <base href="/">
7
8     <meta name="viewport" content="width=device-width, initial-scale=1">
9     <link rel="icon" type="image/x-icon" href="favicon.ico">
10  </head>
11  <body>
12    <app-root></app-root>
13  </body>
14  </html>
15
```



Component-Annotation

- Eine Component ist eine TypeScript-Klasse mit der `@Component`-Annotation
  - Annotationen enthalten Metadaten, die der Klasse zugeordnet werden
  - Diese müssen von einem Framework interpretiert werden
- Annotationen sind damit so etwas wie statische Eigenschaften einer Klasse
  - Allerdings sind diese syntaktisch nicht Bestandteil der Klassendeklaration
  - Damit ergibt sich eine sauber Trennung
- Eine typische Component definiert
  - Den Selector als Bezug zum Template
  - Die Lokation des Templates
- Vollständige Liste unter
  - *`https://angular.io/api/core/Component`*

Aufgabe der Component

- Die Eigenschaften der Component-Klasse dienen als Schnittstelle zum HTML-Code
  - Werte werden über Interpolation oder Direktiven gebunden
  - Funktionen dienen als Event-Handler
- Bei Bedarf implementiert die Component-Klasse Lifecycle-Funktionen
  - `ngOnChanges`
  - `ngOnInit`
  - *`https://angular.io/guide/lifecycle-hooks`*

Angular-Module: Die Component

```
import {ApplicationRef, Component, OnDestroy, OnInit} from "@angular/core";

@Component({
   selector: “app-root",
   templateUrl: "./app.component.html",
   styleUrls: ["./app.component.scss"],
   animations: [fadeAnimation]
})
export class AppComponent implements OnInit, OnDestroy {

   public title = "theFrontend";
```

TypeScript-Import der Angular-Core-Bibliothek

Referenz auf den Einstiegspunkt

Referenz auf Template und CSS

Definition einer Variablen und Wert-Zuweisung

Angular-Module: Das HTML-Template

app.component.html

```
1   <!--The content below is only a placeholder and can be replaced.-->
2   <div style="text-align:center">
3     <h1>
4       Welcome to the great {{ title }}!
5     </h1>
6     <img width="300" alt="Angular Logo"
•     src="
•     3IDQwLjl6IiAvPgogIDwvc3ZnPg==">
7   </div>
8   <h2>Here are some links to help you start: </h2>
9   <ul>
10    <li>
11      <h2><a target="_blank" rel="noopener" href="https://angular.io/t
12    </li>
13    <li>
14      <h2><a target="_blank" rel="noopener" href="https://github.com/a
15    </li>
16    <li>
17      <h2><a target="_blank" rel="noopener" href="https://blog.angular
18    </li>
19  </ul>
```

Referenzieren der Component-Variable mit Interpolation

Angular-Module: Module-Deklaration

app.module.ts

```
1   import { BrowserModule } from '@angular/platform-browser';
2   import { NgModule } from '@angular/core';
3
4
5   import { AppComponent } from './app.component';
6
7
8   @NgModule({
9     declarations: [
10      AppComponent
11    ],
12    imports: [
13      BrowserModule
14    ],
15    providers: [],
16    bootstrap: [AppComponent]
17  })
18  export class AppModule { }
19
```

Import von Angular-Core und dem Browser-Support

Import der eigenen Komponente

Definition aller Eigenschaften des Moduls

Durch den Export wird das Modul aktiviert

Ablauf bis hier her

- Ablauf bis hier
  - Aufruf bootstrapModule mit Modul (AppModule)
  - Modul ist Referenz auf eine Klasse
  - Klasse hat einen Decorator
  - Modul-Decorator enthält Komponenten
- Woher weiß Angular welche Komponente für das Bootstrapping verwendet werden soll?

Angular-Module: Module-Deklaration

app.module.ts

```
1   import { BrowserModule } from '@angular/platform-browser';
2   import { NgModule } from '@angular/core';
3
4
5   import { AppComponent } from './app.component';
6
7
8   @NgModule({
9     declarations: [
10      AppComponent
11    ],
12    imports: [
13      BrowserModule
14    ],
15    providers: [],
16    bootstrap: [AppComponent]
17  })
18  export class AppModule { }
19
```

„Main“ Component

Kompletter Ablauf

- Ablauf insgesamt
  - Aufruf bootstrapModule mit Modul (AppModule)
  - Modul ist Referenz auf eine Klasse
  - Klasse hat einen Decorator
  - Modul-Decorator hat Feld bootstrap
  - Feld bootstrap hält Referenz auf Komponenten Klasse (AppComponent)
  - Komponenten-Klasse hat Decorator
  - Komponenten-Decorator hat Feld selector
  - Selector wird in DOM aus index.html gesucht
  - Komponente wird instanziiert

Deklaration, Imports und Exports

```
@NgModule({
  declarations: [ ReleaseListComponent, ReleaseCardComponent ],
  imports: [ BrowserModule ],
  exports: [ ReleaseListComponent ]
})
export class ReleaseModule { }
```

Module generieren

- Angular CLI legt neue Module an
  - Module-Klasse + Annotation
  - Module-Komponente
- ng generate module *name*
- Wird *NICHT* automatisch am nächsthöherer gelegenen Modul registriert selbst machen

Legacy: DI Konfig

- Woher weiß Angular was zu injizieren ist?
  - Muss konfiguriert werden: *Providers*
  - Injector → Provider → Instance
- Konfiguration an Modulen

```
@NgModule({
providers: [ ] /* array of providers */
})
export class TrackModule { }
```

- Wie sieht ein Provider aus?

Legacy: Wie geht DI in Angular

- Direkte Abhängigkeit
  - einfach nur die Klasse angeben: [ConsoleLogger]
  - Kurz für [{ provide: ConsoleLogger, useClass: ConsoleLogger}]
- Allgemein: Provider-Objekt
  - provide: Unter welchem Namen steht es zur Verfügung → Klasse
  - Ein Rezept, wie soll es erzeugt werden soll →
    useClass, useValue, useFactory oder useExisting

```
@NgModule({
providers: [
        { provide: ConsoleLogger, useClass: ConsoleLogger }, // same as shortcut
        { provide: Logger, useExisting: ConsoleLogger } // different token, same instance
        ]
})
export class TrackModule { }
```

Legacy: Deklaration an Modulen

```
@NgModule({
declarations: [ HasLikesPipe ],
exports: [ HasLikesPipe ]
})
export class AppModule { }
```

Legacy: Routenkonfiguration

- Wird verwendet um Provider für Router Service zu erzeugen
- Provider wird von RouterModule bereitgestellt
  - RouterModule muss in AppModule importiert werden
  - forRoot darf nur einmal im AppModule verwendet werden
  - forChild nur bei Lazy-Loading

Legacy: Beispiel: Routen-Definition

```
const appRoutes: Routes = [
  { path: 'path1', component: Component1},
  { path: ' ',  redirectTo: '/index', pathMatch: 'full'},
  { path: '**', component: PageNotFoundComponent }
];
@NgModule({
  imports: [
    RouterModule.forRoot(
      appRoutes,
      { enableTracing: true } // <-- debugging purposes only
    )
],
  ...
})
export class AppModule { }
```

Legacy: Location-Strategy

- LocationStrategy festlegen
  - PathLocationStrategy **(default)**
    - Bessere SEO Indizierung
  - HashLocationStrategy: *default AngularJS 1*
    - Simpler
- Verwendung von **PathLocationStrategy**
  - <base href="."> als erstes Element im head der index.html

```
import { RouterModule } from '@angular/router';
import { routes } from './app/app.routes';

@NgModule({ imports: [
RouterModule.forRoot(routes, { useHash: true })
]})
export class AppModule { }
```

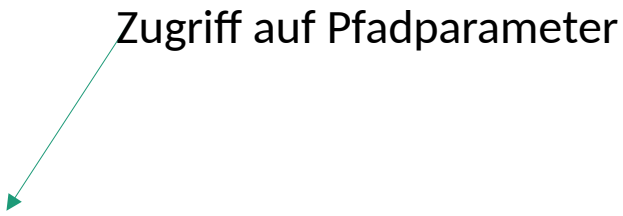
Legacy: ActivatedRoute und params.subscribe

```
import { ActivatedRoute } from "@angular/router";
export class ArtistDetailsComponent implements OnInit, OnDestroy {
    private subscription: Subscription;

    constructor(private route: ActivatedRoute) {}

    ngOnInit() {
        this.subscription = this.route.params
        .subscribe(params => console.log(params['id']));
    }

    ngOnDestroy() { this.subscription.unsubscribe(); }
    }
```



Legacy: Der Service HttpClient

- HttpClient ist ein Service
  - und steht damit via Dependency Injection zur Verfügung
  - **HttpModule in AppModule importieren**
  - XHRBackend führt Requests aus
- Das API
  - umfasst die Standard-http-Methoden
  - ist asynchron konzipiert

```
http.get('/endpoint').subscribe(data => {
      // Read the result field from the JSON response.
      this.results = data['jsonAttribute'];
    });
```

Übung 30: Legacy – Angular Module

Öffne das Projekt 30_modules_legacy (NgModule-basierte Variante)
Backend starten (server: npm install, node server.js)
App starten mit ng serve --proxy-config dev.proxy.json
Analysiere den Aufbau
main.ts: platformBrowserDynamic().bootstrapModule(AppModule)
AppModule: declarations, imports, providers, bootstrap
TrainingsModule und AppRoutingModule (RouterModule.forRoot)
Erweitere das Modul
Erzeuge eine Komponente per ng g c trainings/training-info --standalone=false und deklariere sie im TrainingsModule
Binde eine Standalone-Komponente über imports: [] in ein NgModule ein (Mischbetrieb)
Optional: Migriere das Projekt
ng generate @angular/core:standalone ausführen und mit 23_communication vergleichen

Wahlthema: Reactive Forms

Formulare Bauen mit ReactiveFormsModule

- Model-Klassen um Formular zu beschreiben
  - FormGroup und FormControl
  - Gruppen bilden Baum mit Feldern als Blätter
  - Formular selbst ist eine FormGroup
  - Seit Angular 14 streng typisiert: FormGroup<{ name: FormControl<string> }>

HowTo

```ts
import { FormGroup, FormControl, ReactiveFormsModule } from "@angular/forms";
@Component({ imports: [ReactiveFormsModule], ... })
export class ReleaseFormComponent implements OnInit {
  release = input.required<Release>();
  form = new FormGroup({
    name: new FormControl('', { nonNullable: true }),
    date: new FormControl<string | null>(null)
  });
  ngOnInit() {
    this.form.patchValue({ name: this.release().name });
  }
}
```

```html
<form [formGroup]="form" (ngSubmit)="onSubmit()">
  <label>Name:</label>
  <input type="text"
  [formControl]="form.controls.name">
  <label>Date:</label>
  <input type="date" formControlName="date">
</form>
```

Wahlübung Z1: Reactive Form mit FormBuilder

- Bereite die Anwendung für die Verwendung der Forms-API vor
  - Importiere ReactiveFormsModule im imports-Array von CustomerDetailsComponent
- Implementiere in CustomerDetailsComponent ein Formular
  - Verwende die typisierte Forms-API (FormGroup/FormControl oder inject(FormBuilder).nonNullable) und speichere das Formular in einem Feld
  - Erstelle für jedes Feld in Customer (außer ID und image) ein FormControl
  - Initialisiere die Felder mit den vom Service geholten Daten
  - Implementiere ein passendes Template mit einem Label und passendem Input pro FormControl
  - Tip: *<input type="date">* für lastOrderDate, Daten-Mapping erforderlich *customer.lastOrderDate.toIsoString().substr(0, 10)* und *new Date(inputValue)*
- Implementiere das Speichern der geänderten Daten
  - Verwende das ngSubmit-Event am Formular und einen Submit-Button zum Speichern
  - Übernehme die geänderten Daten in das Domain-Objekt (Tip: Object.assign)

Verwendung

```
this.form = new FormGroup({
        name: new FormControl('', Validators.required),
        city: new FormControl('', Validators.maxLength(10)),
        zip: new FormControl('', Validators.pattern('[0-9]{5}')),
        street: new FormControl('', Validators.compose([
                Validators.required, Validators.minLength(3)
]))
```

Validierungsfehler

- Zustand pro Feld / Gruppe / Formular
  - untouched, touched
  - pristine, dirty
  - valid, errors
  - Value
- Zusammenspiel mit CSS Klassen
  - Angular setzt CSS-Klassen an Formular und Feldern
  - ng-valid, ng-invalid, ng-pristine, ng-dirty

Zugriff auf Fehlerzustand

```
<div class="form-group"
        [class.has-error]="form.controls.email.dirty &&  !
form.controls.email.valid"
        [class.has-warning]="form.controls.email.pristine && !
form.controls.email.valid">
        <input type="email" class="form-control"
formControlName="email">
</div>
```

ErrorsMap

```
<div [hidden]="!form.controls.password.dirty">
    <div [hidden]="!form.controls.password.errors?.required">
    Password ist Pflichtfeld
    </div>
    <div [hidden]="!form.controls.password.errors?.complex">
    Password muss aus 8 Zeichen und mind. 1 Sonderzeichen und
    Großbuchstaben bestehen.
    </div>
    </div>
```

Wahlübung Z1: Validierung (Reactive Forms)

- Validiere in CustomerDetailComponent
  - First- und lastname required
  - Zip als string aber nur zahlen
- Gib für jedes invalide Feld eine Fehlermeldung aus
- Verhindere das Absenden des invaliden Formulares
  - Tip: [disabled] an Submit-Button

Anhang



node.js

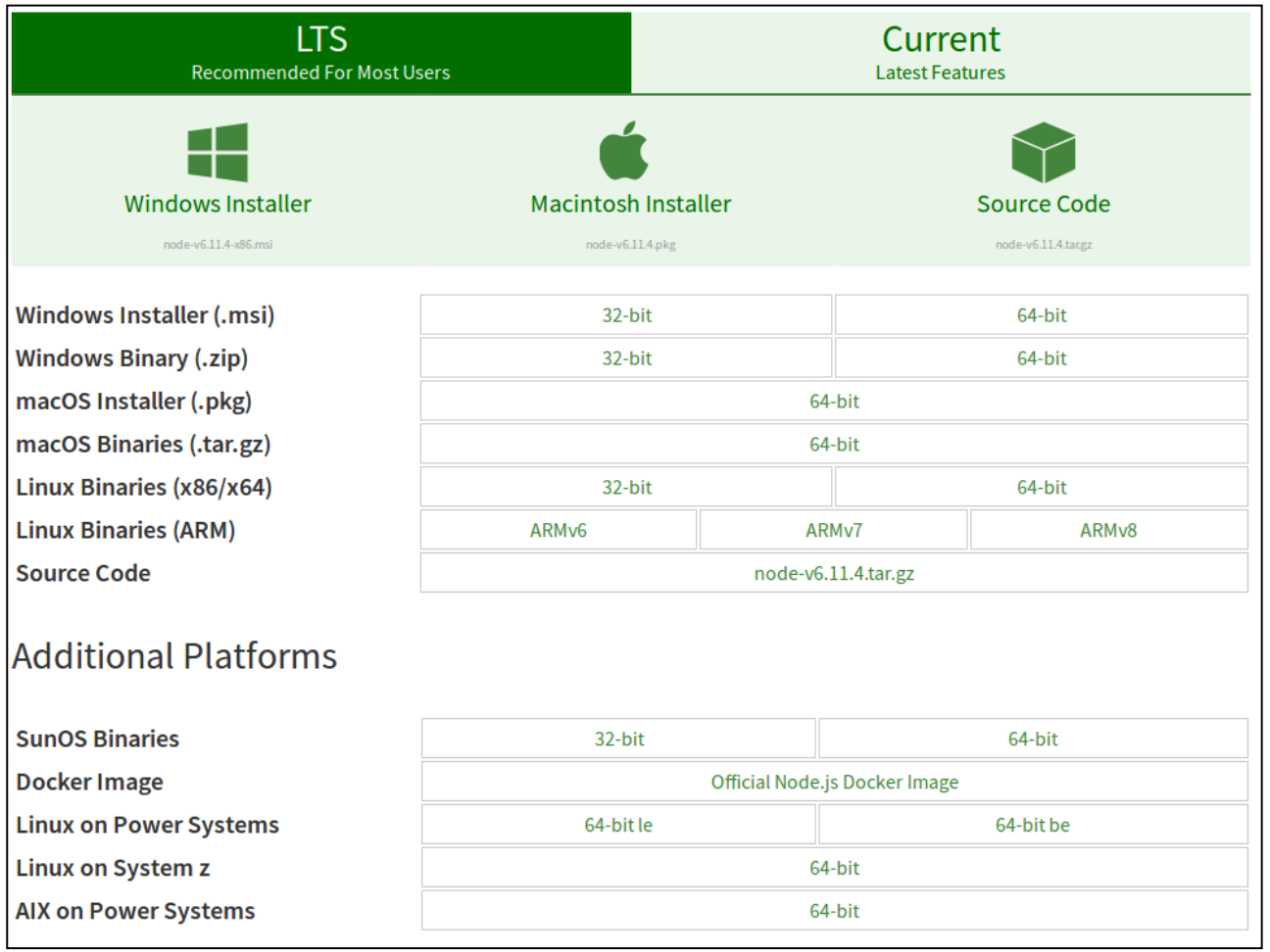
Was ist node.js?

- node.js ist ein Interpreter für Server-seitiges JavaScript
  - Auf Grundlagen der Google V8-Engine
- Mit node.js können damit keine Browser-Anwendungen betrieben werden
  - Keine UI, Keine User-Events
  - Kein Html-Dokument und damit kein DOM
  - Kein Browser-API
    - Window
    - Historie
    - …
- Dafür stellt node.js eigene Bibliotheken zur Verfügung
  - Dateizugriff
  - Multithreading
  - Networking
  - …
  - https://nodejs.org/dist/latest-v8.x/docs/api/

Beispiel: Ein kompletter http-Server

```
var http = require('http');
var fs = require('fs');
http.createServer(function handler(req, res) {
	var url = req.url;
	if (url.match(/.html/)) {
		res.writeHead(200, {
			'Content-Type' : 'text/html'
		});
	} else if …
	var filename = "./static-content" + req.url;
	fs.createReadStream(filename).pipe(res);
}).listen(6061, '127.0.0.1');
```

Installation: node.js



LTS
Recommended For Most Users

Current
Latest Features

Windows Installer
node-v6.11.4-x86.msi

Macintosh Installer
node-v6.11.4.pkg

Source Code
node-v6.11.4.tar.gz

Windows Installer (.msi)	32-bit	64-bit	
Windows Binary (.zip)	32-bit	64-bit	
macOS Installer (.pkg)	64-bit		
macOS Binaries (.tar.gz)	64-bit		
Linux Binaries (x86/x64)	32-bit	64-bit	
Linux Binaries (ARM)	ARMv6	ARMv7	ARMv8
Source Code	node-v6.11.4.tar.gz		

Additional Platforms

SunOS Binaries	32-bit	64-bit
Docker Image	Official Node.js Docker Image	
Linux on Power Systems	64-bit le	64-bit be
Linux on System z	64-bit	
AIX on Power Systems	64-bit	

Testen der Installation

```
node –v
```

- Ausgabe der Versionsnummer

```
node
```

- Starten der REPL zur Eingabe von JavaScript-Befehlen

```
node programm.js
```

- Ausführen der Skript-Datei *programm.js*

Node und Browser-basierte Anwendungen

- Obwohl node.js nicht im Browser ausgeführt wird, wird es trotzdem gerne im Rahmen der Software-Entwicklung genutzt
- Hierzu wird node als Web Server eingesetzt, der die JavaScript-Dateien sowie die statischen Ressourcen (HTML, CSS, …) zum Browser sendet
  - Mit Hilfe eines Browser-Sync-Frameworks triggern Änderungen von JavaScript-Dateien auf Server-Seite einen Browser-Refresh
    - `https://www.browsersync.io/`
    - Damit werden Änderungen ohne weitere Benutzer-Interaktion sofort angezeigt
    - Für eine agile Software-Entwicklung natürlich äußerst praktisch

npm – Der Node Package Manager



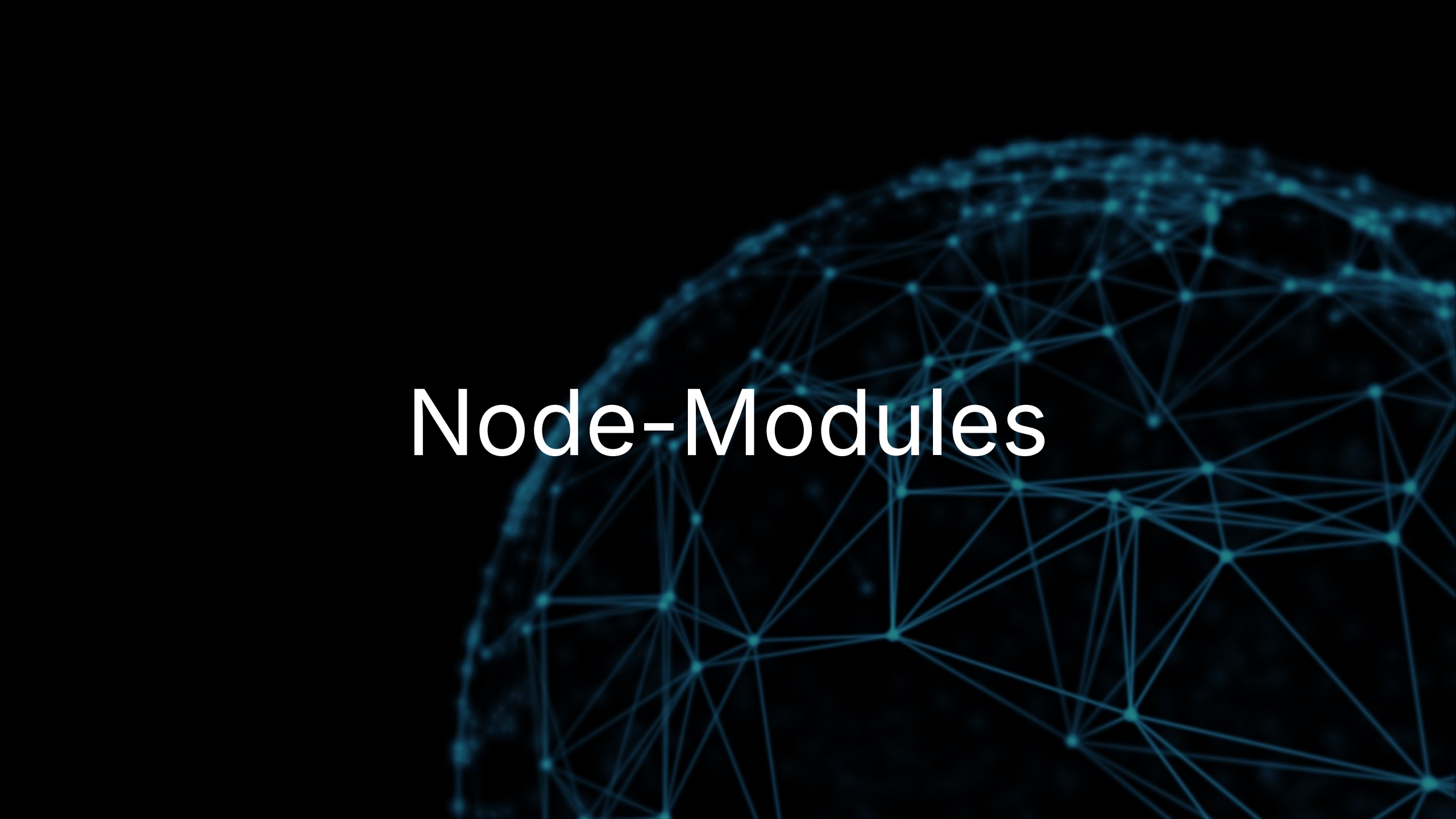
Was ist npm?

- Primär ein Packaging Manager
- `npm` ist Bestandteil der `node`-Installation
  - `npm –v`
- Die offizielle npm Registry liegt im Internet
  - https://docs.npmjs.com/misc/registry
  - Im Wesentlichen eine CouchDB
  - Laden der Software durch RESTful Aufrufe
  - Die `npm`-Registry ist aktuell die größte Sammlung von Software
- Unternehmens-interne oder private Registries können angemietet werden

npm Kommandos

- `npm` wird über die Kommandozeile angesprochen
  - eine grafische Oberfläche wird als separates Modul zur Verfügung gestellt
- Hilfesystem
  - `npm –h`
  - `npm <command> -h`
  - https://docs.npmjs.com/

Node-Modules



Node Modules

- Jede via `npm` geladene Bibliothek wird als Node-Module konzipiert
- Jedes Modul besitzt
  - Eine Informationsdatei, die *`package.json`*, die das Projekt zusätzlich beschreibt
  - Abhängige Bibliotheken im Unterverzeichnis *`node_modules`*
    - Diese sind selbst ebenfalls Node-Module
  - Einen Entry-Point, in dem der Module-Entwickler das Fachobjekt seines Moduls erzeugt und exportiert
    - Dazu wird dem `module`-Objekt die Eigenschaft `exports` gesetzt
  - Zur Benutzung eines Moduls innerhalb eines Scripts dient der Node-Befehl `require`
    - Der Rückgabewert von `require` ist das vom Modul erzeugte und exportierte Fachobjekt

Die package.json

- Enthält die Projektinformation im JSON-Format
- Die Datei enthält
  - Den Projektnamen
  - Die aktuelle Versionsnummer
  - Meta-Informationen wie Autor, Schlüsselwörter, Lizenz
  - Dependencies
  - Ein `scripts`-Objekt mit ausführbaren Befehlen
    - Diese können mit `npm run <script>` ausgeführt werden

Initialisierung eines Projekts

- Jedes `npm`-basierte Projekt ist ein neues Node-Module
- Initialisierung mit `npm init`
  - Dabei werden interaktiv die Informationen abgefragt, die zur Erstellung der initialen *`package.json`* benötigt werden

Projektstruktur

project-root

package.json

src

- script1
- script2
- …

node-modules

- module1
- module2
- …

Beispiel: Ein einfaches Projekt

```
{
  "name": "npm-sample",
  "version": "1.0.0",
  "description": "a simple training project",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [
    "training"
  ],
  "author": "theAuthor",
  "license": "ISC"
}
```

Beispiel: Ein einfaches Node-Module

- Datei *index.js*

```
module.exports = {
      log: function(){
            console.log('Hello')
          }
}
```

- In der REPL

```
var training = require('./index.js')
training.log()
```

Installieren von Abhängigkeiten

- Abhängigkeiten werden mit `npm install` von einer npm-Registry geladen
  - Ohne weitere Konfiguration wird dazu die Standard-Registry benutzt
    - Damit ist eine Internet-Verbindung notwendig
  - Es können aber auch Unternehmens-interne Repository-Server benutzt werden
    - z.B. Nexus
- Rechner-Registry
  - Die Abhängigkeiten werden auf dem Rechner abgelegt
    - Ab jetzt ist damit keine Internet-Verbindung mehr nötig
  - Orte:
    - lokale Ablage in einem Unterverzeichnis namens *`node-modules`*
      - Empfohlenes Standard-Verfahren zur Installation von Dependencies für eigene Software-Projekte
    - globale Ablage
      - Empfohlenes Standard-Verfahren zur Installation von allgemein verwendbaren Werkzeugen